

Componentes/GiftList/STBGiftList.prw

480 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STBGIFTLIST.CH"
4
5 #DEFINE POS_MMODO      1           //Modo de abertura
6 #DEFINE POS_MRET      2           //Modo de retorno
7 #DEFINE POS_MTPL      3           //Tipo da lista
8 #DEFINE POS_MNL       4           //Numero da lista (Filtro ME1)
9 #DEFINE POS_MONL      5           //Online
10 #DEFINE POS_MLNOM     6           //Entrar com nomes dos presenteadores
11 #DEFINE POS_MLENVM    7           //Enviar mensagem
12 #DEFINE POS_MLITAB    8           //Listar itens em aberto
13 #DEFINE POS_MORI      9           //Origem da lista (filtro ME1)
14 #DEFINE POS_MFILT     10          //Filtro (ME2)
15 #DEFINE POS_MMULT     11          //Multi-selecao
16 #DEFINE POS_MMTOD     12          //Marcar todos
17 #DEFINE POS_MQTDU     13          //Quantidade utilizada
18 #DEFINE POS_MLAQTD    14          //Alterar quantidade
19 #DEFINE POS_MAME      15          //Alterar modo de entrega
20 #DEFINE POS_MTPEVE    16          //Tipo de evento (filtro ME1)
21 #DEFINE POS_MSTAT     17          //Status da lista (filtro ME1)
22
23
24 //-----
25 /*{Protheus.doc} STBGLQyCr
26 Monta a String de Pesquisa, conforme a lista de seleção
27 @param      cCampo - campo manipulado
28 @param      cTipo - tipo do campo
29 @param      cCritério - critério
30 @param      cOpConj - opcao para pesquisa
31 @param      cOpDisj - opcao para pesquisa
32 @param      lFrmtTC - opcao para pesquisa
33 @return cRet - Expressão SQL de Busca
34 @sample
35 */
36 //-----
37 Function STBGLQyCr( cCampo ,cTipo,cCritério,cOpConj,;
38                   cOpDisj,lFrmtTC)
39
40 Local aAreaSX2      := SX2->(GetArea())           // area dos campos
41 Local cRet          := ""                        // retorno da funcao
42 Local ni            := 0                        // contador do FOR
43 Local nx            := 0                        // contador do FOR
44 Local aLstCri       := {}                      // array da lista
45 Local nPos          := 1                        // posicao do ascan
46 Local cCarac        := ""                      // criterios
47 Local cTMP          := ""                      // temporario
48 Local nTam          := 0                        // tamanho
49 Local cAlias        := ""                      // area de trabalho
50 Local aLstUNQ       := {}                      //Lista de campos de chave unica da tabela
51
52 Default cCampo      := ""                      // campo manipulado

```

```

53 Default cTipo      := ""           // tipo do campo
54 Default cCritério   := ""           // critério
55 Default cOpConj      := "+"         // opcao para pesquisa
56 Default cOpDisj      := "^"         // opcao para pesquisa
57 Default lFmtTC       := .F.         // opcao para pesquisa
58
59
60 STFMessage("STBGLQyCr", "STOP", STR0002) //"Data inválida!"
61
62 If Empty(cCampo) .OR. Empty(cCritério)
63     Return cRet
64 Endif
65 If Empty(cTipo)
66     If Empty(cTipo := GetSX3Cache(cCampo,"X3_TIPO"))
67         Return cRet
68     Endif
69 Endif
70 //Tamanho do campo
71 nTam      := GetSX3Cache(cCampo,"X3_TAMANHO")
72 cAlias     := GetSX3Cache(cCampo,"X3_ARQUIVO")
73 //Levantar campos da chave unica (que permitem que um campo chave pode ser pesquisado por intervalo)
74 If cTipo == "C"
75     DbSelectArea("SX2")
76     SX2->(DbSetOrder(1))
77     If SX2->(DbSeek(cAlias))
78         aLstUNQ := StrTokArr(FWX2Unico(cAlias),"+")
79     Endif
80 Endif
81 //Montar array de criterios : [1]CAMPO [2]VALOR [3]OPERADOR
82 aAdd(aLstCri,{cCampo,"",""})
83 For ni := 1 to Len(cCritério)
84     cCarac := Substr(cCritério,ni,1)
85     Do Case
86         Case cCarac == cOpConj
87             aTail(aLstCri[nPos]) := "AND"
88             aAdd(aLstCri,{cCampo,"",""})
89             nPos++
90         Case cCarac == cOpDisj
91             aTail(aLstCri[nPos]) := "OR"
92             aAdd(aLstCri,{cCampo,"",""})
93             nPos++
94         Otherwise
95             Do Case
96                 Case cTipo == "D"
97                     If !IsDigit(cCarac) .AND. !cCarac $ "/" "
98                         STFShowMessage("STBGLQyCr")
99                         RestArea(aAreaSX2)
100                        Return cRet
101                    Endif
102                    aLstCri[nPos][2] += cCarac
103                Otherwise
104                    aLstCri[nPos][2] += cCarac

```

```

105         EndCase
106     EndCase
107 Next ni
108 //Em caso de data, limitar a dois criterios (intervalo)
109 If cTipo == "D" .AND. Len(aLstCri) > 2
110     nx := 0
111     For ni := 3 to Len(aLstCri)
112         aDel(aLstCri,ni)
113         nx++
114     Next ni
115     aSize(aLstCri,Len(aLstCri) - nx)
116 Endif
117 //Caso existam mais de 02 criterios, descaracteriza um intervalo, zerar aLstUNQ
118 If cTipo == "C" .AND. Len(aLstCri) # 2 .AND. Len(aLstUNQ) > 0
119     aLstUNQ := Array(0)
120 Endif
121 //Montar a expressao
122 For ni := 1 to Len(aLstCri)
123     //Ajustar o tamanho do criterio de acordo com o tamanho do campo
124     If cTipo $ "C|M"
125         If lFrmtTC .AND. !Empty(nTam)
126             aLstCri[ni][2] := PadR(Substr(aLstCri[ni][2],1,nTam),nTam)
127         Else
128             If !Empty(nTam)
129                 aLstCri[ni][2] := Substr(RTrim(aLstCri[ni][2]),1,nTam)
130             Else
131                 aLstCri[ni][2] := RTrim(aLstCri[ni][2])
132             Endif
133         Endif
134     Endif
135     Do Case
136         Case cTipo $ "C|M"
137             If !Empty(aLstCri[ni])
138                 If ni > 1
139                     If Empty(aTail(aLstCri[ni - 1]))
140                         aTail(aLstCri[ni - 1]) := "+"
141                         cTMP += " AND "
142                     Else
143                         cTMP += " " + aTail(aLstCri[ni - 1]) + " "
144                     Endif
145                 Endif
146                 If Len(aLstUNQ) == 0 .OR. aScan(aLstUNQ,{|x| Upper(AllTrim(x)) == Upper(AllTrim(cCampo))}) == 0
147                     //Caso nao seja pesquisa de intervalo ou o campo nao faça parte da chave unica para pesquisa de intervalo
148                     If Len(aLstCri[ni][2]) == nTam
149                         cTMP += cAlias + "." + cCampo + " = '" + aLstCri[ni][2] + "'"
150                     Else
151                         cTMP += cAlias + "." + cCampo + " LIKE '%" + aLstCri[ni][2] + "%'"
152                     Endif
153                 Else
154                     //Caso seja pesquisa de intervalo
155                     If ni == 1
156                         If AllTrim(aTail(aLstCri[ni])) == "AND"

```

```

157         cTMP += cAlias + "." + cCampo + " BETWEEN '" + PadR(aLstCri[ni][2],nTam) + "'"
158     Else
159         cTMP += cAlias + "." + cCampo + " = '" + PadR(aLstCri[ni][2],nTam) + "'"
160     Endif
161 Else
162     If AllTrim(aTail(aLstCri[ni - 1])) == "AND"
163         cTMP += "'" + PadR(aLstCri[ni][2],nTam) + "'"
164     Else
165         cTMP += cAlias + "." + cCampo + " = '" + PadR(aLstCri[ni][2],nTam) + "'"
166     Endif
167 Endif
168 Endif
169 Endif
170 Case cTipo == "D"
171     If Empty(CtoD(aLstCri[ni][2]))
172
173         STFS showMessage("STBGLQyCr")
174         RestArea(aAreaSX2)
175         Return cRet
176     Endif
177     If !Empty(aLstCri[ni][2])
178         If ni > 1
179             If Empty(aTail(aLstCri[ni - 1]))
180                 aTail(aLstCri[ni - 1]) := "+"
181                 cTMP += " AND "
182             Else
183                 cTMP += " " + aTail(aLstCri[ni - 1]) + " "
184             Endif
185         Endif
186         If Len(aLstCri) == 1
187             cTMP += cAlias + "." + cCampo + " = '" + DtoS(CtoD(aLstCri[ni][2])) + "'"
188         Else
189             If ni == 1
190                 If AllTrim(aTail(aLstCri[ni])) == "AND"
191                     cTMP += cAlias + "." + cCampo + " BETWEEN '" + DtoS(CtoD(aLstCri[ni][2])) + "'"
192                 Else
193                     cTMP += cAlias + "." + cCampo + " = '" + DtoS(CtoD(aLstCri[ni][2])) + "'"
194                 Endif
195             Else
196                 If AllTrim(aTail(aLstCri[ni - 1])) == "AND"
197                     cTMP += "'" + DtoS(CtoD(aLstCri[ni][2])) + "'"
198                 Else
199                     cTMP += cAlias + "." + cCampo + " = '" + DtoS(CtoD(aLstCri[ni][2])) + "'"
200                 Endif
201             Endif
202         Endif
203     Endif
204 Case cTipo == "N"
205     If !Empty(aLstCri[ni])
206         //Validar conteudo
207         lOk := .T.
208         For nx := 1 to Len(aLstCri[ni])

```

```

209         If !Substr(aLstCri[ni],ni,1) $ "0123456789,.-"
210             lOk := .F.
211         Endif
212     Next nx
213     //Se tudo ok, montar expressao
214     If lOk
215         If ni > 1
216             If Empty(aTail(aLstCri[ni - 1]))
217                 aTail(aLstCri[ni - 1]) := "+"
218                 cTMP += " AND "
219             Else
220                 cTMP += " " + aTail(aLstCri[ni - 1]) + " "
221             Endif
222         Endif
223         cTMP += cAlias + "." + cCampo + " = " + cValToChar(GetDToVal(aLstCri[ni][2]))
224     Endif
225 Endif
226 EndCase
227 Next ni
228 cRet := cTMP
229 RestArea(aAreaSX2)
230
231 Return cRet
232
233
234 //-----
235 /*{Protheus.doc} STBGLVLQtd
236 Funcao para validar digitacao de quantidade
237 @param      oGD      - Objeto do tipo GRID
238 @param      nQtde    - Quantidade digitada
239 @param      cExtra   - Aceita quantidades extras alem do determinado na list
240 @param      aHead    - Header do Grid
241 @param      aDados   - Dados do Grid
242 @param      lMessage - Exibe mensagem (default .f.)
243 @return     lRet     - Quantidade Válida
244 @sample
245 */
246 //-----
247 Function STBGLVLQtd(oGD,nQtde,cExtra, aHead, aDados, lMessage)
248
249 Local lRet           := .T.           //retorno da funcao
250 Local nQtdeDisp     := 0 //Quantidade Disponivel
251
252 Default oGD         := NIL           //objeto da lista
253 Default nQtde       := 0             //quantidade digitada
254 Default cExtra      := "2"          //extras na quantidade (não)
255 Default lMessage    := .F.
256
257 If oGD == Nil .OR. ValType(oGD) # "O" .OR. ValType(nQtde) # "N" .OR. Empty(nQtde) .AND. ValType(cExtra) # "C" .OR. !AllTrim(cExtra) $ "1|2"
258     Return !lRet
259 Endif
260

```

```

261 If oGD:nAt > 0
262   nQtdeDisp := STDGLRtCol(2,"DISP0",1,oGD:nAt,aHead,aDados, .f., .f.)
263   //Se a quantidade estiver a maior e a lista nao permitir quantidades extras, rejeitar
264
265   If ValType(nQtdeDisp) # "N"
266     Return lRet := !lRet
267   EndIf
268
269   If (nQtde > 0 .AND. (nQtde > nQtdeDisp .AND. AllTrim(cExtra) # "1"))
270     lRet := !lRet
271   Endif
272
273
274   If !lRet .AND. lMessage
275     STFMessage("STBGLVlQtd", "STOP", STR0003) //"Quantidade informada superior à disponível na lista"
276     STFShowMessage("STBGLVlQtd")
277   Else
278     STFCleanInterfaceMessage()
279   EndIf
280 EndIf
281
282 Return lRet
283
284
285 //-----
286 /*{Protheus.doc} STBGLConf
287 Funcao de Confirmação da Inclusão de Item de Lista/Alteração de Quantidade
288 @param oGetQtde - Get Quantidade
289 @param nGetQtde - Quantidade
290 @param oCbxCbxMet - Combo Método de Entrega
291 @param cGetMet - Método de Entrega
292 @param oGetProd - Get de Produto
293 @param cGetProd - Variavel de Produto
294 @param aItRet - Itens de Lista Seleccionados
295 @param aDados02 - Dados dos Itens de Lista
296 @param aHeader02 - Header dos Itens de Lista
297 @param oGrpPrd - Grupo do Grid de Listas
298 @param aDados03 - Dados de Mensagens
299 @param aHeader03 - Array de Mensagens
300 @param cNumLst - Numero da Lista
301 @param cExtra - Item Extra 1 (Sim)/2 (Não)
302 @param aCbxCbxMet - Array de Métodos de Entrega
303 @param lGrdMsgAtv - Grid de Mensagens Ativo
304 @param aMainCFG - Configurações da Lista de Presentes
305 @return Nil
306 @sample
307 */
308 //-----
309 Function STBGLConf( oGetQtde,nGetQtde,oCbxCbxMet, cGetMet,;
310                   oGetProd, cGetProd , aItRet,aDados02, ;
311                   aHeader02, oGrpPrd, aDados03, aHeader03,;
312                   cNumLst, cExtra, aCbxCbxMet , lGrdMsgAtv, ;

```

```

313             aMainCFG, lButton, cGetCodBar)
314
315 Local nPos      := 0
316 Local cItem := ""
317 Local nTam := 0
318 Local cCodMens := ""
319 Local nX := 0
320 Local lQtdeVal := .T.
321 Local aTmp := {}
322 Local cProd := ""
323 Local cDesc := ""
324 Local lIncluiProd := .F.
325
326 Default lButton      := .T.
327 Default cGetCodBar   := ""
328
329 If ValType(oGrpPrd) == "0" .AND. ValType(nGetQtde) == "N" .AND. Len(aCbxMet) > 0
330
331     // Verifica opção de inclusão ou alteração de quantidade
332     If IIf(FindFunction("STIGlIncProd"),STIGlIncProd(),.F.)
333         lIncluiProd := .T.
334     EndIf
335
336     STFCleanInterfaceMessage()
337
338     //Valida se a lista selecionada no Grid de Lista é a mesma do Grid de Itens
339     If Len(aDados02) > 0
340         If cNumLst <> STDGLRtCol(2,"ME2_CODIGO",1,1,aHeader02,aDados02, .f., .f.)
341             STFMessage("STBGLConf", "STOP", STR0004) //"Posicionar na Grid de Lista Correspondente"
342             STFSHOWMessage("STBGLConf")
343             Return .F.
344         EndIf
345     EndIf
346
347
348     If !oGetQtde:lVisible
349         //Inclusão valida se lista permite Extra
350         lQtdeVal := AllTrim(cExtra) == "1"
351     Else
352         If aMainCfg[POS_MLAQTD]
353             lQtdeVal := STBGLVlQtd(oGrpPrd,nGetQtde,cExtra, aHeader02,aDados02, .T.)
354         EndIf
355     EndIf
356
357     If lQtdeVal .AND. !oGetQtde:lVisible
358         lQtdeVal := !Empty(cGetProd)
359     EndIf
360
361
362     Do While lQtdeVal .and. lGrdMsgAtv .and. (nX := nX + 1) <= Len(aDados03) .AND. Empty(cCodMens)
363         If STDGLRtCol(2,"SEL",1,nX,aHeader03,aDados03, .f., .f.) == "1"
364             cCodMens := STDGLRtCol(2,"MED_CODIGO",1,nX,aHeader03,aDados03, .f., .f.)

```

```

365         EndIf
366     EndDo
367
368     nX := aScan(aItRet, { |l| l[1] == cNumLst})
369
370     //Inserção de Produto
371     If lQtdeVal .AND. lIncluiProd .AND. AllTrim(cExtra) == "1" .AND. oGetProd:lVisible
372
373         //Atualiza os Dados do item
374         aTmp := STDGLFRt(aHeader02, 0, , aDados02, ;
375             cGetProd, "", nGetQtde, cNumLst, ;
376             aCbxMet[1], cCodMens, nGetQtde, , ;
377             aCbxMet)
378
379         aAdd(aDados02, aClone(aTmp[1]))
380         nPos := Len(aDados02)
381         oGrpPrd:SetArray(aDados02)
382         oGrpPrd:GoTo(nPos, .t.)
383         cProd := STDGLRtCol(2, "ME2_PRODUT", 1, oGrpPrd:nAt, aHeader02, aDados02, .f., .f.)
384         nPos := 0
385
386     ElseIf lQtdeVal .AND. oGrpPrd:nAt > 0 .AND. (oGetQtde:lVisible .OR. oCbxMet:lVisible) .AND. !oGetProd:lVisible
387
388         //Atualização de Grid
389         cItem := STDGLRtCol(2, "ME2_ITEM", 1, oGrpPrd:nAt, aHeader02, aDados02, .f., .f.)
390         cProd := STDGLRtCol(2, "ME2_PRODUT", 1, oGrpPrd:nAt, aHeader02, aDados02, .f., .f.)
391         If nX > 0 .And. lButton
392             nPos := aScan(aItRet[nX][05], { |It| It[8] == cNumLst .AND. It[1] == cItem .and. It[2] == cProd})
393         Else
394             nPos := 0
395         EndIf
396
397     ElseIf AllTrim(cExtra) == "2"
398         STFMessage("STBGLConf2", "STOP", STR0005) //"Lista não permite a inclusão de item Extra"
399         STFShowMessage("STBGLConf2")
400         lQtdeVal := .F.
401     EndIf
402
403
404
405     If lQtdeVal
406
407         If nGetQtde = 0
408             //Quantidade Zerada exclui o item do array de retorno
409
410             If nPos > 0
411                 aDel(aItRet[nX,05], nPos)
412                 aSize(aItRet[nX,05], Len(aItRet[nX,05])-1)
413             EndIf
414             cCodMens := ""
415         ElseIf oGrpPrd:nAt > 0
416             //Atualiza o Item do Array de Retorno

```

```

417     If nPos = 0
418         If nX = 0
419             aAdd(aItRet, Array(5))
420             nX := Len(aItRet)
421             aItRet[nX, 01] := cNumLst
422             aItRet[nX, 02] := cExtra
423             aItRet[nX, 03] := "" //Remetente
424             aItRet[nX, 04] := ""
425             aItRet[nX, 05] := {}
426         EndIf
427         aAdd(aItRet[nX, 05], Array(9))
428         nPos := Len(aItRet[nX, 05])
429         aItRet[nX, 05][nPos, 1] := cItem
430         aItRet[nX, 05][nPos, 2] := cProd //Prod
431         aItRet[nX, 05][nPos, 3] := STDGLRtCol(2, "DISP0", 1, oGrpPrd:nAt, aHeader02, aDados02, .f., .f.) //Dipo
432         aItRet[nX, 05][nPos, 8] := cNumLst
433         aItRet[nX, 05][nPos, 6] := ""
434     EndIf
435
436     If aMainCfg[POS_MLAQTD]
437         aItRet[nX, 05][nPos, 4] := nGetQtde
438     Else
439         aItRet[nX, 05][nPos, 4] := STDGLRtCol(2, "QTDE", 1, oGrpPrd:nAt, aHeader02, aDados02, .f., .f.)
440     EndIf
441     aItRet[nX, 05][nPos, 7] := cCodMens //Mensagem
442
443     If aMainCfg[POS_MAME]
444         If Len(AllTrim(cGetMet)) > 1 .AND. !IsDigit(Substr(AllTrim(cGetMet), 1, 1))
445             aItRet[nX, 05][nPos, 5] := aCbxMet[aScan(aCbxMet, {|x| Upper(AllTrim(x)) == Upper(AllTrim(cGetMet))})][1]
446         Else
447             aItRet[nX, 05][nPos, 5] := cGetMet
448         Endif
449     Endif
450
451     Else
452         aItRet[nPos, 5] := Substr(STDGLRtCol(2, "ME1_TIP0", 1, oGrpPrd:nAt, aHeader02, aDados02, .f., .f.), 1, 1)
453     EndIf
454
455
456
457 EndIf
458
459 //Atualiza o Grid de Itens de Listas
460 If aMainCfg[POS_MLAQTD]
461     aDados02[oGrpPrd:nAt, STDGLGtCl("QTDE", aHeader02)] := nGetQtde
462 EndIf
463 aDados02[oGrpPrd:nAt, STDGLGtCl("MED_CODIGO", aHeader02)] := cCodMens
464
465 If nGetQtde > 0 .AND. LEN(aCbxMet) > 1
466     aDados02[oGrpPrd:nAt, STDGLGtCl("ME1_TIP0", aHeader02)] := aCbxMet[ Max(Val(aItRet[nX, 05][nPos, 5]), 1)]
467 Else
468     aDados02[oGrpPrd:nAt, STDGLGtCl("ME1_TIP0", aHeader02)] := aCbxMet[1] //Default

```

```
469      EndIf
470      oGrpPrd:GoTo(oGrpPrd:nAt, .t.)
471
472      STFMessage("STBGLConf3", "STOP", STR0006) //"Item de lista atualizado com sucesso."
473      STFShowMessage("STBGLConf3")
474
475
476      EndIf
477
478 EndIf
479
480 Return
```

Componentes/GiftList/STDGiftList.prw

475 linhas após remoção de metadados de autoria/data

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

```

416 /*/  

417 //000000000000000000000000000000000000000000000000000000000000000000000000000000000000000  

418  

419 Function STDSearCodBar(cCodSearch, cProduct, oGrpPrd, oGetCodBar)  

420  

421 Local aArea      := GetArea()  

422 Local aAreaSLK   := SLK->(GetArea()) //Area do SLK  

423 Local aAreaSB1   := SB1->(GetArea()) //Area do SB1  

424 Local lRet       := .F.  

425 Local nPos       := 0  

426  

427 Default cCodSearch := Space(TamSx3("B1_CODBAR")[1])  

428 Default cProduct    := Space(TamSx3("B1_COD")[1])  

429 Default oGrpPrd     := Nil  

430 Default oGetCodBar  := Nil  

431  

432 //Busca Codigo de Barras  

433 If !Empty(cCodSearch)  

434     //Busca Codigo do Produto  

435     Do Case  

436         Case !Empty(Posicione("SB1", 5, xFilial("SB1") + Padr(cCodSearch, TamSx3("B1_CODBAR")[1]), "B1_COD")) //Busca Codigo de Barras na SB1  

437             cProduct := SB1->B1_COD  

438             Case !Empty(Posicone("SLK", 1, xFilial("SLK") + Padr(cCodSearch, TamSx3("LK_CODBAR")[1]), "LK_CODIGO")) //Busca Codigo de Barras na SLK  

439                 cProduct := SLK->LK_CODIGO  

440             Case !Empty(Posicone("SB1", 1, xFilial("SB1") + Padr(cCodSearch, TamSx3("B1_COD")[1]), "B1_COD")) //Busca Codigo do Produto na SB1  

441                 cProduct := SB1->B1_COD  

442             Otherwise  

443                 cProduct := Space(TamSx3("B1_COD")[1])  

444     EndCase  

445  

446 If !Empty(cProduct)  

447     //Posiciona no produto - Grid  

448     nPos := aScan(oGrpPrd:oData:aArray,{|x,y| AllTrim(x[3]) == AllTrim(cProduct)})  

449  

450     If nPos > 0  

451         oGrpPrd:nAt := nPos  

452         oGrpPrd:Refresh()  

453         lRet := .T.  

454     Else  

455         cCodSearch := Space(TamSx3("B1_CODBAR")[1])  

456         STFMessage("STDSearCodBar1", "STOP", STR0001) //"Produto nao encontrado na Lista de Presentes"  

457         STFShowMessage("STDSearCodBar1")  

458         lRet := .F.  

459     EndIf  

460 Else  

461     cCodSearch := Space(TamSx3("B1_CODBAR")[1])  

462     oGetCodBar:Refresh()  

463     oGetCodBar:SetFocus()  

464     STFMessage("STDSearCodBar2", "STOP", STR0002) //"Codigo de Barras nao encontrado"  

465     STFShowMessage("STDSearCodBar2")  

466     lRet := .F.  

467 EndIf
```

```
468 EndIf
469
470 //Restaura areas
471 RestArea(aAreaSLK)
472 RestArea(aAreaSB1)
473 RestArea(aArea)
474
475 Return lRet
```

Componentes/GiftList/STWGiftList.prw

446 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PARMTYPE.CH"
2 #INCLUDE "STWGIFTLIST.CH"
3 #INCLUDE "Protheus.ch"
4 #INCLUDE "POSCSS.CH"
5 #INCLUDE "STPOS.CH"
6 #INCLUDE "FWBROWSE.CH"
7
8 #DEFINE POS_MMODO      1           //Modo de abertura
9 #DEFINE POS_MRET      2           //Modo de retorno
10 #DEFINE POS_MTPL      3           //Tipo da lista
11 #DEFINE POS_MNL       4           //Numero da lista (Filtro ME1)
12 #DEFINE POS_MONL      5           //Online
13 #DEFINE POS_MLNOM     6           //Entrar com nomes dos presenteadores
14 #DEFINE POS_MLENVM     7           //Enviar mensagem
15 #DEFINE POS_MLITAB    8           //Listar itens em aberto
16 #DEFINE POS_MORI      9           //Origem da lista (filtro ME1)
17 #DEFINE POS_MFILT     10          //Filtro (ME2)
18 #DEFINE POS_MMULT     11          //Multi-selecao
19 #DEFINE POS_MMTOD     12          //Marcar todos
20 #DEFINE POS_MQTDU     13          //Quantidade utilizada
21 #DEFINE POS_MLAQTD    14          //Alterar quantidade
22 #DEFINE POS_MAME      15          //Alterar modo de entrega
23 #DEFINE POS_MTPEVE    16          //Tipo de evento (filtro ME1)
24 #DEFINE POS_MSTAT     17          //Status da lista (filtro ME1)
25
26
27 //-----
28 /*{Protheus.doc} STWGLSearch
29 Função de Busca da Lista de Presentes
30 @param cFil      - Filial da Busca
31 @param aHeader01 - Header da Lista
32 @param aFiltro   - Filtro de Busca
33 @param aDados01  - Array de Dados
34 @param oDlg      - Janela de Retorno
35 @param oGD01     - Grid de Retorno
36 @param aMainCfg  - Configuração principal
37 @return Nil
38 @sample
39 */
40 //-----
41 Function STWGLSearch( cFil      ,aHeader01,aFiltro,aDados01,;
42                     oDlg      ,oGD01  , aMainCfg)
43
44 Local aLstCmp      := {}           //Lista de campos para consulta
45 Local lRet         := .F.         //Controle de retorno
46 Local aRet         := {}           //Retorno do Execute
47
48 Default cFil       := ""           // Filial
49 Default aHeader01  := {}           // cabeçalho
50 Default aFiltro    := {}           // filtro da tabela de itens
51 Default aDados01   := {}           // array dos dados a incluir
52 Default oDlg       := NIL          // objeto do dialog

```

```

53 Default oGD01      := NIL                // objeto do dialog
54 Default aMainCfg    := {}                // array com tamanho do objeto GD
55
56 If ValType(aFiltro) == "A" .AND. Len(aFiltro) > 0
57     //Montar lista de campos a partir do cabeçalho
58     aEval(aHeader01,{|x| aAdd(aLstCmp,x[1])})
59     //Bloco de tratamento de erro
60
61     LjMsgRun( STR0012,, { || lRet := STDGLRetD( "LJ845PESQC", {"",aLstCmp,aFiltro,0,.T.,aMainCfg}, @aRet) } ) //"Pesquisando Lista na Retaguarda. Aguarde..."
62
63     If !lRet .OR. ValType(aRet) <> "A"
64         ConOut(STR0013 + "Lj845PesqC")      //"Erro na execução(Retaguarda) - "
65         Alert(STR0013 + "Lj845PesqC")      //"Erro na execução(Retaguarda) - "
66     ElseIf ValType(aRet) == "A" .AND. Len(aRet) == 0
67         MsgInfo(STR0014,STR0015)           //"Lista não localizada. Verifique os parâmetros de pesquisa." ## //"Consulta Retaguarda"
68     EndIf
69 Endif
70
71 aDados01 := aClone(aRet)
72
73 If oDlg # Nil
74     oGD01:SetArray(aDados01)
75     oGD01:Refresh()
76     oGD01:GoTop(.T.)
77     Eval(oGD01:bLDbClick)
78 Endif
79
80 Return
81
82 //-----
83 /*/{Protheus.doc} STWGLitSearch
84 Função de Busca dos Itens da Lista de Presentes
85 @param      cNumLst - Numero da Lista
86 @param      aDados01 - Dados da Lista
87 @param      aLstC01 - Campos da Lista
88 @param      aItRet - Itens de Listas Selecionados
89 @param      aMsgRet - Mensagens
90 @param      aMainCfg - Configuracao Principal
91 @param      aLstC02 - Campos Itens
92 @param      aLstC03 - Campos Mensagens
93 @param      aHeader02 - Header Itens
94 @param      aDados02 - Dados Itens de Lista
95 @param      oPnlGrdP - Panel de Itens de Listas
96 @param      oGrpPrd - Panel do Grid de Itens
97 @param      oCbxMet - Combo Métodos de Entrega
98 @param      aCbxMet - Array de Combo de Métodos de Entrega
99 @param      oGrpGen - Grupo de Grid de Mensagens/Atores/Entregas
100 @param      aDados03 - Dados de Mensagens
101 @param      nPosCab - Posicao do Grid de Listas
102 @param      lGrdMsgAtv - Grid de Mensagens Ativo
103 @param      cTextoGen - Texto do Grid de Mensagens/atores/Entregas
104 @param      oLblGen - Label de Mensagens/atores/Entrega

```

[illegible]

```

157     lOK := STDGLRetD("LJ845RETD", ;
158         {xFilial("MED"),"MED","",aLstC03,1,.F.,.T.},;
159         @aMsgRet
160     )
161     aEval(aMsgRet, { |l|, aAdd(aDados03, aClone(aIns(aSize(l, len(l)+1), 1))), aTail(aDados03)[1] := space(1) } )
162 Endif
163 If lOK
164     ConOut("Execução com sucesso - " + "LJ845RETD MED" )
165 Else
166     ConOut("Erro na execução - " + "LJ845RETD MED")
167 EndIf
168
169 lExtra := STDGLRtCol(2,"ME1_EXTRA",0,nPosCab,aLstC01,aDados01,.F.,lTCBox) == "1"
170 lListaItAb := aMainCfg[POS_MLITAB]
171 //Se estiver configurado para listar todos os itens, verificar se extras sao permitidos na lista, se nao for permitido bloquear a visualizacao de itens zerados
172 If !lListaItAb .AND. !lExtra
173     lListaItAb := .T.
174 Endif
175 //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA ¿
176 //³Alimentar arquivo temporario ³
177 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA Û
178 cTMP := STDGLRtCol(2,"ME1_TIP0",0,nPosCab,aLstC01,aDados01,.F.,lTCBox)
179 Do Case
180     Case cTMP == "1" //Credito
181         cModoEnt := STR0002 //"1=Credito"
182         aCbxCbxMet := {STR0002,STR0003} //"1=Credito"#"2=Retira"
183     Case cTMP == "2" //Entrega
184         cModoEnt := STR0004 //"1=Entrega"
185         aCbxCbxMet := {STR0004,STR0003} //"1=Entrega"#"2=Retira"
186     Case cTMP == "4" //2-Entrega ou 4-Retira
187         cModoEnt := STR0003 //"2=Retira"
188         aCbxCbxMet := {STR0004,STR0003} //"1=Entrega"#"2=Retira"
189     Otherwise //Entrega programada
190         cModoEnt := STR0004 //"1=Entrega"
191         aCbxCbxMet := {STR0004,STR0003} //"1=Entrega"#"2=Retira"
192 EndCase
193
194 aDados02 := STDGLIt(aDados02, aMainCfg, aLstC02, lTCBox, ;
195     lListaItAb, aHeader02, aItRet, cNumLst, ;
196     cModoEnt, aCbxCbxMet)
197
198 If oCbxCbxMet # NIL
199     oCbxCbxMet:SetItems(aCbxCbxMet)
200
201     If cTMP == "4"
202         oCbxCbxMet:nat := 2
203     EndIf
204
205     oCbxCbxMet:Refresh()
206 EndIf
207
208 If oPnlGrdP # Nil

```

```

209         oGrpPrd:SetArray(aDados02)
210         oGrpPrd:GoTop(.T.)
211         Eval(oGrpPrd:bChange)
212     Endif
213
214     lPosicionado := .T.
215     If ValType(oBtnConf) == "0"
216         oBtnConf:Refresh()
217     EndIf
218
219     If !lGrdMsgAtv
220         //Monta a grid de mensagens
221         STWGLGdGe( oPnlGen, @aDados03, aHeader03, aAuxCombo, ;
222                 @oGrpGen, @lGrdMsgAtv, @cTextoGen, @oLblGen, ;
223                 @cTextoBtn, @oBtnCnc, @oBtnCnf)
224     Else
225
226         //Monta o Grid de Mensagens
227         If oGrpGen # NIL .and. Len(aDados03) > 0
228             oGrpGen:SetArray(aDados03)
229             oGrpGen:Refresh()
230             lGrdMsgAtv := .T.
231         EndIf
232     EndIf
233
234 Return lRet
235
236 //-----
237 /*{Protheus.doc} STWGLGdGe
238 Função de Montagem do Grid de Mensagens
239 @param oPnlGen - Panel Genérico
240 @param aDados03 - Dados da Mensagem
241 @param aHeader03 - Header da Mensagem
242 @param aAuxCombo - Combo da Mensagem
243 @param oGrpGen - Grupo de Campos Genérico
244 @param lGrdMsgAtv - Grid de Mensagem Ativo
245 @param cTextoGen - Texto Genérico
246 @param oLblGen - Label Genérico
247 @param cTextoBtn - Texto do botão
248 @param oBtnCnc - Botao cancela
249 @param oBtnCnf - Botao confirmar
250 @return Nil
251 @sample
252 */
253 //-----
254
255 Function STWGLGdGe( oPnlGen, aDados03, aHeader03, aAuxCombo, ;
256                   oGrpGen, lGrdMsgAtv, cTextoGen, oLblGen, ;
257                   cTextoBtn, oBtnCnc, oBtnCnf)
258
259     STIGLGdGe( oPnlGen, aDados03, aHeader03, aAuxCombo, ;
260               @oGrpGen,"aDados03[oBrwConten3:nAt]", .F.)

```

```

261
262     lGrdMsgAtv:= .T.
263     cTextoGen := STR0005 //"Mensagens"
264     oLblGen:Refresh()
265
266     cTextoBtn := STR0006 //"Cancelar"
267     oBtnCnc:cCaption := STR0006
268
269     oBtnCnc:lVisible := .T.
270     oBtnCnf:lVisible := .T.
271
272     oBtnCnc:Refresh()
273     oBtnCnf:Refresh()
274 Return
275
276
277 //-----
278 /*{Protheus.doc} STWGLBtCnf
279 Função de Finalização do Panel da Lista
280 @param aListaRet - Itens de Lista Selecionados
281 @return lRet - Sucesso
282 @sample
283 */
284 //-----
285
286 Function STWGLBtCnf(aListaRet)
287
288 Local nItemLine      := 0           // Linha do item
289 Local oModelSale     := STDGPBModel() // Model de venda
290 Local lRet           := .F.         // Retorno
291 Local nX             := 0           //Contador 2
292 Local lItemFiscal    := .T.         //Item Fiscal
293 Local aItRet         := {}          //Item de Retorno
294 Local nY             := 0           //Contador 1
295
296
297 STIExchangePanel( { || STIPanItemRegister() } )
298
299 For nY := 1 to Len(aListaRet)
300     aItRet := {}
301     aItRet := aClone(aListaRet[nY, 05]) //Itens da Lista
302
303
304     For nX := 1 to Len(aItRet)
305         //Seta a quantidade
306         STBSetQuant( aItRet[nX, 04] )
307
308         If oModelSale:GetModel("SL2DETAIL"):Length() == 1 .AND. Empty(STDGPBasket("SL2","L2_NUM",1))
309             nItemLine := 1
310         Else
311             nItemLine := oModelSale:GetModel("SL2DETAIL"):Length()+1
312         EndIf

```

```

313
314     aItRet[nX, 05] := Substr(aItRet[nX, 05], 1,1 )
315     If Empty(aItRet[nX, 05]) .OR. aItRet[nX, 05] <> "2"
316         aItRet[nX, 05] := "3"
317     EndIf
318
319     lItemFiscal := aItRet[nX, 05] == "2"
320
321
322     // Dispara o registro de item
323     STWItemReg( nItemLine      , aItRet[nX, 02]      , /*cCliCode */ , /*cCliLoja */ , ;
324         /*nMoeda*/ , /*nDiscount*/ , /*cTypeDesc */ , /*lAddItem*/ , ;
325         /*cItemTES */ , /*cCliType */ , lItemFiscal , /*nPrice*/ , ;
326         /*cTypeItem*/ , /*lInfoCNPJ*/ , /*lRecovery*/ , /*nSecItem */ , ;
327         /*lServFinal*/ , /*lProdBonif*/ , .T. , aItRet[nX, 08] , ;
328         aItRet[nX, 01] , aItRet[nX, 07] , aItRet[nX, 05] )
329
330     //Atualiza interface
331     STIGridCupRefresh()
332
333     Next nX
334 Next nY
335
336 STIBtnActivate()
337
338 Return lRet
339
340
341 //-----
342 /*{Protheus.doc} STWGLAtEnt
343 Função de Exibição dos Dados de Atores/Entregas da Lista
344 @param lGrdMsgAtv - Grid de Mensagens Ativo
345 @param nOpc = Opção de Seleção 1 - Entregas Programadas/ 2 - Atores
346 @param cNumLst - Numero da Lista
347 @param cTipoLst - Tipo de Lista
348 @param oPnlGen - Panel do Grid
349 @param aDados04 - Dados de Atores/Entregas
350 @param oGrpGen - Grid Genérico
351 @param cTextoGen - Texto Genérico
352 @param cLblGen - Label Genérico
353 @param cTextoBtn - Texto Botão
354 @param oBtnCnc - Botão Cancelar/Fechar
355 @param oBtnCnf - Botão Confirmar
356 @return lOK - Sucesso
357 @sample
358 */
359 //-----
360 Function STWGLAtEnt(lGrdMsgAtv, nOpc, cNumLst, cTipoLst, ;
361     oPnlGen, aDados04, oGrpGen, cTextoGen, ;
362     cLblGen, cTextoBtn, oBtnCnc, oBtnCnf )
363
364 Local cAlias := ""

```

```

365 Local aHeader01          := {} //Header di Grid de Mensagens /Entregas
366 Local aLstCmpNL          := {"MEH_CODLIS","MEE_CODLIS"} //Lista de campos para nao listar
367 Local aLstCmp            := {} //Lista de Campos
368 Local lOK                := .F.
369
370 Do Case
371   Case nOpc == 1
372     cAlias := "MEH" //Entregas programada
373     //Pesquisar se a lista alvo eh do tipo de entrega programada
374     If cTipoLst # "3"
375       STFMessage("STWGLab", "STOP",STR0007) //"Este tipo de lista de presente não é de entrega programada!"
376       STFShowMensagem("STWGLab")
377
378       Return lOK
379     Endif
380     cTextoGen := STR0008 //"Entregas programadas"
381   Case nOpc == 2
382     cAlias := "MEE" //Atores
383     cTextoGen := STR0009 //"Atores"
384   Otherwise
385     Return lOK
386 EndCase
387
388 //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAA¿
389 //³Lista de campos e dados da GD ³
390 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAÙ
391 DbSelectArea("SX3")
392 SX3->(DbSetOrder(1))
393 SX3->(DbSeek(cAlias))
394 While !SX3->(Eof()) .AND. SX3->X3_ARQUIVO == cAlias
395   If X3Uso(SX3->X3_USADO) .AND. cNivel >= SX3->X3_NIVEL .AND. aScan(aLstCmpNL,{|x| x == Upper(AllTrim(SX3->X3_CAMPO))}) == 0 .AND. ;
396     SX3->X3_CONTEXT # "V"
397
398     aAdd(aLstCmp,SX3->X3_CAMPO)
399     //Cabecalho para a GD de lista
400     aAdd(aHeader01,{SX3->X3_CAMPO,SX3->X3_PICTURE,AllTrim(X3Titulo()),SX3->X3_TAMANHO,SX3->X3_TIPO})
401   Endif
402   SX3->(DbSkip())
403 EndDo
404 //UAAAAAAAAAAAAAAAAAAAAAAAAAAAA¿
405 //³Montar a lista de dados ³
406 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAÙ
407 lOK = STDGLRetD( "LJ845RETD", ;
408   {xFilial(cAlias),cAlias,cAlias + "." + cAlias + "_CODLIS = '" + cNumLst + "'",aLstCmp,1,.T.},;
409   @aDaDos04
410 )
411 If lOK
412   ConOut("Execução com sucesso - " + "LJ845RETD " + cAlias )
413 Else
414   ConOut("Erro na execução - " + "LJ845RETD " + cAlias )
415 EndIf
416

```

```
417 If !lOk .OR. Len(aDados04) == 0
418     lOk := .F.
419 Endif
420
421 If lOk
422
423     If lGrdMsgAtv
424         //Ativo o Grid de Mensagens
425         cTextoBtn := STR0011 //"Mensagens"
426         oBtnCnc:cCaption := cTextoBtn
427         oBtnCnc:lVisible := .T.
428         oBtnCnf:lVisible := .F.
429
430         oBtnCnc:Refresh()
431         oBtnCnf:Refresh()
432         lGrdMsgAtv := .F.
433
434     EndIf
435     cLblGen:Refresh()
436
437     //Monta o Grid de Atores/Entregas
438     STIGLGd2Gen( oPnlGen, @aDados04, aHeader01, @oGrpGen,"aDados04[oBrwConten3:nAt]", .F.)
439
440     oGrpGen:SetArray(aDados04)
441
442     oGrpGen:Refresh()
443
444 EndIf
445
446 Return lOk
```

Componentes/ImportSale/STBImportSale.prw

2818 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STBIMPORTSALE.CH"
4 #INCLUDE "FWMBROWSE.CH"
5 #INCLUDE "FWMVCDEF.CH"
6 #INCLUDE "APWIZARD.CH"
7 #INCLUDE "FWADAPTEREAI.CH"
8 #INCLUDE "STPOS.CH"
9
10 Static aCmpsImpL1 := STBImpSFld("SL1") //Relcacao de campos PADRAO da tabela SL1 utilizaos na importacao de orcamento
11 Static aCmpsImpL2 := STBImpSFld("SL2") //Relcacao de campos PADRAO da tabela SL2 utilizaos na importacao de orcamento
12 Static aCmpsImpL4 := STBImpSFld("SL4") //Relcacao de campos PADRAO da tabela SL4 utilizaos na importacao de orcamento
13
14 Static aSL4Bkp := {} //Backup da SL4 importada
15
16 Static lPerguntouCPF := .F. //Se por acaso fez a pergunta "Deseja Imprimir CPF/CNPJ no Comprovante da Venda ?" uma vez - STR0007
17 Static lRetCPF := .F. //Opção escolhida após a pergunta "Deseja Imprimir CPF/CNPJ no Comprovante da Venda ?" - STR0007 - .T. (Sim) ou .F. (Não)
18
19 Static nDescImp := 0
20 Static lPDVOnline := ExistFunc("STFGetOnPdv") .AND. STFGetOnPdv() // Variável para definir se o totvs PDV está no modo online
21
22 Static aOrcPayOri := {"",0,0}
23 //-----
24 /*{Protheus.doc} STBISSearchOptions
25 Busca Opções de orçamento em outro ambiente
26
27 @param1 nOption Opcao de busca
28 @param2 cField Campo utilizado para busca
29 @param3 cFieldAssigned Conteúdo do campo para busca
30 @param4 lSearchAll Define se perquisa todos ( Redução Z do PAF )
31 @param5 aFieldsAdd Campos adicionais a serem buscados
32 @param6 dDataMov Data do movimento a realizar a Busca PAFECE
33 @return aRet[1] lUserSelect - Retorna se deve ser selecionada opção de orçamento
34 @return aRet[2] aOptionSales - Array com as opções de orçamento
35 @obs
36 @sample
37 */
38 //-----
39 Function STBISSearchOptions( nOption , cField , cFieldAssigned , lSearchAll , ;
40 aFieldsAdd , dDataMov )
41
42 Local lUserSelect := .F. // Define se usuário deve selecionar orçamento
43 Local aOptionSales := {} // Opções de orçamento buscadas da retaguarda
44 Local lContinue := .T. // Fluxo lógico função
45 Local ni := 1 // Variavel de loop
46 Local cNomeCli := "" // Nome do cliente
47 Local nVlrTot := 0 // Valor do orçamento importado
48
49 Default nOption := 0
50 Default cField := ""
51 Default cFieldAssigned := ""
52 Default lSearchAll := .F.

```

```

53 Default aFieldsAdd      := {}
54 Default dDataMov        := dDataBase
55
56
57 /*/
58     OBS: Para pesquisa pelo campo, a premissa eh que eh pesquisa de usuário, sendo assim nao contempla Redução Z do PAF (LOJA160)
59     Para contemplar, deve ser passado parâmetro TODOS ou DATA
60 /*/
61
62 LjGrvLog("Importa_Orcamento:STBISearchOptions","nOption :", nOption)
63 LjGrvLog("Importa_Orcamento:STBISearchOptions","cField  :", cField)
64 LjGrvLog("Importa_Orcamento:STBISearchOptions","cFieldAssigned: ", cFieldAssigned)
65 LjGrvLog("Importa_Orcamento:STBISearchOptions","aFieldsAdd:    ", aFieldsAdd)
66 LjGrvLog("Importa_Orcamento:STBISearchOptions","lSearchAll:    ", lSearchAll)
67 LjGrvLog("Importa_Orcamento:STBISearchOptions","aFieldsAdd:    ", aFieldsAdd)
68 LjGrvLog("Importa_Orcamento:STBISearchOptions","dDataMov:      ", dDataMov)
69
70 Do Case
71
72     Case nOption == 1          // Pelo campo
73
74         /*/
75         Para Pesquisa por numeração, chamar importação direto
76         /*/
77         If cField == "L1_NUM" .OR. cField == "L1_NUMORC"
78
79             LjGrvLog("Importa_Orcamento:STBISearchOptions","Chama rotina remoto: STBISGetOp")
80             lContinue := STBRemoteExecute( "STBISGetOp" , ;
81                 { nOption , cField , cFieldAssigned , Nil , Nil , aFieldsAdd } , ;
82                 NIL , ;
83                 .F. , ;
84                 @aOptionSales )
85
86             LjGrvLog("Importa_Orcamento:STBISearchOptions","Parâmetro de retorno lContinue :", lContinue)
87             If !lContinue
88                 LjGrvLog("Importa_Orcamento:STBISearchOptions","Parâmetro de retorno aOptionSales :", aOptionSales)
89             EndIf
90
91             // Percorre array para pegar os dados do cliente e valor para apresentar na tela de importação
92             While ni <= LEN(aOptionSales)
93
94                 If StrZero(Val(cFieldAssigned),TamSX3(cField)[1]) == aOptionSales[ni][1]
95                     cNomeCli := aOptionSales[ni][3]
96                     //"Atribuindo ao Valor total da venda o valor do acréscimo e do desconto((L1_VLRTOT + L1_VLRJUR) - L1_DESCONT) "
97                     nVlrTot := STBarred( aOptionSales[ni][4] )
98
99                 EndIf
100                 ni++
101             EndDo
102
103             lUserSelect := .F. // Nao precisa de seleção
104

```

```

105      /*/
106      Para Pesquisa por cliente ou por algum outro campo do SL1, pode existir mais de 1 orcamento, pesquisar opções e depois importar
107      /*/
108      ElseIf Subst( cField, 1, 2 ) == "A1" .OR. Subst( cField, 1, 2 ) == "L1"
109
110          LjGrvLog("Importa_Orcamento:STBISearchOptions","Chama rotina remoto: STBISGetOp")
111          lContinue := STBRemoteExecute( "STBISGetOp"
112              { nOption , cField , cFieldAssigned , Nil , Nil , aFieldsAdd } , ;
113              NIL
114              .F.
115              @aOptionSales
116              )
117
118          LjGrvLog("Importa_Orcamento:STBISearchOptions","Parâmetro de retorno lContinue :", lContinue)
119
120      If !lContinue
121          LjGrvLog("Importa_Orcamento:STBISearchOptions","Parâmetro de retorno aOptionSales :", aOptionSales)
122          STFMessage("STImportSale","STOP",STR0001) //"Não foi possível estabelecer conexão."
123          STFShowMessage("STImportSale")
124          LjGrvLog("Importa_Orcamento:STBISearchOptions", STR0001) //"Não foi possível estabelecer conexão."
125      EndIf
126
127      If lContinue
128
129          If Len(aOptionSales) == 0
130
131              /*/
132              Mensagem
133              /*/
134              If Subst( cField, 1, 2 ) == "A1"
135                  STFMessage("STImportSale","STOP",STR0002) //"Nao foi encontrado orcamento em aberto para este cliente"
136                  LjGrvLog("Importa_Orcamento:STBISearchOptions", STR0002) //"Nao foi encontrado orcamento em aberto para este cliente"
137              Else
138                  STFMessage("STImportSale","STOP",STR0003) //"Nao foi encontrado orcamento em aberto"
139                  LjGrvLog("Importa_Orcamento:STBISearchOptions", STR0003) //"Nao foi encontrado orcamento em aberto"
140              EndIf
141
142              STFShowMessage("STImportSale")
143              lContinue := .F.
144          EndIf
145      EndIf
146
147      If lContinue
148          If Len(aOptionSales) > 1
149              lUserSelect := .T. // Indica que devem ser selecionados os orçamentos a importar
150          Else
151              If cField $ "A1_CGC/L1_NUM/L1_NUMORC"
152                  lUserSelect := .F. // Retornou apenas 1 orçamento, importar direto, sem seleção
153              Else
154                  lUserSelect := .T.
155              EndIf
156          EndIf

```

```

157         EndIf
158
159     EndIf
160
161     Case nOption == 2 .OR. nOption == 3 // 2 - Todos ; 3 - Data
162
163         /*/
164         Buscar opcoes de orcamentos
165         /*/
166         LjGrvLog("Importa_Orcamento:STBISearchOptions","Chama rotina remoto: STBISGetOp")
167         lContinue := STBRemoteExecute( "STBISGetOp" , ;
168                                     { nOption , NIL , NIL , (STFGetCfg("lPafEcf") .OR. STFGetCfg("lPAFNFCF")) , lSearchAll , aFieldsAdd , dDataMov } , ;
169                                     NIL , ;
170                                     .F. , ;
171                                     @aOptionSales )
172
173         LjGrvLog("Importa_Orcamento:STBISearchOptions","Parâmetro de retorno lContinue :", lContinue)
174         If lContinue
175             While ni <= LEN(aOptionSales)
176                 //"Atribuindo ao Valor total da venda o valor do acréscimo e do desconto((L1_VLRTOT + L1_VLRJUR) - L1_DESCONT) "
177                 nVlrTot := STBArred( aOptionSales[ni][4] )
178                 aOptionSales[ni][4] := nVlrTot
179                 ni++
180             EndDo
181         EndIf
182
183         If !lContinue
184             LjGrvLog("Importa_Orcamento:STBISearchOptions","Parâmetro de retorno aOptionSales :", aOptionSales)
185             STFMessage("STImportSale","STOP",STR0001) //"Nao foi possivel estabelecer conexão."
186             STFShowMessage("STImportSale")
187             LjGrvLog("Importa_Orcamento:STBISearchOptions", STR0001) //"Não foi possível estabelecer conexão."
188         EndIf
189
190         If lContinue
191             If ValType(aOptionSales) <> "A" .OR. Len(aOptionSales) == 0
192                 STFMessage("STImportSale","STOP", STR0003 ) //"Nao foi encontrado orcamento em aberto"
193                 STFShowMessage("STImportSale")
194                 LjGrvLog("Importa_Orcamento:STBISearchOptions", STR0003) //"Nao foi encontrado orcamento em aberto"
195                 aOptionSales := {}
196                 lContinue := .F.
197             EndIf
198         EndIf
199
200         lUserSelect := .T. // Indica que orçamento precisa ser selecionado
201
202     EndCase
203
204     aRet := { lUserSelect , aOptionSales }
205     If Len(aRet) = 0
206         LjGrvLog("Importa_Orcamento:STBISearchOptions", "Parâmetro de retorno aRet : Conteúdo vazio." )
207     EndIf
208

```

```

209 Return aRet
210
211
212 //-----
213 /*{Protheus.doc} STBISGetOptions
214 Busca de opções orçamentos a serem importados
215
216 @param  nOption          Opcao de busca
217 @param  cField           Campo utilizado para busca
218 @param  cFieldAssigned   Conteúdo do campo para busca
219 @param  lIsPafPdv        Indica se o PDV que solicitou eh PAF
220 @param  lSearchAll       Indica se pesquisa
221 @param  aFieldsAdd       Campos adicionais a serem buscados
222 @param  dDataMov         Data do movimento a realizar a Busca PAFECE
223 @return  aOptions        Retorna opções de orçamento
224 @obs
225 @sample
226 */
227 //-----
228 Function STBISGetOptions( nOption , cField , cFieldAssigned , lIsPafPdv , ;
229                          lSearchAll , aFieldsAdd , dDataMov )
230
231 Local aOptions          := {}          // Retorno funcao
232
233 Default nOption         := 0
234 Default cField          := ""
235 Default cFieldAssigned  := ""
236 Default lSearchAll      := .F.
237 Default aFieldsAdd      := {}
238 Default dDataMov        := dDataBase
239
240 LjGrvLog("Importa_Orçamento:STBISGetOptions","Comunicação OK")
241 LjGrvLog("Importa_Orçamento:STBISGetOptions","nOption :", nOption)
242 LjGrvLog("Importa_Orçamento:STBISGetOptions","cField :", cField)
243 LjGrvLog("Importa_Orçamento:STBISGetOptions","Parâmetro cFieldAssigned: ", cFieldAssigned)
244 LjGrvLog("Importa_Orçamento:STBISGetOptions","Parâmetro lIsPafPdv:      ", lIsPafPdv)
245 LjGrvLog("Importa_Orçamento:STBISGetOptions","Parâmetro lSearchAll:      ", lSearchAll)
246 LjGrvLog("Importa_Orçamento:STBISGetOptions","Parâmetro aFieldsAdd:      ", aFieldsAdd)
247 LjGrvLog("Importa_Orçamento:STBISGetOptions","Parâmetro dDataMov:        ", dDataMov)
248
249 Do Case
250
251     Case nOption == 1 // Busca Pelo Campo
252
253         If !Empty(cField) .AND. !Empty(cFieldAssigned)
254
255             If Subst( cField, 1, 2 ) == "A1" .OR. Subst( cField, 1, 2 ) == "L1"
256
257                 LjGrvLog("Importa_Orçamento:STBISGetOptions","Chama rotina: STDISGetFieldOptions")
258                 aOptions := STDISGetFieldOptions( cField , cFieldAssigned , aFieldsAdd )
259
260             EndIf

```

```

261
262         EndIf
263
264         Case nOption == 2 .OR. nOption == 3
265
266             LjGrvLog("Importa_Orcamento:STBISGetOptions","Chama rotina: STDISGetOptions")
267             aOptions := STDISGetOptions( nOption , lIsPafPdv , lSearchAll , aFieldsAdd , dDataMov )
268
269         EndCase
270
271         If !Empty(aOptions) .And. ExistBlock("STBGetOrc")
272
273             LjGrvLog("Importa_Orcamento:STBISGetOptions","Antes da Execução do Ponto de entrada STBGetOrc", aOptions)
274             aOptions := ExecBlock("STBGetOrc", .F.,.F., aOptions)
275             LjGrvLog("Importa_Orcamento:STBISGetOptions","Retorno do Ponto de Entrada STBGetOrc", aOptions)
276
277         EndIf
278
279         If Len(aOptions) = 0
280             LjGrvLog("Importa_Orcamento:STBISGetOptions","Parâmetro de retorno aOptions: Conteúdo vazio.")
281         EndIf
282
283         Return aOptions
284
285
286         //-----
287         /*{Protheus.doc} STBImportRemote
288         Busca do Orcamento em outro ambiente
289
290         @param      cField          Campo utilizado para busca
291         @param      cFieldAssigned  Conteúdo do campo para busca
292         @param      lValDate        Indica se valida data de vencimento do orcamento
293         @Param      lPafEcf         Usa PAfecf
294         @Param      lGetOrcExpired  Pega orcamentos expirados PafECF
295         @return      aSale
296         @obs
297         @sample
298         */
299         //-----
300         Function STBImportRemote( cField , cFieldAssigned , lValDate , lPafEcf , ;
301                                 lGetOrcExpired )
302
303         Local aArea      := GetArea()    // Armazena alias corrente
304         Local aSale      := {}           // Retorno da funcao
305         Local aSL1       := {}           // Armazena SL1
306         Local aSL2       := {}           // Armazena SL2
307         Local aSL4       := {}           // Armazena SL4
308         Local lContinue  := .T.          // Fluxo lógico funcao
309         Local cPAFTType  := ""           // Se DAV ("D") ou PRE-VENDA ("P")
310         Local cSL1Status := ""           // Armazena STATUS
311         Local nI         := 0            // Contador
312         Local nL1_NUM    := 0            // Armazena posicao do L1_NUM

```

```

313 Local aRetCred      := {}           // Array com as informações da Avaliação de Crédito
314 Local cCredLj       := SuperGetMV( "MV_CREDLJ",,"N")// Indica se deve ser feita a Análise de Crédito do Cliente
315 Local lSTVLDIMP     := ExistBlock("STVLDIMP")
316 Local lSL1Locked    := .F.
317 Local aFPgtNPerm    := {}           // Forma de Pagamento não permitido no PDV mas q consta no orçamento
318 Local cPgts         := ""           // Variável para ordenar as formas de pagamentos para exibir na mensagem
319 Local aMDJ          := {}
320
321 Default cField       := ""
322 Default cFieldAssigned := ""
323 Default lValDate     := .T.
324 Default lPafEcfcf    := .F.
325 Default lGetOrcExpired := .F.
326
327 ParamType 0 Var      cField          As Character      Default ""
328 ParamType 1 Var      cFieldAssigned As Character      Default ""
329 ParamType 2 Var      lValDate        As Logical        Default .T.
330
331 lContinue := !Empty(cField) .AND. !Empty(cFieldAssigned)
332
333 LjGrvLog("Importa_Orcamento:STBImportRemote","Comunicação OK")
334 LjGrvLog( " 01 - Importação do PDV", "Campo " + cField + ", cFieldAssigned = " + cFieldAssigned, Nil )
335
336 If !lContinue
337     AADD( aSale , "NOTFOUND" )
338 EndIf
339
340 If lContinue
341     Do Case
342         Case cField == "L1_NUM"
343             LjGrvLog( " 02 - Importação do PDV", "aSale", Nil )
344             LjGrvLog("Importa_Orcamento:STBImportRemote","Chama rotina: STDISSL1Get")
345             aSL1 := STDISSL1Get( cFieldAssigned, @lSL1Locked )
346
347             If Len(aSL1) == 0
348                 lContinue := .F.
349                 AADD( aSale , "NOTFOUND" )
350                 LjGrvLog("Importa_Orcamento:STBImportRemote","Parâmetro de retorno aSL1: Conteúdo vazio")
351             EndIf
352
353         Case cField == "L1_NUMORC"
354
355             If SuperGetMv("MV_LJPRVEN",,.T.)
356                 cPAFType := "P"
357             Else
358                 cPAFType := "D"
359             EndIf
360
361             LjGrvLog("Importa_Orcamento:STBImportRemote","Chama rotina: STDISPafSL1Get, cPafType = ", cPafType)
362             aSL1 := STDISPafSL1Get( cFieldAssigned , cPAFType )
363
364             LjGrvLog("Importa_Orcamento:STBImportRemote","Parâmetro de retorno aSL1: ", aSL1)

```

```

365
366     If Len(aSL1) == 0
367         lContinue := .F.
368         AADD( aSale , "NOTFOUND" )
369         LjGrvLog("Importa_Orcamento:STBImportRemote","Parâmetro de retorno aSL1: Conteúdo vazio")
370     EndIf
371
372     Otherwise
373
374         lContinue := .F.
375
376 EndCase
377
378 If lContinue
379
380     /*/
381     Validacoes
382     /*/
383     LjGrvLog("Importa_Orcamento:STBImportRemote","Monta SL1")
384     LjGrvLog("Importa_Orcamento:STBImportRemote","Chama rotina: STBISValSL1")
385
386     cSL1Status := STBISValSL1( aSL1 , lValDate )
387
388     LjGrvLog("Importa_Orcamento:STBImportRemote","Parâmetro de retorno cSL1Status: ", cSL1Status)
389
390     //Verifica se esta solicitando orçamentos vencidos
391     If cSL1Status == "EXPIRADO" .AND. lGetOrcExpired
392         lContinue := .T.
393         cSL1Status := ""
394     ElseIf !Empty(cSL1Status)
395         AADD( aSale , cSL1Status )
396         lContinue := .F.
397         LjGrvLog("Importa_Orcamento:STBImportRemote","Adicionado conteúdo de cSL1Status em aSale e não irá continuar.")
398     EndIf
399
400     If lContinue
401         nL1_NUM := AScan( aSL1 , { |x| x[1] == "L1_NUM" } ) // Armazena posicao L1_NUM
402     EndIf
403
404     LjGrvLog("Importa_Orcamento:STBImportRemote","Posição L1_NUM: ", nL1_NUM)
405
406     /*/
407     Monta SL2
408     /*/
409     If lContinue
410
411         LjGrvLog("Importa_Orcamento:STBImportRemote","Monta SL2")
412         LjGrvLog("Importa_Orcamento:STBImportRemote","Chama rotina: STDISSL2Get")
413         LjGrvLog("Importa_Orcamento:STBImportRemote","Parâmetro aSL1[nL1_NUM][2]: ", aSL1[nL1_NUM][2])
414         LjGrvLog("Importa_Orcamento:STBImportRemote","Parâmetro lPafEcf: ", lPafEcf)
415
416         aSL2 := STDISSL2Get( aSL1[nL1_NUM][2] , lPafEcf )

```

```

417
418      /*/
419      Valida estoque
420      /*/
421      If !( STBISValStock( aSL2 ) )
422          lContinue := .F.
423          AADD( aSale , "SEMESTOQUE" )
424          LjGrvLog("Importa_Orcamento:STBImportRemote","Adicionado SEMESTOQUE em aSale e não irá continuar: aSL2 vazio.")
425
426      EndIf
427
428  EndIf
429
430  /*/
431  Monta SL4
432  /*/
433  If lContinue
434
435      LjGrvLog("Importa_Orcamento:STBImportRemote","Monta SL4")
436      LjGrvLog("Importa_Orcamento:STBImportRemote","Chama rotina: STDISSL4Get")
437      LjGrvLog("Importa_Orcamento:STBImportRemote","Parâmetro aSL1[nL1_NUM][2]: ", aSL1[nL1_NUM][2])
438
439      aSL4 := STDISSL4Get( aSL1[nL1_NUM][2] )
440
441      If Len(aSL4) = 0
442          LjGrvLog("Importa_Orcamento:STBImportRemote","Parâmetro de retorno aSL4: Conteúdo vazio.")
443      Elseif STBFPgtNPe(aSL4, @aFPgtNPerm)
444          aEval( aFPgtNPerm, { |x| cPgts += x + "|" } )
445          cPgts := PADR(AllTrim(cPgts), Len(AllTrim(cPgts))-1)
446          AADD( aSale , "FPGTONAOPERMITIDO" )
447          AADD( aSale , cPgts )
448      EndIf
449
450  EndIf
451
452  /*/
453  AVALIA CREDITO DO CLIENTE
454  /*/
455  If cCredLj == "S"
456      aRetCred := STBISValCred(2, aSL1 , aSL2 , aSL4)//2 - Salvar como Venda
457      If !aRetCred[1]
458
459          AADD( aSale , "SEMCREDITO" )
460          lContinue := aRetCred[1]
461      Endif
462  Endif
463
464  If lSTVLDIMP
465      LjGrvLog("Importa_Orcamento:STBImportRemote","Antes da chamada do Ponto de Entrada: STVLDIMP",lContinue)
466      lContinue := ExecBlock( "STVLDIMP",.F.,.F.,{aSL1,aSL2,aSL4})
467      LjGrvLog("Importa_Orcamento:STBImportRemote","Depois da chamada do Ponto de Entrada: STVLDIMP",lContinue)
468      If !lContinue

```

```

469         AADD( aSale , "BLOCKPE" )
470     EndIf
471 EndIf
472
473 // Tratamento para retornar MDJ e MDK quando for Central de PDV
474 If ExistFunc("STDBuscaNccOrc") .AND. (LjGetCPDV()[1] .OR. LjGetCPDV()[2])
475     aMDJ:= STDBuscaNccOrc(cFieldAssigned)
476 Endif
477
478
479 /*/
480 Tratamento do retorno
481 /*/
482 If lContinue // Indica que deve ser retornado o orcamento (tudo certo)
483     If Len(aSale) == 0 // Nenhum STATUS atribuído
484         If lSL1Locked
485             AADD( aSale , "SL1LOCK" )
486         Else
487             AADD( aSale , "OK" )
488         Endif
489     EndIf
490     AADD( aSale , { aSL1 , aSL2 , aSL4, aMDJ } )
491 Else
492     If Len(aSale) == 0
493         AADD( aSale , "ERROR" )
494         LjGrvLog("Importa_Orcamento:STBImportRemote","Parâmetro de retorno aSale: Conteúdo vazio. (ERROR)")
495     EndIf
496 EndIf
497
498 EndIf
499 EndIf
500
501 If Len(aSale) = 0
502     LjGrvLog("Importa_Orcamento:STBImportRemote","Parâmetro de retorno aSale: Conteúdo vazio.")
503 EndIf
504
505 Return aSale
506
507
508 //-----
509 /*/{Protheus.doc} STBISValSL1
510 Valida orcamento a ser importado
511
512 @param  aSL1          Array com a estrutura do SL1. Bidimensional: Campo, Valor
513 @param  lValDate      Indica se valida a data limite
514 @return cStatus       Retorna oStatus do orcamento
515 @obs
516 @sample
517 /*/
518 //-----
519 Static Function STBISValSL1( aSL1 , lValDate )
520

```

```

521 Local cStatus           := ""           //Status
522 Local dL1_DTLIM         := cToD(" / / ")
523
524 Default aSL1             := {}
525 Default lValDate         := .T.
526
527 ParamType 0 Var          aSL1           As Array           Default {}
528 ParamType 1 Var          lValDate       As Logical         Default .T.
529
530 If Len(aSL1) > 0
531
532     dL1_DTLIM := aSL1[AScan( aSL1 , { |x| x[1] == "L1_DTLIM" } )][2]
533     If ValType(dL1_DTLIM) == "C"
534         dL1_DTLIM := STOD( dL1_DTLIM )
535     EndIf
536
537     Do Case
538         Case !Empty(AllTrim(aSL1[AScan( aSL1 , { |x| x[1] == "L1_DOC" } )][2])) .OR. !Empty(AllTrim(aSL1[AScan( aSL1 , { |x| x[1] == "L1_DOCPED" } )][2]))
539             cStatus := "VENDIDO"
540
541
542         Case !STBVldImp(aSL1[AScan( aSL1 , { |x| x[1] == "L1_NUM" } )][2]) // Arquivo de controle de orcamento
543             //Esta validacao tem que vir sempre antes da validacao do orcamento expirado Expirado
544             cStatus := "JAIMPORTADO"
545
546         Case lValDate .AND. ( dL1_DTLIM < dDataBase )
547             cStatus := "EXPIRADO"
548
549     EndCase
550
551 EndIf
552
553 EndIf
554
555 LjGrvLog("Importa_Orcamento:STBISValSL1","Parâmetro de retorno cStatus: ", cStatus)
556
557 Return cStatus
558
559
560
561 //-----
562 /*{Protheus.doc} STBVldImp
563 Valida se o arquivo referente ao orcamento importado ja existe
564
565 @param  cNumOrc          numero do orçamento
566 @return lRet             Retorna Validacao
567 @obs
568 @sample
569 */
570 //-----
571
572 Static Function STBVldImp( cNumOrc )

```

```

573
574 Local lRet      := .T.                //Retorno da funcao. Se .T. nao foi importado ainda
575 Local cPatchOrc := ""                //Arquivo do orcamento importado
576 Local cArqTemp  := ""                //Arquivo para testar se existe a pasta "\AUTOCOM\IMP" + cEmpAnt + cFilAnt
577 Local nHandle   := 0                 //Handle do arquivo
578 Local cDirLog   := ""                // Diretorio para o LOG
579
580 cDirLog := "\AUTOCOM\IMP" + StrTran(cEmpAnt + cFilAnt , " ", "")+ "\"
581
582 cPatchOrc := cDirLog + cNumOrc + ".TXT"
583 cArqTemp  := cDirLog + "TEMP.TMP"
584
585 If !File( cArqTemp )
586     MakeDir( "\AUTOCOM" )
587     MakeDir( "\AUTOCOM\IMP" + cEmpAnt + cFilAnt )
588     nHandle := FCreate( cArqTemp )
589     FClose( nHandle )
590 Endif
591
592 If File( cPatchOrc )
593     lRet := .F.
594 Endif
595
596 Return lRet
597
598
599 //-----
600 /*{Protheus.doc} STBCtrImpOrc
601 Cria arquivo de controle no server para saber se orcamento
602 ja foi importado. O objetivo e nao permitir importar mais de uma vez
603
604 @param   cNumOrc      numero do orçamento
605 @param   lVerifica    Controla se faz apenas a verificacao da existencia do arquivo na Retaguarda
606 @return  lRet         Retorna Validacao
607 @obs
608 @sample
609 */
610 //-----
611 Function STBCtrImpOrc(cNumOrc, cPDV , lVerifica, lTotvsPdv, lValidRet)
612
613 Local aRet      := {}                //Retorna numero dos orcamentos que nao gravaram o arquivo (erro)
614 Local nX        := 0                 //Controle de loop
615 Local cCRLF     := Chr(13) + Chr(10) //Controle de linha
616 Local cMsg      := ""                //Mensagem dos orcamentos com erro na criacao do arquivo na RET
617 Local lRet      := .T.                //Retorno
618 Local lContinua := .F.                //Controle de execucao
619 Local aOrcsImp  := {}                //Array de orcamentos a confirmar
620
621 Default cNumOrc := ""                //Numero do Orcamento para criar arquivo de conttrole
622 Default cPDV    := ""                //Numero do PDV gravado no SL2->L2_PDV
623 Default lVerifica := .F.              //Controla se faz apenas a verificacao da existencia do arquivo na Retaguarda
624 Default lTotvsPdv := .F.              //E totvs pdv?

```

```

625 Default lValidRet := .F. //Validar arquivo .RET na Retaguarda
626
627 If !Empty(cNumOrc)
628
629     AADD(aOrcsImp , { cNumOrc , cPDV } )
630
631     //Atualizando status do orcamento
632     LjGrvLog("Importa_Orcamento:STBCtrImpOrc","Chama rotina remoto: FR271HArq")
633     LjGrvLog("Importa_Orcamento:STBCtrImpOrc","Parâmetro aOrcsImp: ",aOrcsImp)
634     LjGrvLog("Importa_Orcamento:STBCtrImpOrc","Parâmetro lVerifica: ",lVerifica)
635     LjGrvLog("Importa_Orcamento:STBCtrImpOrc","Parâmetro lTotvsPdv: ",lTotvsPdv)
636     lContinua := STBRemoteExecute( "FR271HArq" , { aOrcsImp , lVerifica, Nil, lTotvsPdv, lValidRet }, Nil, .F. , @aRet )
637     LjGrvLog("Importa_Orcamento:STBCtrImpOrc","Parâmetro de retorno lContinua: ",lContinua)
638     If !lContinua
639         LjGrvLog("Importa_Orcamento:STBCtrImpOrc","Parâmetro de retorno aRet: ",aRet)
640     EndIf
641
642     If lContinua
643
644         If ValType(aRet) <> "A"
645             aRet := {} //Volta array pra nao dar erro
646             CONOUT("ERRO NO RETORNO DA CONEXÃO ARQUIVO DE CONTROLE ORCAMENTO. STBCtrImpOrc: ")
647             CONOUT(cNumOrc + cPDV)
648             LjGrvLog("Importa_Orcamento:STBCtrImpOrc","ERRO NO RETORNO DA CONEXÃO ARQUIVO DE CONTROLE ORCAMENTO. STBCtrImpOrc: ", cNumOrc + cPDV)
649             If lVerifica
650                 CONOUT("lVerifica = .T.")
651             Else
652                 CONOUT("lVerifica = .F.")
653             EndIf
654             If lTotvsPdv
655                 CONOUT("lTotvsPdv = .T.")
656             Else
657                 CONOUT("lTotvsPdv = .F.")
658             EndIf
659         EndIf
660
661         If !lValidRet .And. !lVerifica
662             //Se foi criado arquivo .TXT, apaga o arquivo .RET
663             //Todo controle passa para o arquivo .TXT
664             STBRemoteExecute( "FR271HArq" , { aOrcsImp , lVerifica, .T., lTotvsPdv, .T. }, Nil, .F. , @aRet )
665
666             If ValType(aRet) <> "A"
667                 aRet := {} //Volta array pra nao dar erro
668             EndIf
669         EndIf
670     EndIf
671
672 EndIf
673
674 //Apresenta mensagem se houve erro na criacao do arquivo de algum orcamento importado
675 If lContinua .AND. Len(aRet) > 0 .AND. !lTotvsPdv
676

```

```

677 For nX := 1 to Len(aRet)
678     cMsg += aRet[nX] + "\"
679 Next nX
680 cMsg := Substr(cMsg, 1, Len(cMsg) - 1)
681
682 lRet := .F.
683
684 If lVerifica
685     STFMesssage("STBIMPORTSALE","POPUP", STR0004)    //"Não é possível finalizar a venda, este orçamento já foi finalizado por outro PDV."
686     STFShowMessage("STBIMPORTSALE")
687     LjGrvLog("Importa_Orcamento:STBCtrImpOrc", STR0004) //"Não é possível finalizar a venda, este orçamento já foi finalizado por outro PDV."
688 Else
689     STFMesssage("STBIMPORTSALE","POPUP", STR0005 + cCRLF + ;//Nao foi possivel criar um ou mais arquivos referentes a importacao do orcamento."
690             STR0006 + cMsg) //"Orcamentos: "
691     STFShowMessage("STBIMPORTSALE")
692     LjGrvLog("Importa_Orcamento:STBCtrImpOrc", STR0005 + " - " + STR0006 + cMsg)    //Nao foi possivel criar um ou mais arquivos referentes a importacao do
orcamento.###"Orcamentos: "
693 EndIf
694
695 EndIf
696
697 Return aRet
698
699
700 //-----
701 /*{Protheus.doc} STBISValStock
702 Valida estoque na importação de orçamento
703
704 @param aSL2          Array com a estrutura do SL2. Bidimensional: Campo, Valor
705 @return lRet          Retorna a validacao do estoque dos itens
706 @obs
707 @sample
708 */
709 //-----
710 Static Function STBISValStock( aSL2 )
711
712 Local aArea          := GetArea()          // Armazena alias corrente
713 Local lRet           := .T.                // Retorno da funcao
714 Local nI             := 0                 // Contador
715 Local nX             := 0                 // Contador
716 Local nL2_RESERVA    := 0                 // Posição da reserva no array
717 Local aItensConsult := {}                // Itens a consultar estoque
718 Local lValThisItem   := .F.               // Define se avalia estoque para esse item
719 Local lItemIncluded  := .F.               // Define se o item ja foi incluido no array
720 Local nL2_ENTREGA    := 0                 // Posição do campo L2_ENTREGA no array
721
722
723 Default aSL2         := {}
724
725 ParamType 0 Var      aSL2                As Array          Default {}
726
727 If Len(aSL2) > 0

```

```

728 If SuperGetMV( "MV_LJESTOR" , , .F. ) .AND. ( SuperGetMV( "MV_ESTNEG" , , "S" ) == "N" )
729
730 nL2_RESERVA := AScan( aSL2[1] , { |x| x[1] == "L2_RESERVA" } )
731 nL2_ENTREGA := AScan( aSL2[1] , { |x| x[1] == "L2_ENTREGA" } )
732 // Armazena os produtos a serem avaliados estoque
733 For nI := 1 To Len(aSL2)
734
735 lValThisItem := .F.
736
737 If Empty(aSL2[nI][nL2_RESERVA][2]) .AND. !aSL2[nI][nL2_ENTREGA][2] == "5" //pedido entrega sem reserva não valida estoque
738
739 lValThisItem := .T.
740
741 lItemIncluded := .F.
742
743 For nX := 1 To Len(aItensConsult)
744
745 If AllTrim(aItensConsult[nX][1]) == AllTrim(aSL2[nI][AScan( aSL2[1] , { |x| x[1] == "L2_PRODUTO" } )][2]) ;
746 .AND. AllTrim(aItensConsult[nX][2]) == AllTrim(aSL2[nI][AScan( aSL2[1] , { |x| x[1] == "L2_LOCAL" } )][2])
747
748 lItemIncluded := .T.
749 nItemIncluded := nX
750 Exit
751
752 EndIf
753
754 Next nX
755
756 EndIf
757
758 If lValThisItem
759
760 If lItemIncluded
761
762 aItensConsult[nItemIncluded][4] += aSL2[nI][AScan( aSL2[nI] , { |x| x[1] == "L2_QUANT" } )][2]
763
764 Else
765
766 AADD( aItensConsult , {
767 aSL2[nI][AScan( aSL2[1] , { |x| x[1] == "L2_PRODUTO" } )][2] , ;
768 aSL2[nI][AScan( aSL2[1] , { |x| x[1] == "L2_LOCAL" } )][2] , ;
769 aSL2[nI][AScan( aSL2[1] , { |x| x[1] == "L2_TES" } )][2] , ;
770 aSL2[nI][AScan( aSL2[1] , { |x| x[1] == "L2_QUANT" } )][2] } )
771
772 EndIf
773
774 EndIf
775
776 Next nI
777
778 // Avaliar estoque
779 For nI := 1 To Len(aItensConsult)

```

```

780
781     DbSelectArea("SF4")
782     DbSetOrder(1)
783     If DbSeek( xFilial("SF4") + aItensConsult[nI][3] )
784         If SF4->F4_ESTOQUE == "S"
785
786             // Valida SB2
787             DbSelectArea("SB2")
788             DbSetOrder(1)
789
790             If DbSeek( xFilial("SB2") + aItensConsult[nI][1] + aItensConsult[nI][2] )
791
792                 If ((SB2->B2_QATU - SB2->B2_RESERVA) <= 0) .OR. ((SB2->B2_QATU - SB2->B2_RESERVA) < aItensConsult[nI][4])
793
794                     lRet := .F.
795                     Exit
796
797                 EndIf
798
799             Else
800                 lRet := .F.
801             EndIf
802
803         EndIf
804     EndIf
805
806     Next nI
807
808     Else
809         lRet := .T.
810     EndIf
811 EndIf
812
813 RestArea(aArea)
814
815 LjGrvLog("Importa_Orcamento:STBISValStock", "Parâmetro de retorno lRet: ",lRet)
816
817 Return lRet
818
819 //-----
820 /*{Protheus.doc} STBImportSale
821 Adiciona as informações do orçamento na estrutura da venda
822
823 @param1   aSale           ,array ,           Array com a estrutura do orçamento: L1, L2 e L4
824 @Param2   lCancel         ,lógico,           Cancela orçamento apos importar Pafecf
825 @Param3   lImpMultOrc     ,lógico,           Importação de multiplos Orçamento (parametro utilizado no pdv-online)
826 @return   lRet            Retorna se importou orçamento com sucesso
827 @obs
828 @sample
829 */
830 //-----
831 Function STBImportSale( aSale , lCancel, lImpMultOrc , lCancOrc )

```

```

832 Local lRet           := .T.           // Retorno da função
833 Local lCpfRet        := .F.           // Retorno da função STWInfoCNPJ()
834 Local aSL1           := {}            // Cabeçalho
835 Local aSL2           := {}            // Itens
836 Local aSL4           := {}            // Pagamentos
837 Local aItem          := {}            // Informacoes do item
838 Local nTotalSale     := 0             // Valor total da venda
839 Local oTotal         := STFGetTot()    // Recebe o Objeto total
840 Local cCliCode       := ""            // Codigo Cliente
841 Local cCliStore       := ""            // Loja Cliente
842 Local cCliType        := ""            // Tipo do cliente
843 Local cNumOrig        := ""            // Numeração do orçamento original vindo da retaguarda
844 Local nI              := 0             // Contador
845 Local nX              := 0
846 Local nY              := 0             // Contador
847 Local nNextItem       := 0             // Próximo item
848 Local lItemFiscal     := .T.           // Indica se o Item eh fiscal, vai para o cupom fiscal
849 Local lItemRegister   := .T.           // Indica se registrou o item
850 Local nL2_PRODUTO     := 0             // Posição
851 Local nL2_VALDESC     := 0             // Posição
852 Local nL2_TES         := 0             // Posição
853 Local nL2_PRCTAB      := 0             // Posição
854 Local nL2_QUANT        := 0             // Posição
855 Local nL2_CODBAR       := 0             // Posição
856 Local nL2_ITEMGAR      := 0             // Posição L2_ITEMGAR
857 Local nL2_KIT          := ""            // Posição Código do kit de produto utilizado
858 Local oCliModel       := NIL           // Model do Cliente
859 Local lIsRetPost       := .F.           // Identifica se é finalizacao de venda de orcamento somente com itens de "RETIRA POSTERIOR", para imprimir o Cupom Fiscal e
      Finalizar.
860 Local cFilRes         := ""            // Filial do Orcamento da reserva
861 Local cOrcRes         := ""            // Codigo do Orcamento da reserva
862 Local cPedRes         := ""            // Codigo do Pedido da reserva
863 Local lFinServ        := SuperGetMV("MV_LJCSF",,.F.) // Valida implementação do servico financeiro
864 Local aAcres          := {}            // array que contera os valoresde acrescimo na importacao
865 Local cCondPg         := ""
866 Local lStImpSale      := ExistBlock("STIMPSALE")
867 Local xStImpSale      :=               // Retorno do ponto de entrada STIMPSALE (pode ser Logico ou Array)
868 Local lStImpField     := ExistBlock( "STIMPFIELD") // PE para importação de campos customizados da SL1 e SL2
869 Local aStImpField     := {}            // Retorno do ponto de entrada STIMPFIELD
870 Local aSL1PE          := {}            // campos customizados da tabela SL1
871 Local aSL2PE          := {}            // campos customizados da tabela SL2
872 Local nPos            := 0
873 Local lRegistrou       := .F.
874 Local cVendedor        := ""
875 Local cCodProdReg      := ""            //Codigo do produto a ser registrado (codigo ou codigo de barras)
876 Local oVendModel       := Nil
877 Local cCodItemGar      := ""            //Código do Item de Garantia Vinculada
878 Local lInfoCNPJ        := .F.           //Verifica se deverá enviar o CNPJ na abertura do cupom, valida parametro MV_LJDCCLI para permitir alterar CNPJ de
      orcamento importado
879 Local nCallCPF        := SuperGetMV("MV_LJDCCLI",,0) //O momento onde será mostrado o CPF na tela
880 Local nScanJuros      := 0             //Variavel de controle para o aScan
881 Local nScanVLJur       := 0             //Variavel de controle para o aScan

```

```

882 Local nDescTot      := 0 //Valor de Desconto Total do Orcamento
883 Local nPercDesc     := 0 //Percentual de Desconto Total do Orcamento
884 Local nTotOrc       := 0 //Valor Total do Orcamento
885 Local aDiscount     := {} //Array com Valor e Percentual de Desconto concedido no Total do Orcamento
886 Local nDescProp     := 0 //Valor do Desconto Proporcionalizado no item
887 Local nDescDesc     := TamSX3("L2_VALDESC")[2] //Qtde de Casas decimais desconto no item
888 Local nVlrDescIt    := 0 //Valor do desconto no Item
889 Local nDesTotProp   := 0
890 Local lImportSale   := .T.
891 Local lZeraPay       := .F. // Indica se é para zerar os pagamentos
892 Local lPosCrd       := CrdxInt(.F.,.F.) .AND. ExistFunc("STBSetCrdIdent") // Ativa Integração TOTVSPDV x SIGACRD
893 Local cCgcCart      := "" // Cartão para SIGACRD
894 Local cIndPres      := ""
895 Local cTransp        := ""
896 Local nPosL1INDP    := 0 // Posição do campo L1_INDPRES no array aSL1[] , caso não existir o campo na RET não terá essa posição.
897 Local lTemReserv    := .F.
898 Local cTpFrete       := "" //Tipo de Frete
899 Local lEmitenNFe     := .F. //Se emite NF pelo PDV envia o parametro para MafisIni()
900 Local cFormpg       := "" // Valor do campo L1_FORMPG
901 Local aMDJ          := {}
902 Local nDescNF        := 0 //Valor do campo L1_DESCNF do Orçamento a ser importado
903 Local nIcmsRet       := 0 //Valor do campo L1_ICMSRET do Orçamento a ser importado
904 Local cCliente       := "" //Valor do campo L1_CLIENTE do Orçamento a ser importado
905 Local nVlrLiq        := 0 //Valor do campo L1_VLR LIQ do Orçamento a ser importado
906 Local nValMerc       := 0 //Valor do campo L1_VALMERC do Orçamento a ser importado
907 Local lCRdesItTt    := SuperGetMv("MV_LJRGDES",,.F.) .AND. SuperGetMV("MV_LJCRDPT",,"0") == "1" .AND.
FindFunction("totvs.protheus.retail.desconto.RegraDescProdutoTotal.LjCallCalcRegDescProdTotal", .T.) // Verifica se o calculo do desconto por item esta sendo feito no
final da venda
908 Local cTabPad       := SuperGetMV("MV_TABPAD") // Tabela de preço utilizada
909 Local lDesligaRD     := .F. // Desabilita a Regra de Desconto do Varejo
910 Local lCalRDImp      := lCRdesItTt .And. SuperGetMv("MV_LJRDIMP",,"0") == "0"
911
912 Default aSale       := {}
913 Default lImpMultOrc  := .F.
914 Default lCancOrc     := .T. // Determina se continua com o cancelamento do orçamento quando se tem o PE STIMPSALE
915
916 /*/
917 Valida se o orçamento passado contém cabeçalho, itens e pagamento
918 /*/
919 lRet := Len(aSale) > 0 .AND. Len(aSale[1]) > 0 .AND. Len(aSale[2]) > 0 .AND. Len(aSale[3]) > 0
920 If lRet
921 aSL1 := aSale[1]
922 aSL2 := aSale[2]
923 aSL4 := aSale[3]
924 aMDJ := Iif(Len(aSale[4])>0, aSale[4], {})
925
926 //Faço Backup para uso posteriormente na finalização do pagamento
927 aSL4Bkp := aSL4
928
929 nTotOrc := aSL1[AScan( aSL1 , { |x| x[1] == "L1_VLRTOT" } )][2] + aSL1[AScan( aSL1 , { |x| x[1] == "L1_DESCONT" } )][2]
930 nDescTot := aSL1[AScan( aSL1 , { |x| x[1] == "L1_DESCONT" } )][2] + aSL1[AScan( aSL1 , { |x| x[1] == "L1_DESCFIN" } )][2]
931 nDescNF := aSL1[AScan( aSL1 , { |x| x[1] == "L1_DESCNF" } )][2]

```

```

932     nIcmsRet      := aSL1[AScan( aSL1 , { |x| x[1] == "L1_ICMSRET" } )][2]
933     cCliente      := aSL1[AScan( aSL1 , { |x| x[1] == "L1_CLIENTE" } )][2]
934     nVlrLiq       := aSL1[AScan( aSL1 , { |x| x[1] == "L1_VLR LIQ" } )][2]
935     nValMerc      := aSL1[AScan( aSL1 , { |x| x[1] == "L1_VALMERC" } )][2]
936
937     If nIcmsRet > 0 .AND. cCliente <> SuperGetMV("MV_CLIPAD",, "") .AND. (nVlrLiq > nValMerc) .AND. nTotOrc > nValMerc .AND. nDescNF > 0
938         nTotOrc := nTotOrc - nDescTot - nIcmsRet
939     Endif
940
941     nDesTotProp := nDescTot
942
943     aDiscount := STBDiscConvert( nDescTot , "V" , nTotOrc )
944     nDescTot  := aDiscount[1]
945     nPercDesc := aDiscount[2]
946
947     nTotalSale := oTotal:GetValue("L1_VLRTOT") + nTotOrc
948
949     STBSetDscImp(STBGetDscImp() + nDescTot )
950
951     LjGrvLog("Importa_Orcamento:STBImportSale","Valor total do orçamento:"+cValToChar(nTotalSale))
952 Else
953     LjGrvLog("Importa_Orcamento:STBImportSale", ">>> NC <<< Nao contem dados completos. aSale:", aSale)
954 EndIf
955
956 /*/
957     STIMPSALE - Ponto de Entrada para validar a importação do orçamento
958 /*/
959 If lStImpSale
960
961     LjGrvLog("Importa_Orcamento:STBImportSale", "Antes da chamada do Ponto de Entrada: STIMPSALE", {aSL1,aSL2,aSL4})
962     xStImpSale := ExecBlock( "STIMPSALE", .F., .F., {aSL1,aSL2,aSL4})
963     LjGrvLog("Importa_Orcamento:STBImportSale", "Apos a chamada do Ponto de Entrada: STIMPSALE", xStImpSale)
964     If ValType(xStImpSale) == "L"
965         lRet := xStImpSale
966     ElseIf ValType(xStImpSale) == "A"
967         If Len(xStImpSale) > 2
968             lRet := xStImpSale[1]
969             aSL1PE := xStImpSale[2]
970             aSL2PE := xStImpSale[3]
971         EndIf
972     ElseIf ValType(xStImpSale) == "C" .And. xStImpSale $ "2|3"
973         lRet := .F.
974         If xStImpSale == "3"
975             lCancOrc := .F.
976         EndIf
977     EndIf
978 EndIf
979
980 /*/
981     STIMPFIELD - Ponto de Entrada para permitir a importação de campos adicionais
982 /*/
983 If lStImpField

```

```

984
985 aStImpField := ExecBlock( "STIMPFIELD", .F., .F.)
986 If ValType(aStImpField) == "A"
987
988     //organizamos o vetor para que os campos sejam sequenciais (L1 > L2)
989     aSort(aStImpField)
990
991     For nI := 1 To Len(aStImpField)
992         If SubStr(aStImpField[nI], 1, 2) == "L1"
993             nPos := AScan( aSL1 , { |x| x[1] == aStImpField[nI] } )
994             If nPos > 0
995                 Aadd( aSL1PE, aSL1[nPos] )
996             EndIf
997         EndIf
998     Next
999
1000 EndIf
1001 EndIf
1002
1003 If lRet .And. FindFunction("totvs.protheus.retail.desconto.RegraDescProdutoTotal.HasDescManu", .T.)
1004     lDesligaRD := totvs.protheus.retail.desconto.RegraDescProdutoTotal.HasDescManu("TOTVSPDV", STDGPBasket('SL1','L1_NUM'), STDGPBasket('SL1','L1_CLIENTE'),
1005     STDGPBasket('SL1','L1_LOJA'))
1006
1007     If !lDesligaRD .And. lCalRDImp
1008         lDesligaRD := .T.
1009     ElseIf !lDesligaRD .And. !lCalRDImp
1010         lDesligaRD := .F.
1011     EndIf
1012 EndIf
1013
1014 /*/
1015 Chamada Atualiza Cliente Venda
1016 /*/
1017 If lRet
1018
1019     cCliCode := aSL1[AScan( aSL1 , { |x| x[1] == "L1_CLIENTE" } )][2]
1020     cCliStore := aSL1[AScan( aSL1 , { |x| x[1] == "L1_LOJA" } )][2]
1021     cCliType := aSL1[AScan( aSL1 , { |x| x[1] == "L1_TIPOCLI" } )][2]
1022     cFilRes := aSL1[AScan( aSL1 , { |x| x[1] == "L1_FILRES" } )][2]
1023     cOrcRes := aSL1[AScan( aSL1 , { |x| x[1] == "L1_ORCRES" } )][2]
1024     cPedRes := aSL1[AScan( aSL1 , { |x| x[1] == "L1_PEDRES" } )][2]
1025     cCondPg := aSL1[AScan( aSL1 , { |x| x[1] == "L1_CONDPG" } )][2]
1026     cVendedor := aSL1[AScan( aSL1 , { |x| x[1] == "L1_VEND" } )][2]
1027     cNumOrig := aSL1[AScan( aSL1 , { |x| x[1] == "L1_NUM" } )][2]
1028     cCgcCart := aSL1[AScan( aSL1 , { |x| x[1] == "L1_CGCCART" } )][2]
1029     nPosL1INDP := AScan(aSL1, { |x| x[1] == "L1_INDPRES" })
1030     If nPosL1INDP > 0
1031         cIndPres := aSL1[nPosL1INDP][2]
1032     Endif
1033     cTransp := aSL1[AScan( aSL1 , { |x| x[1] == "L1_TRANSP" } )][2]
1034     cTpFrete := aSL1[AScan( aSL1 , { |x| x[1] == "L1_TPFRET" } )][2]

```

```

1035 cFormpg      := aSL1[AScan( aSL1      , { |x| x[1] == "L1_FORMPG" } )][2]
1036
1037 lIsRetPost   := Empty(cPedRes) .And. !Empty(cOrcRes)
1038 lTemReserv   := STBExitRes(aSL2) //Verifica se tem item de entrega/reserva no orçamento
1039
1040 //Validamos se será considerado o cliente informado no PDV ou na Retaguarda.
1041 If !lTemReserv
1042     STBCliRet(@cCliCode, @cCliStore, @cCliType)
1043 EndIf
1044 oCliModel := STWCustomerSelection(cCliCode+cCliStore,cNumOrig, aMDj)
1045
1046 LjGrvLog( "Importa_Orcamento:STBImportSale","cCliCode/cCliStore:", cCliCode+"/"+cCliStore)
1047
1048 If !lImpMultOrc .AND. lPDVOnline .AND. Empty(STDPBLength("SL2"))
1049     STDSPBasket( "SL1" , "L1_NUM"      , cNumOrig )
1050 EndIf
1051
1052 STDSPBasket( "SL1" , "L1_CLIENTE"    , cCliCode )
1053 STDSPBasket( "SL1" , "L1_LOJA"      , cCliStore )
1054 STDSPBasket( "SL1" , "L1_TIPOCLI"    , cCliType )
1055 STDSPBasket( "SL1" , "L1_CONDPG"     , cCondPg )
1056 STDSPBasket( "SL1" , "L1_CGCCART"    , cCgcCart )
1057 STDSPBasket( "SL1" , "L1_INDPRES"    , cIndPres )
1058 STDSPBasket( "SL1" , "L1_TRANSP"     , cTransp )
1059 STDSPBasket( "SL1" , "L1_FORMPG"     , cFormpg )
1060 /* Setando vendedor na importação */
1061 oVendModel := STWSalesmanSelection(cVendedor)
1062 STDSPBasket("SL1","L1_VEND" ,oVendModel:GetValue("SA3MASTER","A3_COD"))
1063 STDSPBasket("SL1","L1_VEND2" ,oVendModel:GetValue("SA3MASTER","A3_SUPER"))
1064 STDSPBasket("SL1","L1_VEND3" ,oVendModel:GetValue("SA3MASTER","A3_GEREN"))
1065 STDSPBasket("SL1","L1_COMIS" ,STDGComission( oVendModel:GetValue("SA3MASTER","A3_COD") ))
1066
1067 If lIsRetPost
1068     STDSPBasket( "SL1" , "L1_FILRES" , cFilRes )
1069     STDSPBasket( "SL1" , "L1_ORCRES" , cOrcRes )
1070     STDSPBasket( "SL1" , "L1_TIPO"   , aSL1[AScan( aSL1,      { |x| x[1] == "L1_TIPO" } )][2] )
1071 EndIf
1072
1073 cCliente     := oCliModel:GetValue("SA1MASTER", "A1_COD" )
1074 cLojaCli     := oCliModel:GetValue("SA1MASTER", "A1_LOJA")
1075 cNomeCli     := oCliModel:GetValue("SA1MASTER", "A1_NOME")
1076 cEndCli      := oCliModel:GetValue("SA1MASTER", "A1_END" )
1077 cCgcCli      := oCliModel:GetValue("SA1MASTER", "A1_CGC" )
1078
1079 If lPosCrd    //Integração TOTVSPDV x SIGACRD
1080     STBSetCrdIdent(cCgcCart, cCgcCli, cCliente, cLojaCli)
1081 EndIf
1082
1083 If ExistFunc("STBEmiteNF") //Define se vai emitir NF
1084     lEmiteNFe:= STBEmiteNF(cCliente, cLojaCli)
1085 EndIf
1086

```

```

1087 If lEmitaNFe .AND. Empty(cTpFrete)
1088     cTpFrete:= "S"
1089 Endif
1090
1091 STDSPBasket("SL1", "L1_TPFRET", cTpFrete)
1092
1093 If !STBTaxFoun("IT", 1)
1094     STBTaxEnd()
1095     STBTaxIni( cCliCode, ;
1096               cCliStore, ;
1097               "C", ;
1098               "S", ;
1099               .F., ;
1100               "SB1", ;
1101               "LOJA701", ;
1102               .T., ;
1103               cCliType, ;
1104               cTpFrete , ;
1105               lEmitaNFe )
1106 Endif
1107
1108 If !lCancel
1109     If (nCallCPF = 2 .OR. nCallCPF = 3)
1110         STDSPBasket("SL1","L1_CGCCLI",cCGCCLI)
1111     ElseIf (nCallCPF = 0 .OR. nCallCPF = 1)
1112         /*Quando MV_LJDCCLI = 0 ou 1, verificar se possui CPF, se não possuir,
1113         limpar os campos cNomeCli e cEndCli. Motivo: Quando lInfoCNPJ = .T.,
1114         esta abrindo o cupom com Nome/end mesmo quando nao possui o CPF,
1115         com isso impacta na finalizacao da venda onde nao eh possuivel informar o CPF */
1116         If !Empty(AllTrim(cCgcCli))
1117             If Iif(GetApoInfo("STFRESTART.PRW")[4] >= Ctod("22/02/2019"), !STBGetPgtCPF(), .T.)
1118                 STFMessage(ProcName(0),"YESNO", STR0007) //"Deseja Imprimir CPF/CNPJ no Comprovante da Venda ?"
1119                 lCpfRet := STFSHOWMessage(ProcName(0))
1120                 STBSetCpfRet(lCpfRet)
1121                 STBSetPgtCPF(.T.)
1122             Else
1123                 lCpfRet := STBGetCpfRet()
1124             EndIf
1125         Else
1126             If !lCpfRet
1127                 cCgcCli := ""
1128                 cNomeCli:= ""
1129                 cEndCli := ""
1130             EndIf
1131         //Quando eu tenho emissão de NFC-e preciso deste campo
1132         STDSPBasket("SL1","L1_CGCCLI",cCGCCLI)
1133     Else
1134         //Se não possuir, limpar os campos cNomeCli e cEndCli.
1135         //Motivo: Quando lInfoCNPJ = .T., está abrindo o cupom com Nome/end mesmo quando não possui o CPF, com isso impacta na finalização da venda onde não é
1136         possível informar o CPF
1137         cNomeCli:= ""
1138         cEndCli := ""

```

```

1138         EndIf
1139         //Procedimento de impressão de CPF antes dos itens
1140         LjGrvLog("Importa_Orcamento:STBImportSale","Chama rotina: STD7CPFOverReceipt")
1141         STD7CPFOverReceipt(cCgcCli,cNomeCli,cEndCli)
1142         LjGrvLog("Importa_Orcamento:STBImportSale","Chama rotina: STI7InfCPF")
1143         STI7InfCPF(.T.)
1144     EndIf
1145 Else //lCancel
1146     LjGrvLog("Importa_Orcamento:STBImportSale","Chama rotina: STICGCCConfirm")
1147     STICGCCConfirm(@cCgcCli, cNomeCli, cEndCli, .T. , .F. )
1148 EndIf
1149 EndIf
1150
1151 If lRet
1152     // Armazena numeração original da retaguarda
1153     STDSPBasket( "SL1" , "L1_NUMORIG" , cNumOrig )
1154
1155     If STFGetCfg("lPafEcf") .OR. STFGetCfg("lPAFNFCE")
1156         //Grava Numero do Orcamento
1157         STDSPBasket( "SL1" , "L1_NUMORC" , aSL1[AScan( aSL1, { |x| x[1] == "L1_NUMORC" } )][2] )
1158         STDSPBasket( "SL1" , "L1_TPORC" , aSL1[AScan( aSL1, { |x| x[1] == "L1_TPORC" } )][2] )
1159         STDSPBasket( "SL1" , "L1_COODAV" , aSL1[AScan( aSL1, { |x| x[1] == "L1_COODAV" } )][2] )
1160     EndIf
1161 EndIf
1162
1163 // Atualiza os campos customizados
1164 If Len(aSL1PE) > 0
1165     For nI := 1 To Len(aSL1PE)
1166         STDSPBasket( "SL1" , aSL1PE[nI][1] , aSL1PE[nI][2])
1167     Next nI
1168 EndIf
1169
1170 If lRet .AND. SuperGetMv("MV_LJITCHK",,0) == 1 .AND. ExistFunc("STIItemChk")
1171     lRet := STIItemChk(@aSL2, @lZeraPay)
1172 EndIf
1173
1174 /*/
1175     Adicionar os Itens na Venda
1176 /*/
1177 If lRet
1178     /*/
1179         Armazena posições
1180     /*/
1181     nL2_PRODUTO := AScan( aSL2[1] , { |x| x[1] == "L2_PRODUTO" } )
1182     nL2_VALDESC := AScan( aSL2[1] , { |x| x[1] == "L2_VALDESC" } )
1183     nL2_TES := AScan( aSL2[1] , { |x| x[1] == "L2_TES" } )
1184     nL2_PRCTAB := AScan( aSL2[1] , { |x| x[1] == "L2_PRCTAB" } )
1185     nL2_QUANT := AScan( aSL2[1] , { |x| x[1] == "L2_QUANT" } )
1186     nL2_CODBAR := AScan( aSL2[1] , { |x| x[1] == "L2_CODBAR" } )
1187     nL2_ITEMGAR := AScan( aSL2[1] , { |x| x[1] == "L2_ITEMGAR" } )
1188     nL2_KIT := AScan( aSL2[1] , { |x| x[1] == "L2_KIT" } )
1189

```

```

1190 LjGrvLog("Importa_Orcamento:STBImportSale","Inicia For de Items SL2. Qtde de Items:"+cValToChar(Len(aSL2)))
1191
1192 For nI := 1 To Len(aSL2)
1193     // Seta quantidade
1194     STBSetQuant( aSL2[nI][nL2_QUANT][2] )
1195
1196     nNextItem := STDPBLength("SL2") + 1
1197
1198     For nY := 1 To Len(aStImpField)
1199         If SubStr(aStImpField[nY], 1, 2) == "L2"
1200             nPos := AScan( aSL2[nI] , {|x| x[1] == aStImpField[nY]} )
1201             If nPos > 0
1202                 Aadd( aSL2PE, aSL2[nI][nPos] )
1203             EndIf
1204         EndIf
1205     Next
1206
1207     //-----
1208     // Trata o Desconto no Total do Orcamento
1209     // Orcamento com desconto no total importado passa a ter o valor do desconto rateado entre os itens, ou seja,
1210     // o sistema registra o item do orcamento já com o desconto no total, como se fosse um desconto no item
1211     // Caso na telka de conferência seja cancelado algum item os descontos concedidos no total na Retaguarda serão desconsiderados
1212     //-----
1213     If !lZeraPay
1214         nVlrDescIt := aSL2[nI][nL2_VALDESC][2]
1215         nDescProp := ((aSL2[nI][nL2_QUANT][2] * aSL2[nI][nL2_PRCTAB][2]) - nVlrDescIt) * ( nPercDesc / 100 )
1216         nDescProp := STBRound( nDescProp, nDecDesc )
1217         nDesTotProp := nDescProp
1218     Else
1219         nVlrDescIt := 0
1220         nDescProp := 0
1221         nDescProp := 0
1222         nDesTotProp := 0
1223     EndIf
1224
1225     //Caso seja o ultimo item do Orcamento Importado verifica se nao houve residuo(sobra) no rateio
1226     If nI == Len(aSL2)
1227         If Abs(nDesTotProp) > 0
1228             nDescProp += nDesTotProp
1229         EndIf
1230     EndIf
1231
1232     //Inclui como desconto no item, o valor de desconto total vindo do Orcamento que foi importado.
1233     aSL2[nI][nL2_VALDESC][2] := aSL2[nI][nL2_VALDESC][2] + nDescProp
1234
1235     // Verifica se o item é fiscal
1236     LjGrvLog("Importa_Orcamento:STBImportSale","Verifica se item é fiscal")
1237     lItemFiscal := lIsRetPost .Or. STBISIsFiscal( aSL2[nI], lTemReserv )
1238     LjGrvLog("Importa_Orcamento:STBImportSale","Verifica se item é fiscal. Retorno(lItemFiscal):",lItemFiscal)
1239
1240
1241     //Validacao Garantia Estendida e Serviço Financeiro

```

```

1242 If (ExistFunc("STBIsGarEst") .AND. STBIsGarEst(aSL2[nI][nL2_PRODUTO][2])) .OR.;
1243     (lFinServ .AND. STBIsFinService(aSL2[nI][nL2_PRODUTO][2]))
1244     lItemFiscal := .F.
1245 EndIf
1246
1247 /*/
1248     Busca item na base de dados
1249 /*/
1250 aItem := STWFindItem( aSL2[nI][nL2_PRODUTO][2] )
1251
1252 // Caso nao encontre o item na base de dados, sera feito seu cadastro
1253 If !aItem[ITEM_ENCONTRADO]
1254     LjGrvLog("Importa_Orcamento:STBImportSale","Item Nao encontrado na base, chama rotina: STBUpdProducts")
1255     STBUpdProducts( aSL2[nI][nL2_PRODUTO][2] )
1256 EndIf
1257
1258 LjGrvLog( "ORCAMENTO PDV: " + STDGPBasket('SL1','L1_NUM'), "Item 01" + Str(nNextItem) )
1259 LjGrvLog( "ORCAMENTO PDV: " + STDGPBasket('SL1','L1_NUM'), "Codigo " + aSL2[nI][nL2_PRODUTO][2] )
1260 LjGrvLog( "ORCAMENTO PDV: " + STDGPBasket('SL1','L1_NUM'), "Preco " + Alltrim(Str(aSL2[nI][nL2_PRCTAB][2])) )
1261
1262 If nL2_CODBAR > 0 .And. !Empty(aSL2[nI][nL2_CODBAR][2])
1263     cCodProdReg := aSL2[nI][nL2_CODBAR][2]
1264     LjGrvLog( "ORCAMENTO PDV: " + STDGPBasket('SL1','L1_NUM'), "Codigo de barras:" + Alltrim(cCodProdReg) )
1265 Else
1266     cCodProdReg := aSL2[nI][nL2_PRODUTO][2]
1267 EndIf
1268
1269 //Verificar se há item de código de garantia do produto
1270 If nL2_ITEMGAR > 0 .And. !Empty(aSL2[nI][nL2_ITEMGAR][2])
1271     cCodItemGar := aSL2[nI][nL2_ITEMGAR][2]
1272 EndIf
1273
1274 While !lRegistrou
1275
1276     LjGrvLog("Importa_Orcamento:STBImportSale", "Registro do Item Importado - Inicio "+;
1277         "Item:"+cValToChar(nNextItem)+;
1278         ";Cliente:"+cCliCode+;
1279         ";Loja:"+cCliStore+;
1280         ";Codigo:"+cCodProdReg+;
1281         ";ValorDesconto:"+cValToChar(aSL2[nI][nL2_VALDESC][2])+;
1282         ";TES:"+cValToChar(aSL2[nI][nL2_TES][2])+;
1283         ";PrecoTab:"+cValToChar(aSL2[nI][nL2_PRCTAB][2]))
1284
1285     // Registra Item
1286     lItemRegister := STWItemReg(
1287         nNextItem           , ; // Item
1288         cCodProdReg         , ; // Codigo Prod
1289         cCliCode            , ; // Codigo Cli
1290         cCliStore           , ; // Loja Cli
1291         1                   , ; // Moeda
1292         aSL2[nI][nL2_VALDESC][2] , ; // Valor desconto
1293         "V"                 , ; // Tipo desconto ( Percentual ou Valor )
1294         NIL                 , ; // Item adicional?

```

```

1294          aSL2[nI][nL2_TES][2]      , ;      // TES
1295          cCliType                    , ;      // Tipo do cliente (A1_TIPO)
1296          lItemFiscal                 , ;      // Registra item no cupom fiscal?
1297          aSL2[nI][nL2_PRCTAB][2]    , ;      // Preço
1298          "IMP"                       , ;      // Indica que é importação de orçamento
1299          lInfoCNPJ                   , ;      // Se deve imprimir o CNPJ, informacao sera passada na abertura do cupom
1300          ,,,,,,,,,,                  , ;      // lRecovery,nSecItem,lServFinal,lProdBonif,lListProd,cCodList,cCodListIt,cCodMens,cEntrega
1301          cCodItemGar,,               , ;      // Código do item de produto garantia vinculado, Id do item Relacionado, Vale Presente
1302          aSL2[nI][nL2_KIT][2])      , ;      // Código do kit utilizado para lançar o item
1303
1304      If !lItemRegister
1305          If !MSGYESNO( STR0008, STR0009 )    //"Erro ao registrar o item, deseja tentar novamente?""###"ERRO"
1306              lRet := .F.
1307              lRegistrou := .T.
1308          Else
1309              STBSetQuant( aSL2[nI][nL2_QUANT][2] )
1310          EndIf
1311      Else
1312          lRegistrou := .T.
1313      EndIf
1314
1315  EndDo
1316
1317  lRegistrou := .F.
1318
1319  If !lRet
1320      Exit //Caso ocorra algum erro ao registrar o item não é necessário executar os passos seguintes
1321  EndIf
1322
1323  LjGrvLog( "ORCAMENTO PDV 2 : " + STDGPBasket('SL1','L1_NUM'), "Codigo " + aSL2[nI][nL2_PRODUTO][2] )
1324
1325  // Numero do orcamento da retaguarda..
1326  aSL2[nI][AScan( aSL2[nI] , { |x| x[1] == "L2_NUMORIG" } )][2] := cNumOrig
1327
1328  LjGrvLog( "ORCAMENTO PDV 3 : " + STDGPBasket('SL1','L1_NUM'), "cNumOrig " + cNumOrig )
1329
1330
1331  /*/
1332      Atualiza Cesta
1333  /*/
1334  STBISL2Refresh( aSL2[nI] , nNextItem, lEmitenFe )
1335
1336  LjGrvLog( "ORCAMENTO PDV: " + STDGPBasket('SL1','L1_NUM'), "Item 02" + Str(nNextItem) )
1337
1338
1339  // Atualiza os campos customizados
1340  If Len(aSL2PE) > 0
1341      LjGrvLog( "ORCAMENTO PDV: " + STDGPBasket('SL1','L1_NUM'), "Tamanho do aSL2PE " + Str(Len(aSL2PE)) )
1342      For nX := 1 To Len(aSL2PE)
1343          STDSPBasket( "SL2", aSL2PE[nX][1], aSL2PE[nX][2], nNextItem )
1344      Next
1345  EndIf

```

```

1346
1347 // Atualiza Banco de Dados
1348 LjGrvLog( "ORCAMENTO PDV: " + STDGPBasket('SL1','L1_NUM'), "====inicio grava item (Import)====" + aSL2[nI][nL2_PRODUTO][2] )
1349 STDSaveSale(nNextItem)
1350 LjGrvLog( "ORCAMENTO PDV: " + STDGPBasket('SL1','L1_NUM'), "====Fim grava item (Import) =====")
1351
1352 /*/
1353 PAF-ECF: Verifica se item foi cancelado na Retaguarda enquanto Orcamento,
1354 para realizar o cancelamento logo apos a impressao do Item no ECF
1355 /*/
1356 If STFGetCfg("lPafEcf") .AND. aSL2[nI][AScan( aSL2[nI] , { |x| x[1] == "L2_VENDIDO" } )][2] == "N"
1357 /*/
1358 Realiza Cancelamento do Item
1359 /*/
1360 LjGrvLog("Importa_Orcamento:STBImportSale",">>> NC <<< Cancela Item")
1361 STWLastCancel(lImportSale)
1362 EndIf
1363
1364 Next nI
1365
1366 EndIf
1367
1368 Iif ((!lDesligaRD .And. lCRdesItTt),;
1369 (STIClearDisc(.T.),;
1370 totvs.protheus.retail.desconto.RegraDescProdutoTotal.LjCallCalcRegDescProdTotal("TOTVSPDV", STDGPBasket("SL1","L1_CLIENTE"), STDGPBasket("SL1","L1_LOJA"), cTabPad)),;
1371 Nil)
1372 /*/
1373 Atualiza campos do cabeçalho
1374 /*/
1375 If lRet
1376
1377 /*/
1378 Indica que já importou algum orçamento. Para multi-orçamentos, varrer o L2
1379 /*/
1380 aSL1[AScan( aSL1 , { |x| x[1] == "L1_NUMORIG" } )][2] := cNumOrig
1381
1382 aSL1[AScan( aSL1 , { |x| x[1] == "L1_OPERACA" } )][2] := "C" // Nao sei pq mas deixei, quem souber fala aí
1383
1384 LjGrvLog("Importa_Orcamento:STBImportSale","Chama rotina: STBISL1Refresh")
1385
1386 STBISL1Refresh( aSL1 )
1387
1388 /*/
1389 Chamada componente de frete
1390 /*/
1391 LjGrvLog("Importa_Orcamento:STBImportSale","Chama rotina: STWAddFreight")
1392 STWAddFreight(aSL2)
1393
1394 /*/
1395 Chamada Acrescimo Financeiro
1396 /*/
1397 nScanJuros := AScan( aSL1 , { |x| x[1] == "L1_JUROS" } )

```

```

1398
1399   If SL1->(ColumnPos("L1_VLRJUR")) > 0
1400
1401       nScanVLJur := AScan( aSL1 , { |x| x[1] == "L1_VLRJUR" } )
1402       If nScanVLJur > 0
1403           STFSetTot( "L1_VLRJUR" , aSL1[nScanVLJur][2])
1404           STDSPBasket("SL1","L1_VLRJUR",STDGPBasket('SL1','L1_VLRJUR') + aSL1[nScanVLJur][2])
1405       Else
1406           STFSetTot( "L1_VLRJUR",0)
1407           STDSPBasket("SL1","L1_VLRJUR",0)
1408       EndIf
1409
1410       AADD ( aAcre , STDGPBasket("SL1","L1_VLRJUR"))
1411       AADD ( aAcre , aSL1[nScanJuros][2])
1412
1413   Else
1414       /*
1415       Calcula o valor do acrescimo (Legado)
1416       A função abaixo irá retornar o valor correto apenas para juros simples ou multi-negociação
1417       Caso seja Condição negociada (juros composto ou Price) o valor total da venda não irá bater com o Total da SL4
1418       */
1419       aAcre := STBDiscConvert( aSL1[nScanJuros][2] , 'P' )
1420       LjGrvLog("Importa_Orcamento:STBImportSale","0 campo L1_VLRJUR não existe na sua base de dados, a falta do campo afetará o cálculo do total da venda.")
1421   EndIf
1422
1423   If aAcre[1] > 0           //Caso houver acréscimo
1424       STBConfJur(@aAcre[1],STDGPBasket( "SL1" , "L1_VLRTOT" ),aSL4)
1425   EndIf
1426
1427   LjGrvLog("Importa_Orcamento:STBImportSale","Chama rotina: STWAddIncrease")
1428   STWAddIncrease( aAcre[1] , aSL1[nScanJuros][2] )
1429
1430   /*
1431   Atualiza Totalizador
1432   */
1433   LjGrvLog("Importa_Orcamento:STBImportSale","Chama rotina: STFRefTot")
1434   STFRefTot()
1435
1436 EndIf
1437
1438 /* Atualiza SL4 */
1439 //Desabilitei a linha abaixo porque o pagamento na cesta de vendas 'e inserido dentro da funcao STIUpdBask
1440 //que 'e chamada quando finaliza a venda pela funcao STBConfPay()
1441 //STBISL4Refresh( aSL4 )
1442
1443 LjGrvLog("Importa_Orcamento:STBImportSale","Parâmetro de retorno lRet: ", lRet)
1444
1445 Return lRet
1446
1447
1448 //-----
1449 /*{Protheus.doc} STBISL4Refresh

```

```

1450 Atualiza Item cesta
1451
1452 @param  aSL4           Array com a estrutura da Forma de Pagamento(SL4)
1453 @return  Nil
1454 /*/
1455 //-----
1456 Function STBISL4Refresh( aSL4 )
1457 Local nI                := 0                      // Contador
1458 Local nJ                := 0                      // Contador
1459 Local oModelCesta       := STDGPBModel()          // Model Cesta
1460 Local oModelSL4         := oModelCesta:GetModel("SL4DETAIL") // Model Formas de Pagamento
1461
1462 DEFAULT aSL4           := {}
1463
1464 If Empty(aSL4)
1465     aSL4 := aSL4Bkp
1466 Endif
1467
1468 For nI := 1 To Len(aSL4)
1469
1470     If oModelSL4:Length() < nI
1471         oModelSL4:AddLine(.T.)
1472     EndIf
1473
1474     oModelSL4:GoLine(nI)
1475
1476     For nJ := 1 To Len(aSL4[nI])
1477         Do Case
1478             Case aSL4[nI][nJ][1] == "L4_FILIAL"
1479                 oModelSL4:SetValue( "L4_FILIAL" , STDGPBasket( "SL1" , "L1_FILIAL" ) )
1480             Case aSL4[nI][nJ][1] == "L4_NUM"
1481                 oModelSL4:SetValue( "L4_NUM" , STDGPBasket( "SL1" , "L1_NUM" ) )
1482             Otherwise
1483                 If aSL4[nI][nJ][1] <> "R_E_C_N_0_" .AND. aSL4[nI][nJ][1] <> "R_E_C_D_E_L_"
1484                     oModelSL4:SetValue( aSL4[nI][nJ][1] , aSL4[nI][nJ][2] )
1485                 Endif
1486             EndCase
1487         Next nJ
1488     Next nI
1489 Next nI
1490
1491 Return Nil
1492
1493
1494 //-----
1495 /*/{Protheus.doc} STBISL2Refresh
1496 Atualiza Item cesta
1497
1498 @param  aSL2           Array com a estrutura do Item(SL2)
1499 @param  nLine          Linha do Item
1500 @return  Nil
1501 @obs

```

```

1502 @sample
1503 /**
1504 //-----
1505 Function STBISL2Refresh( aSL2 , nLine, lEmiteNFe )
1506
1507 Local lStImpField   := ExistBlock("STIMPFIELD")
1508 Local aStImpField   := {}
1509 Local nI            := 0
1510 Local aFieldsNot    := {} // Campos que nao devem receber a informacao da retaguarda, pois serao alimentados no Totvs PDV
1511 Local aTipoSx3      := {} //Tipo do campo no dicionario de dados
1512 Local oModel        := STDGPBModel() //Model
1513 Local lStQuant      := ExistBlock("STQUANT") //PE STQUANT
1514
1515 Default aSL2        := {} // Array SL2
1516 Default nLine       := 0 // Numero da linha
1517
1518 aAdd( aFieldsNot, PadR( "L2_FILIAL" , 10) )
1519 aAdd( aFieldsNot, PadR( "L2_NUM" , 10) )
1520 aAdd( aFieldsNot, PadR( "L2_VENDIDO", 10) )
1521
1522 If Len(aSL2) > 0 .AND. nLine > 0
1523
1524 //Garante que está posicionado no item correto
1525 oModel:GetModel("SL2DETAIL"):GoLine(nLine)
1526
1527 //Seta as infomacoes da tabela SL2 do orcamento importado da Retaguarda
1528 For nI := 1 To Len(aSL2)
1529     If aScan( aFieldsNot, PadR( aSL2[nI][1], 10) ) == 0
1530         If AllTrim(aSL2[nI][1]) <> "R_E_C_N_O_" .AND. AllTrim(aSL2[nI][1]) <> "R_E_C_D_E_L_" .AND. "L2_ITEM" <> UPPER(AllTrim(aSL2[nI][1]))
1531
1532             //Verifica o tipo do campo no dicionario SX3 - caso retorne um array vazio é pq o campo so existe na retaguarda.
1533             aTipoSx3 := TamSX3(aSL2[nI][1])
1534             If Len(aTipoSx3) == 0
1535                 LjGrvLog( "L1_NUM: " + STDGPBasket('SL1','L1_NUM'), "Campo '"+aSL2[nI][1]+' não localizado na base local (PDV)" )
1536             EndIf
1537
1538             If Len(aTipoSx3) > 0 .AND. aTipoSx3[3] == "D" .AND. Len(aSL2[nI]) > 1 .AND. ValType(aSL2[nI][2]) == "C"
1539                 aSL2[nI][2] := sTOd(aSL2[nI][2])
1540                 LjGrvLog( "L1_NUM: " + STDGPBasket('SL1','L1_NUM'), "Tipo de dado ajustado " + aSL2[nI][1])
1541             EndIf
1542
1543             // Se o PE STQUANT estiver compilado, significa que caso tenha alguma alteração no preço unitário e/ou na
1544             // quantidade, já foram feitas no STWItemReg(), por isso não pode pegar do aSL2 para atualizar o Bascket
1545             If (!lStQuant .OR. !(aSL2[nI][1] == "L2_PRCTAB" .OR. aSL2[nI][1] == "L2_QUANT"));
1546                 .AND. !(lEmiteNFe .AND. aSL2[nI][1] == "L2_CF")
1547                 STDSPBasket( "SL2", aSL2[nI][1], aSL2[nI][2], nLine )
1548             Endif
1549         EndIf
1550     EndIf
1551 Next
1552
1553

```

```

1554
1555   If !Empty( aSL2[AScan( aSL2 , { |x| x[1] == "L2_VENDIDO" } )][2] )
1556     STDSPBasket( "SL2" , "L2_VENDIDO" , aSL2[AScan( aSL2 , { |x| x[1] == "L2_VENDIDO" } )][2] , nLine )
1557   EndIf
1558
1559   /*
1560     STIMPFIELD - Ponto de Entrada para permitir a importação de campos adicionais
1561   */
1562   If lStImpField
1563     aStImpField := ExecBlock( "STIMPFIELD",.F.,.F.)
1564     If ValType(aStImpField) <> "A"
1565       aStImpField := {}
1566     EndIf
1567
1568     //organiza o vetor para que os campos sejam sequenciais (L1 > L2)
1569     ASort(aStImpField)
1570
1571     For nI := 1 To Len(aStImpField)
1572       If SubStr(aStImpField[nI], 1, 2) == "L2"
1573         nPos := Ascan( aSL2, { |x| x[1] == aStImpField[nI] } )
1574         If nPos > 0 .AND. UPPER(AllTrim(aSL2[nPos][1] )) <> "L2_ITEM"
1575           STDSPBasket( "SL2", aSL2[nPos][1], aSL2[nPos][2], nLine )
1576         EndIf
1577       EndIf
1578     Next
1579   EndIf
1580
1581 Else
1582   LjGrvLog( "L1_NUM: " + STDGPBasket('SL1','L1_NUM'), "Item nao sera atualizado ")
1583 EndIf
1584
1585 Return Nil
1586
1587
1588
1589 //-----
1590 /*{Protheus.doc} STBISL1Refresh
1591 Atualiza cabeçalho cesta
1592
1593 @param  aSL1          Array com a estrutura do Item(SL1)
1594 @return Nil
1595 @obs
1596 @sample
1597 */
1598 //-----
1599 Static Function STBISL1Refresh( aSL1 )
1600
1601 Local aRefreshFields      := {}           // Array com os campos a atualizar
1602 Local nI                  := 0           // Contador
1603 Local x
1604 Local nCampo              := 0           //posição do campo
1605

```

```

1606 Default aSL1          := {}
1607
1608 ParamType 0 Var        aSL1          As Array          Default {}
1609
1610 If Len(aSL1) > 0
1611
1612     AADD( aRefreshFields , "L1_OPERACA"      )
1613     AADD( aRefreshFields , "L1_MIDIA"        )
1614     AADD( aRefreshFields , "L1_NUMORIG"      )
1615     AADD( aRefreshFields , "L1_JUROS"        )
1616     AADD( aRefreshFields , "L1_NUMATEN"      ) //Campo TMK
1617
1618     /*
1619     Campos referentes a Frete / Transporte
1620     */
1621     AADD( aRefreshFields , "L1_TRANSP"      )
1622     AADD( aRefreshFields , "L1_ENDCOB"      )
1623     AADD( aRefreshFields , "L1_BAIRROC"      )
1624     AADD( aRefreshFields , "L1_MUNC"        )
1625     AADD( aRefreshFields , "L1_CEPC"        )
1626     AADD( aRefreshFields , "L1_ESTC"        )
1627     AADD( aRefreshFields , "L1_ENDENT"      )
1628     AADD( aRefreshFields , "L1_BAIRROE"      )
1629     AADD( aRefreshFields , "L1_MUNE"        )
1630     AADD( aRefreshFields , "L1_CEPE"        )
1631     AADD( aRefreshFields , "L1_ESTE"        )
1632
1633     If SL1->(ColumnPos('L1_VEICUL1')) > 0
1634         AADD( aRefreshFields , "L1_VEICUL1"      )
1635     EndIf
1636     /*
1637     Campos referentes a Frete / Dados Complementares
1638     */
1639     AADD( aRefreshFields , "L1_VOLUME"      )
1640     AADD( aRefreshFields , "L1_ESPECIE"      )
1641     AADD( aRefreshFields , "L1_MARCA"        )
1642     AADD( aRefreshFields , "L1_NUMERO"      )
1643     AADD( aRefreshFields , "L1_PLIQUI"      )
1644     AADD( aRefreshFields , "L1_PBRUTO"      )
1645     AADD( aRefreshFields , "L1_PLACA"        )
1646     AADD( aRefreshFields , "L1_UFPLACA"      )
1647     AADD( aRefreshFields , "L1_ESPECI1"      )
1648     /*
1649     Campos referente a Frete / Valores
1650     */
1651     AADD( aRefreshFields , "L1_FRETE"        )
1652     AADD( aRefreshFields , "L1_SEGURO"      )
1653     AADD( aRefreshFields , "L1_DESPESA"      )
1654     aAdd( aRefreshFields , "L1_TIPOJUR"      )
1655
1656     /*
1657     Campos referentes a Comissao/Vendedor

```

```

1658  /*/
1659  AADD( aRefreshFields , "L1_VEND"      )
1660  AADD( aRefreshFields , "L1_COMIS"    )
1661
1662  /*/
1663      Campos referentes a Impostos
1664  /*/
1665  AADD( aRefreshFields , "L1_VALICM"    )
1666  AADD( aRefreshFields , "L1_VALISS"    )
1667  AADD( aRefreshFields , "L1_VALIPI"    )
1668
1669  /*/
1670      Atualiza os campos na cesta
1671  /*/
1672  For nI := 1 To Len(aRefreshFields)
1673
1674      nCampo := AScan( aSL1 , { |x| x[1] == aRefreshFields[nI] } ) // valida se o campo existe no array aSL1
1675
1676      //Caso o campo não exista no array com os valores ele não prossegue
1677      If nCampo > 0
1678
1679          STDSPBasket( "SL1" , aRefreshFields[nI] , aSL1[nCampo][2] )
1680
1681      EndIf
1682
1683  Next nI
1684
1685 EndIf
1686
1687 Return Nil
1688
1689
1690 //-----
1691 /*{Protheus.doc} STBISIsFiscal
1692 Verifica se o Item é ou não item fiscal
1693
1694
1695 @param aSL2, Array, Array com a estrutura do Item(SL2)
1696 @param lTemReserv, Lógico, Indica que tem item de entrega/reserva no orçamento
1697
1698 @return Lógico, Retorna se o Item é fiscal
1699 /*/
1700 //-----
1701 Static Function STBISIsFiscal( aSL2, lTemReserv )
1702
1703 Local lItemFiscal      := .F.      // Retorna se o item eh fiscal(vai para o cupom fiscal), ou nao fiscal(Cupom nao fiscal)
1704 Local lContinue        := .T.      // Fluxo logico função
1705
1706 /*/
1707 1)
1708 Fiscal Se:
1709     - Usa impressora fiscal

```

```

1710     Não Fiscal se:
1711         - Não Usa Impressora Fiscal
1712
1713     1.1)
1714     Se usa Impressora Fiscal
1715     Não Fiscal se:
1716         A) Possui reserva e Entrega <> "2" - (Retira)
1717         B) Item de Lista de Presente - desabilitado
1718         C) Item de Garantia Estendida
1719
1720 /*/
1721
1722
1723 If Len(aSL2) > 0
1724     lContinue := .T.
1725 Else
1726     lContinue := .F.
1727 EndIf
1728
1729 If lContinue
1730
1731     Do Case
1732
1733         /*/
1734         A) Possui reserva
1735         /*/
1736         Case !Empty(      aSL2[AScan( aSL2 , { |x| x[1] == "L2_RESERVA"   }   )][2] ) .AND.      ;
1737                 aSL2[AScan( aSL2 , { |x| x[1] == "L2_ENTREGA"   }   )][2] <> "2"      //2 - RETIRA
1738
1739                 lItemFiscal := .F.
1740                 LjGrvLog("Importa_Orcamento:STBISIsFiscal","A) Possui reserva")
1741
1742         /*/
1743         B) Item de Lista de Presente
1744         /*/
1745         Case .F.
1746
1747             /*/
1748             !Empty( aSL2[AScan( aSL2 , { |x| x[1] == "L2_CODLPRE"   }   )][2] ) // Indica que veio de lista de presentes -> Não Fiscal
1749             Atualmente esse verificação não eh utilizada, pois os itens de lista caem na verificação A
1750             /*/
1751
1752             /*/
1753             OBS: Originalmente, os orçamentos de lista de presente vem com L2_RESERVA preenchido com "S"(Mesmo sem Reserva,
1754             com lista tipo crédito), caindo no item A para que sejam destinados ao cupom nao fiscal, para implementação dessa
1755             verificação deve ser alterada a gravação da retaguarda posteriormente.
1756             Lista de Presentes pode haver itens do tipo Retira que gera cupom fiscal
1757             /*/
1758
1759             /*lItemFiscal := .F. */
1760
1761         /*/

```

```

1762         C) Item de Garantia Estendida
1763     /*/
1764     Case .F.
1765
1766         /*/
1767         OBS: Garantia Estendida nao irá sair mais nesta versão(maio 2012), ficará pra mais tarde
1768         /*/
1769
1770     /*/
1771     D) Item de Servico
1772     Verifica se esta ativa a implementacao de venda com itens de "produto" e itens de "servico" em Notas Separadas (RPS), parametro MV_LJPRDSV
1773 /*/
1774 Case SuperGetMv("MV_LJPRDSV",.F.,.F.) .And. LjIsTesISS( aSL2[AScan( aSL2 , { |x| x[1] == "L2_NUM"} )][2], aSL2[AScan( aSL2 , { |x| x[1] == "L2_TES"} )][2] ) .And. ;
1775 ( IIf(STFGetCfg("lUseECF"), !lTemReserv, ((ExistFunc("Lj7RPSNew") .And. Lj7RPSNew()) .Or. !lTemReserv) ) )
1776
1777     lItemFiscal := .F.
1778     LjGrvLog("Importa_Orcamento:STBISIsFiscal","D) Item de Serviço")
1779
1780 /*/
1781 E) Item ENTREGA C/ PEDIDO S/ RESERVA
1782 /*/
1783 Case Empty(aSL2[AScan(aSL2, {|x|x[1] == "L2_RESERVA"} )][2]) .AND. ;
1784     aSL2[AScan(aSL2, {|x|x[1] == "L2_ENTREGA"} )][2] == "5" // 5- ENTREGA C/ PEDIDO S/ RESERVA
1785
1786     lItemFiscal := .F.
1787     LjGrvLog("Importa_Orcamento:STBISIsFiscal","E) Item ENTREGA C/ PEDIDO S/ RESERVA")
1788
1789
1790 Otherwise
1791
1792     lItemFiscal := .T.
1793
1794 EndCase
1795 EndIf
1796
1797 LjGrvLog("Importa_Orcamento:STBISIsFiscal","Parâmetro de retorno lItemFiscal: ", lItemFiscal)
1798
1799 Return lItemFiscal
1800
1801
1802 //-----
1803 /*/{Protheus.doc} STBISCanImport
1804 Valida se pode importar multiplos orçamentos
1805
1806 @param
1807 @return lRet          Retorna se pode importar orçamento
1808 @obs
1809 @sample
1810 /*/
1811 //-----
1812 Function STBISMultSales()
1813

```

```

1814 Local lRet      := .T.      // Retorno função
1815
1816 /*/
1817 Se ja tem L1_NUMORIG quer dizer que algum orçamento ja foi importado
1818 /*/
1819 If !( Empty(STDGPBasket("SL1","L1_NUMORIG")) )
1820
1821     If STFGetCfg("lPafEcfc") .OR. !( SuperGetMv( "MV_LJMLTOR" , NIL , .T. ) )
1822
1823         lRet := .F.
1824
1825     EndIf
1826
1827 EndIf
1828
1829 LjGrvLog("Importa_Orcamento:STBISMultSales","Parâmetro de retorno lRet: ", lRet)
1830
1831 Return lRet
1832
1833 //-----
1834 /*/{Protheus.doc} STBFormatOrcs
1835 Formata os orcamento para inclusao das formas de pagamento
1836
1837 @param
1838 @return
1839 @obs
1840 @sample
1841 /*/
1842 //-----
1843 Function STBFormatOrcs()
1844
1845 Local aOrc      := STIGetOrc()      //Orcamento que foi importado
1846 Local aRet      := Array(5,6)      //Retorno da funcao 1o - R$; 2o - CD|FI; 3o - CC; 4o - CH;5o em diante - Outras genericas
1847 Local nI        := 0               //Variavel de loop
1848 Local nValCash  := 0               //Valor do dinheiro
1849 Local nValCc    := 0               //Valor do credito
1850 Local nValCh    := 0               //Valor dos cheques
1851 Local nValCd    := 0               //Valor do debito
1852 Local nValFi    := 0               //Valor do financiado
1853 Local cData     := ""              //Data do pagamento
1854 Local nParcCh   := 0               //Parcelas do cheque
1855 Local nParcCc   := 0               //Parcelas do cartao credito
1856 Local nParcCd   := 0               //Parcelas do cartao debito
1857 Local nParcFi   := 0               //Parcelas do financiado
1858 Local cFormaPg  := ""              //Forma de pagamento da importação
1859 Local nValForm  := 0               //Valor de formas de pagamentos
1860 Local nParcForm := 0               //Valor de parcelas de pagamento
1861 Local nPos      := 0               //Posição do elemento do array
1862 Local cMvCondPad := Iif (ExistFunc("STBCondPad"), STBCondPad(),SuperGetMv('MV_CONDPAD',.F.,'')) //Condicao de pagamento padrao
1863 Local cMvSimbl  := SuperGetMV("MV_SIMB1")           // Simbolo da moeda da venda (parcela em dinheiro)
1864 Local aCondicoes := STDCondPg()                     //Recebe as condicoes de pagamento
1865 Local nX        := 0               //Variável de loop

```

```

1866 Local nJ          := 0
1867 Local nY          := 0
1868 Local lImpPgtOrc   := ExistFunc("STBSFormImp") //Se importa as formas de pgto de orçamento original
1869 Local nPosCred     := 0                      // Posicao do campo L1_CREDITO
1870 Local nCredito     := 0                      // Valor total do credito utilizado no orçamento
1871 Local cFormaId     := ""
1872 Local cIdCCAnt     := "."                   // Se MV_TEFMULT .F. ele retorna L4_FORMAID vazio
1873 Local cIdCDAnt     := "."                   // Se MV_TEFMULT .F. ele retorna L4_FORMAID vazio
1874 Local nContCC      := 0
1875 Local nContCD      := 0
1876 Local aPagCC       := {}
1877 Local aPagCD       := {}
1878 Local nPosForma    := 0 //Posicao da Forma
1879 Local nPosFrmId     := 0 //Posicao da Forma ID
1880 Local nPosData      := 0 //Posicao da data
1881 Local nParcCo       := 0 //Parcelas do convenio
1882 Local aPgOrcImp    := {}//Pagamentos do Orcamento
1883
1884 For nJ := 1 To Len(aOrc)
1885
1886   If Len(aOrc[nJ][3]) > 0
1887     nPosForma    := aScan(aOrc[nJ][3][1], { |x| x[1] == "L4_FORMA"}) //Posicao da Forma
1888     nPosFrmId    := aScan(aOrc[nJ][3][1], { |x| x[1] == "L4_FORMAID"}) //Posicao da Forma ID
1889     nPosData     := aScan(aOrc[nJ][3][1], { |x| x[1] == "L4_DATA"}) //Posicao da Forma ID
1890
1891     If nPosForma > 0 .AND. nPosFrmId > 0 .AND. nPosData > 0
1892       ASort(aOrc[nJ][3] ,,, { |x, y| x[nPosForma,2]+ x[nPosFrmId,2]+x[nPosData,2] < y[nPosForma,2]+y[nPosFrmId,2]+y[nPosData,2] } )
1893     EndIf
1894   EndIf
1895
1896   For nI := 1 To Len(aOrc[nJ][3])
1897
1898     If Empty(cMvCondPad) //Tratamento para nao trazer o valor do pagamento da importação
1899       Exit
1900     EndIf
1901
1902     cFormaPg := AllTrim(aOrc[nJ][3][nI][aScan(aOrc[nJ][3][nI], { |x| x[1] == "L4_FORMA"})][2])
1903
1904     Do Case
1905       Case cFormaPg == 'R$'
1906         nValCash += aOrc[nJ][3][nI][aScan(aOrc[nJ][3][nI], { |x| x[1] == "L4_VALOR"})][2]
1907         cData    := aOrc[nJ][3][nI][aScan(aOrc[nJ][3][nI], { |x| x[1] == "L4_DATA"})][2]
1908
1909         aRet[1][1] := nValCash
1910         If Empty(aRet[1][2])
1911           aRet[1][2] := cData
1912         EndIf
1913         aRet[1][3] := 'R$'
1914
1915       Case cFormaPg == 'CH'
1916
1917         nValForm    := aOrc[nJ][3][nI][aScan(aOrc[nJ][3][nI], { |x| x[1] == "L4_VALOR"})][2]

```

```

1918         nValCh      += nValForm
1919         cData       := a0rc[nJ][3][nI][aScan(a0rc[nJ][3][nI], { |x| x[1] == "L4_DATA"})][2]
1920         cFormaId    := a0rc[nJ][3][nI][aScan(a0rc[nJ][3][nI], { |x| x[1] == "L4_FORMAID"})][2]
1921         nParcCh += 1
1922
1923         aRet[4][1] := nValCh
1924         If Empty(aRet[4][2])
1925             aRet[4][2] := cData
1926         EndIf
1927         aRet[4][3] := 'CH'
1928         aRet[4][4] := nParcCh
1929
1930         //Tratamento para guardar as informações dos pagamentos em CH
1931         //do orçamento importado, para que seus valores sejam mantidos no PDV.
1932         If lImpPgt0rc
1933             STBSFormImp({cFormaPg,nValForm,cData,nParcCh,cFormaId})
1934         EndIf
1935
1936     Case cFormaPg == 'CC'
1937
1938         nValForm    := a0rc[nJ][3][nI][aScan(a0rc[nJ][3][nI], { |x| x[1] == "L4_VALOR"})][2]
1939         cData       := a0rc[nJ][3][nI][aScan(a0rc[nJ][3][nI], { |x| x[1] == "L4_DATA"})][2]
1940         cFormaId    := a0rc[nJ][3][nI][aScan(a0rc[nJ][3][nI], { |x| x[1] == "L4_FORMAID"})][2]
1941
1942         If cFormaId <> cIdCCAnt
1943             AAdd(aPagCC, Array(6))
1944             nValCc := nValForm
1945             cIdCCAnt:= cFormaId
1946             nParcCc := 1
1947             nContCC += 1
1948         ElseIf cFormaId == cIdCCAnt
1949             nValCc += nValForm
1950             nParcCc += 1
1951         EndIf
1952
1953         aPagCC[nContCC][1] := nValCc
1954
1955         If Empty(aPagCC[nContCC][2])
1956             aPagCC[nContCC][2] := cData
1957         EndIf
1958
1959         aPagCC[nContCC][3] := 'CC'
1960         aPagCC[nContCC][4] := nParcCc
1961
1962         aRet[3] := aPagCC
1963
1964         //Tratamento para guardar as informações dos pagamentos em CC
1965         //do orçamento importado, para que seus valores sejam mantidos no PDV.
1966         If lImpPgt0rc
1967             STBSFormImp({cFormaPg,nValForm,cData,nParcCc,cFormaId})
1968         EndIf
1969

```

```

1970      Case cFormaPg == 'CD'
1971
1972          nValForm      := a0rc[nJ][3][nI][aScan(a0rc[nJ][3][nI], { |x| x[1] == "L4_VALOR"})][2]
1973          cData         := a0rc[nJ][3][nI][aScan(a0rc[nJ][3][nI], { |x| x[1] == "L4_DATA"})][2]
1974          cFormaId      := a0rc[nJ][3][nI][aScan(a0rc[nJ][3][nI], { |x| x[1] == "L4_FORMAID"})][2]
1975
1976          If cFormaId <> cIdCDAnt
1977              AAdd(aPagCD, Array(6))
1978              nValCd     := nValForm
1979              cIdCDAnt:= cFormaId
1980              nParcCd   := 1
1981              nContCD += 1
1982          ElseIf cFormaId == cIdCDAnt
1983              nValCd     += nValForm
1984              nParcCd   += 1
1985          EndIf
1986
1987          aPagCD[nContCD][1] := nValCd
1988
1989          If Empty(aPagCD[nContCD][2])
1990              aPagCD[nContCD][2] := cData
1991          EndIf
1992
1993          aPagCD[nContCD][3] := 'CD'
1994          aPagCD[nContCD][4] := nParcCd
1995
1996          aRet[2] := aPagCD
1997
1998          //Tratamento para guardar as informações dos pagamentos em CD
1999          //do orçamento importado, para que seus valores sejam mantidos no PDV.
2000          If lImpPgtOrc
2001              STBSFormImp({cFormaPg,nValForm,cData,nParcCd,cFormaId})
2002          EndIf
2003
2004      Case cFormaPg == 'FI'
2005
2006          nValForm      := a0rc[nJ][3][nI][aScan(a0rc[nJ][3][nI], { |x| x[1] == "L4_VALOR"})][2]
2007          nValFi        += nValForm
2008          cData         := a0rc[nJ][3][nI][aScan(a0rc[nJ][3][nI], { |x| x[1] == "L4_DATA"})][2]
2009          cFormaId      := a0rc[nJ][3][nI][aScan(a0rc[nJ][3][nI], { |x| x[1] == "L4_FORMAID"})][2]
2010          nParcFi      += 1
2011
2012          aRet[5][1] := nValFi
2013          If Empty(aRet[5][2])
2014              aRet[5][2] := cData
2015          EndIf
2016          aRet[5][3] := 'FI'
2017          aRet[5][4] := nParcFi
2018
2019          //Tratamento para guardar as informações dos pagamentos em FI
2020          //do orçamento importado, para que seus valores sejam mantidos no PDV.
2021          If lImpPgtOrc

```

```

2022         STBSFormImp({cFormaPg,nValForm,cData,nParcFi,cFormaId})
2023     EndIf
2024
2025     Otherwise
2026
2027         //Tratamento para formas de pagamentos genericas (diferentes das citadas acima) ou
2028         //formas de pagamento criadas pelo usuario.
2029         nValForm := aOrc[nJ][3][nI][aScan(aOrc[nJ][3][nI], { |x| x[1] == "L4_VALOR"})][2]
2030         cData := aOrc[nJ][3][nI][aScan(aOrc[nJ][3][nI], { |x| x[1] == "L4_DATA"})][2]
2031         nParcForm := 1
2032
2033         //Se foi pago com NCC na venda assistida e gravou como orçamento (sendo que não há nada no SL4), ler o SL1.
2034         //Caso a pessoa não escolheu nenhum NCC e resolveu ir para a forma de pagamento,
2035         // passará por aqui e aplicará o default como que está em MV_CONDPAD.
2036         //Pesquisando em SE4->E4_CODIGO com o valor MV_CONDPAD, obtenho o SE4->E4_FORMA.
2037         nPosCred := aScan(aOrc[nJ][1], { |x| x[1] == "L1_CREDITO"})
2038         If nPosCred > 0
2039             nCredito := aOrc[nJ][1][nPosCred][2]
2040         EndIf
2041
2042         If Len(aOrc[nJ][3]) = 1 .AND. nCredito > 0
2043             // Farei a correspondência cMvCondPad (E4_CODIGO) com a E4_FORMA. O resultado será o cFormaPg. Ex: 001=R$,002=CH...
2044             If (nX := aScan(aCondicoes,{|x| Alltrim(x[1]) == Alltrim(cMvCondPad) }) > 0
2045                 cFormaPg := Substr(aCondicoes[nX][3],1,2)
2046             Else
2047                 cFormaPg := "CH" //Default conforme regras da função STBChkTpCP() em STBPayCdPg.prw
2048             EndIf
2049             nValForm := nCredito
2050             cData := aOrc[nJ][1][aScan(aOrc[nJ][1], { |x| x[1] == "L1_EMISSAO"})][2]
2051             nParcForm := IIF(cFormaPg == cMvSimb1, 0, 1) // Se dinheiro, não há parcela.
2052         EndIf
2053
2054         If !Empty(cFormaPg) .AND. nValForm > 0
2055             If (nPos := aScan(aRet, {|x| UPPER(AllTrim(x[3])) == cFormaPg}, 6)) > 0 //Caso seja a mesma forma de pagto
2056                 aRet[nPos][1] += nValForm
2057                 aRet[nPos][4] += nParcForm
2058             Else
2059                 aAdd(aRet, Array(6))
2060                 nPos := Len(aRet)
2061
2062                 aRet[nPos][1] := nValForm
2063                 If Empty(aRet[nPos][2])
2064                     aRet[nPos][2] := cData
2065                 EndIf
2066                 aRet[nPos][3] := cFormaPg
2067                 aRet[nPos][4] := nParcForm
2068             EndIf
2069
2070             //Tratamento para guardar as informações dos pagamentos em CH
2071             //do orçamento importado, para que seus valores sejam mantidos no PDV.
2072             If lImpPgtOrc .AND. !(cFormaPg == 'CO')
2073

```

```

2074             STBSFormImp({cFormaPg,nValForm,cData,aRet[nPos][4],a0rc[nJ][3][nI][aScan(a0rc[nJ][3][nI], { |x| x[1] == "L4_FORMAID"})][2]})
2075         EndIf
2076     EndIf
2077
2078     If lImpPgtOrc .AND. cFormaPg == 'C0'
2079         nParcCo += 1
2080         STBSFormImp({cFormaPg,nValForm,cData,nParcCo,a0rc[nJ][3][nI][aScan(a0rc[nJ][3][nI], { |x| x[1] == "L4_FORMAID"})][2]})
2081     EndIf
2082
2083     EndCase
2084
2085     Next nI
2086 Next nJ
2087
2088 If Len(aRet) = 0
2089     LjGrvLog("Importa_Orcamento:STBFormatOrcs","Parâmetro de retorno aRet: Conteúdo vazio")
2090 EndIf
2091
2092 If ExistBlock("STISelForm")
2093     For nX := 1 To Len(aRet)
2094         If nX <> 2 .AND. nX <> 3 .AND. ValType(aRet[nX][1]) == "N"
2095             AADD(aPgOrcImp,{aRet[nX][3],aRet[nX][1],aRet[nX][4]})
2096         ElseIf ( nX == 2 .OR. nX == 3 ) .AND. ValType(aRet[nX][1]) == "A"// CC ou CD
2097             For nY := 1 To Len(aRet[nX])
2098                 AADD(aPgOrcImp,{aRet[nX][nY][3],aRet[nX][nY][1],aRet[nX][nY][4]})
2099             Next nY
2100         EndIf
2101     Next nX
2102
2103     SetOrcP0ri(aPgOrcImp)
2104 EndIf
2105
2106 Return aRet
2107
2108 //-----
2109 /*/{Protheus.doc} STBImpProduct
2110 Importa as informacoes do produto da retaguarda, para posteriormente cadastra-lo na base local
2111
2112 @param
2113 @return
2114 @obs
2115 @sample
2116 */
2117 //-----
2118 Function STBUpdProducts( cProdCode )
2119 Local aProduct      := {}      // -- Array com retorno da função STDIMPPROD
2120 Local aFieldsProd   := {}      // -- Campos que serão trazidos para a gravação no PDV
2121 Local lRet          := .F.     // -- Retorno da função
2122 Local aRetPE        := {}      // -- Retorno do P.E STSELFELD
2123 Local nI            := 0       // -- Variavel para For
2124
2125 DEFAULT cProdCode := ""

```

```

2126
2127 Aadd(aFieldsProd,"B1_FILIAL")
2128 Aadd(aFieldsProd,"B1_COD")
2129 Aadd(aFieldsProd,"B1_DESC")
2130 Aadd(aFieldsProd,"B1_TIPO")
2131 Aadd(aFieldsProd,"B1_UM")
2132 Aadd(aFieldsProd,"B1_LOCPAD")
2133 Aadd(aFieldsProd,"B1_CODBAR")
2134 Aadd(aFieldsProd,"B1_GRUPO")
2135 Aadd(aFieldsProd,"B1_BITMAP")
2136 Aadd(aFieldsProd,"B1_BALANCA")
2137 Aadd(aFieldsProd,"B1_POSIPI")
2138 Aadd(aFieldsProd,"B1_ORIGEM")
2139 Aadd(aFieldsProd,"B1_CEST")
2140 Aadd(aFieldsProd,"B1_GRTRIB")
2141 Aadd(aFieldsProd,"B1_PICMRET")
2142
2143 //Ponto de entrada para retornar campos adicionais no aCamposBusca
2144 If ExistBlock("STSELFIELD")
2145     aRetPE := ExecBlock("STSELFIELD",.F.,.F.,{"SB1"})
2146 EndIf
2147
2148 For nI := 1 To Len(aRetPE)
2149     /*Remover IF abaixo em 07/2020 -- Proteção, Agora o ponto de entrada é usado em mais de um ponto,
2150     anteriormente esse ponto de entrada era usado apenas no fonte STBCostumerSelection
2151     incluindo novos campos na tabela SA1.*/
2152     If SubStr(aRetPE[nI],1,3) == "B1_" .Or. SubStr(aRetPE[nI],1,3) == "b1_"
2153         aAdd(aFieldsProd, aRetPE[nI])
2154     EndIf
2155 Next nI
2156
2157 LjGrvLog("Importa_Orcamento:STBUpdProducts","Chama rotina remoto: STDIMPPROD")
2158 LjGrvLog("Importa_Orcamento:STBUpdProducts","Parâmetro cProdCode:      ", cProdCode)
2159 LjGrvLog("Importa_Orcamento:STBUpdProducts","Parâmetro aFieldsProd    ", aFieldsProd)
2160
2161 lRet := STBRemoteExecute( "STDIMPPROD", { cProdCode, aFieldsProd }, Nil, .F. , @aProduct )
2162
2163 LjGrvLog("Importa_Orcamento:STBUpdProducts","Parâmetro de retorno aProduct:", aProduct)
2164
2165 If lRet .AND. ValType( aProduct ) == "A" .AND. Len( aProduct ) > 0
2166     LjGrvLog("Importa_Orcamento:STBUpdProducts","Chama rotina : STBIncProducts")
2167     STBIncProducts( aProduct )
2168 Else
2169     lRet := .F.
2170 EndIf
2171
2172 LjGrvLog("Importa_Orcamento:STBUpdProducts","Parâmetro de retorno lRet   :", lRet)
2173
2174 Return lRet
2175
2176 //-----
2177 /*/{Protheus.doc} STBIncProducts

```

```

2178 Importa as informacoes do produto da retaguarda, para posteriormente cadastra-lo na base local
2179
2180 @param
2181 @return
2182 @obs
2183 @sample
2184 /*/
2185 //-----
2186 Static Function STBIncProducts( aProduct )
2187 Local nX := 0
2188
2189 DbSelectArea("SB1")
2190 SB1->(DbSetOrder())
2191 Begin Transaction
2192
2193     RecLock("SB1",.T.)
2194     For nX := 1 To Len(aProduct)
2195         Replace &("SB1->" + aProduct[nX,1]) With aProduct[nX,2]
2196     Next nX
2197     SB1->(MsUnlock())
2198
2199 End Transaction
2200
2201 LjGrvLog("Importa_Orcamento:STBIncProducts","Campos gravados na tabela SB1:", nX)
2202
2203 Return
2204
2205 //-----
2206 /*/{Protheus.doc} STBISRetPos
2207 Verifica se é orcamento (com itens de RETIRA POSTERIOR) apenas para finalizacao (impressao do Cupom Fiscal)
2208
2209 @param    aSL1          Array com a estrutura do Cabecalho do Orcamento (SL1)
2210 @return   lRet          Retorna se é orcamento (com itens de RETIRA POSTERIOR) apenas para finalizacao (impressao do Cupom Fiscal)
2211 @obs
2212 @sample
2213 /*/
2214 //-----
2215 Function STBISRetPos( aSL1 )
2216
2217 Local lRet          := .F.      // Retorna se é orcamento (com itens de RETIRA POSTERIOR) apenas para finalizacao (impressao do Cupom Fiscal)
2218 Local nL1_ORCRES    := 0
2219 Local nL1_PEDRES    := 0
2220
2221 nL1_ORCRES := AScan( aSL1, { |x| x[1] == "L1_ORCRES" } )
2222 nL1_PEDRES := AScan( aSL1, { |x| x[1] == "L1_PEDRES" } )
2223
2224 If nL1_ORCRES > 0 .And. nL1_PEDRES > 0
2225     If Empty(aSL1[nL1_PEDRES][2]) .And. !Empty(aSL1[nL1_ORCRES][2])
2226         lRet := .T.
2227     EndIf
2228 EndIf
2229

```

```

2230 LjGrvLog("Importa_Orcamento:STBIsRetPos","Parâmetro de retorno lRet      :", lRet)
2231
2232 Return lRet
2233
2234 //-----
2235 /*{Protheus.doc} STBISL4ADD
2236 Adiciona itens da forma de pagamento do orcamento filho importado (com somente itens retira posterior)
2237
2238 @param   aSL4              Array com a estrutura da Forma de Pagamento(SL4)
2239 @return  Nil
2240 @obs
2241 @sample
2242 */
2243 //-----
2244 Static Function STBISL4ADD( aSL4 )
2245
2246 Local nI                      := 0                      // Contador
2247 Local oModelCesta            := STDGPBModel()           // Model Cesta
2248 Local oModelSL4              := oModelCesta:GetModel("SL4DETAIL") // Model Formas de Pagamento
2249 Local cFormaPG               := ""
2250 Local nValorPG               := 0
2251 Local lRet                   := .T.                      //retorno se add todos pagamentos
2252 Local lAddPay                := .T.                      //controle se add cada pagamentos
2253
2254 For nI := 1 To Len(aSL4)
2255
2256     cFormaPG := aSL4[nI][AScan( aSL4[nI] , { |x| x[1] == "L4_FORMA" } )][2]
2257     nValorPG := aSL4[nI][AScan( aSL4[nI] , { |x| x[1] == "L4_VALOR" } )][2]
2258
2259     lAddPay := STIAddPay(AllTrim(cFormaPG), Nil, 1, Nil, Nil, nValorPG)
2260     lRet    := lRet .AND. lAddPay
2261
2262 Next nI
2263
2264 LjGrvLog("Importa_Orcamento:STBIsL4ADD","Parâmetro de retorno lRet      :", lRet)
2265
2266 Return lRet
2267
2268 //-----
2269 /*{Protheus.doc} STBISFinPos
2270 Importa, finaliza orcamento filho que contem apenas itens "Retira Posterior", imprimindo o Cupom Fiscal.
2271
2272 @param   aSale              Array com a estrutura SL1, SL1 e SL4
2273 @return  Nil
2274 @obs
2275 @sample
2276 */
2277 //-----
2278 Function STBISFinPos(aSale)
2279
2280 Local aSL4              := aSale[3]          // Pagamentos
2281 Local oMdl               := Nil              // Model Pgto

```

```

2282 Local oModelPay      := Nil           // Model Pgto
2283 Local oModelParc      := Nil           // Model Pgto
2284 Local nValNcc         := 0             // Valor de pagamento em NCC
2285
2286 LjGrvLog("Importa_Orcamento:STBISFinPos","Chama rotina: STBImportSale(aSale,.F.)")
2287 LjGrvLog("Importa_Orcamento:STBISFinPos","Parâmetro aSale : ", aSale)
2288 STBImportSale( aSale , .F. )
2289 oMdl := ModelPayme()
2290
2291 oModelPay := oMdl:GetModel('PARCELAS')
2292 oModelPay:Activate()
2293 oModelPay:ClearData()
2294 oModelPay:InitLine()
2295
2296 oModelParc := oMdl:GetModel('APAYMENTS')
2297 oModelParc:Activate()
2298 oModelParc:ClearData()
2299 oModelParc:InitLine()
2300
2301 //Adiciona itens da forma de pagamento do orcamento filho importado (com somente itens retira posterior)
2302 If STBISL4ADD( aSL4 )
2303     LjGrvLog("Importa_Orcamento:STBISFinPos","Chama rotina: STIConfPay(.T.)")
2304     If ExistFunc("STDGetValSL4")
2305         nValNcc := STDGetValSL4(aSL4)
2306         If nValNcc > 0
2307             STIAddPay("CR", Nil, 1, Nil, Nil, nValNcc)
2308         EndIf
2309     EndIf
2310     STIConfPay(.T.)
2311 Else
2312     STFMessage("STBISFinPos","POPUP", STR0010 ) //"Pagamento Inválido. A Venda será Cancelada"
2313     STFShowMessage("STBISFinPos")
2314     LjGrvLog("Importa_Orcamento:STBISFinPos","Chama rotina: STWCancelSale(.T.)")
2315     STWCancelSale( .T. ) //Cancela Venda Atual
2316     LjGrvLog("Importa_Orcamento:STBISFinPos","Chama rotina: STIRegItemInterface")
2317     STIRegItemInterface()
2318 EndIf
2319
2320 Return
2321
2322 //-----
2323 /*{Protheus.doc} STBIsImpOrc
2324 Retornar se é importação de Orçamento ou nao
2325
2326 @param
2327 @return
2328 */
2329 //-----
2330 Function STBIsImpOrc()
2331 //Se ja tem L1_NUMORIG quer dizer que algum orçamento ja foi importado
2332 Return (!(Empty(STDGPBasket("SL1","L1_NUMORIG"))))
2333

```

```

2334 //-----
2335 /*{Protheus.doc} STBImpGFld
2336 Resgata a relacao de campos utilizados na importacao de orcamento.
2337
2338 @param
2339 @return
2340 */
2341 //-----
2342 Function STBImpGFld(cTable)
2343 Local aRetFields := {}
2344
2345 Do Case
2346
2347     Case cTable == "SL1" //Campos da tabela SL1
2348
2349         aRetFields := aCmpsImpL1
2350
2351     Case cTable == "SL2" //Campos da tabela SL2
2352
2353         aRetFields := aCmpsImpL2
2354
2355     Case cTable == "SL4" //Campos da tabela SL4
2356
2357         aRetFields := aCmpsImpL4
2358
2359 EndCase
2360
2361 Return aRetFields
2362
2363 //-----
2364 /*{Protheus.doc} STBImpSFld
2365 Define a relacao de campos utilizados na importacao de orcamento.
2366
2367 @param
2368 @return
2369 */
2370 //-----
2371 Function STBImpSFld(cTable)
2372 Local aRetFields      := {}
2373 Local aStImpField     := {}
2374 Local nI              := 0
2375 Local aStructCmp      := {}
2376 Local cMVCODBAR := IIf(SuperGetMv("MV_CODBAR",,"N") == "S", .T., .F.) // Indica se imprime codigo de barras no cupom ao inves do codigo do produto
2377
2378 //Ponto de Entrada para permitir a importação de campos adicionais
2379 If ExistBlock( "STIMPFIELD" )
2380     aStImpField := ExecBlock( "STIMPFIELD",.F.,.F.)
2381     If ValType(aStImpField) <> "A"
2382         aStImpField := {}
2383     EndIf
2384
2385     //Organizamos o vetor para que os campos sejam sequenciais (L1 > L2)

```

```

2386     aSort(aStImpField)
2387 EndIf
2388
2389
2390 Do Case
2391
2392     Case cTable == "SL1" //Campos da tabela SL1
2393
2394         //-----
2395         //Campos Padrao do sistema que serao considerados na importacao de orcamento
2396         //-----
2397         aAdd( aRetFields, "L1_NUM"      )
2398         aAdd( aRetFields, "L1_VLRTOT"   )
2399         aAdd( aRetFields, "L1_CLIENTE"   )
2400         aAdd( aRetFields, "L1_LOJA"      )
2401         aAdd( aRetFields, "L1_TIPOCLI"   )
2402         aAdd( aRetFields, "L1_FILRES"    )
2403         aAdd( aRetFields, "L1_ORCRES"    )
2404         aAdd( aRetFields, "L1_PEDRES"    )
2405         aAdd( aRetFields, "L1_CONDPG"    )
2406         aAdd( aRetFields, "L1_TIPO"      )
2407         aAdd( aRetFields, "L1_NUMORC"    )
2408         aAdd( aRetFields, "L1_COODAV"    )
2409         aAdd( aRetFields, "L1_NUMORIG"    )
2410         aAdd( aRetFields, "L1_OPERACA"    )
2411         aAdd( aRetFields, "L1_DESCONT"    )
2412         aAdd( aRetFields, "L1_JUROS"     )
2413         aAdd( aRetFields, "L1_MIDIA"     )
2414         aAdd( aRetFields, "L1_NUMATEN"    )
2415         aAdd( aRetFields, "L1_TRANSP"    )
2416         aAdd( aRetFields, "L1_ENDCOB"    )
2417         aAdd( aRetFields, "L1_BAIRROC"    )
2418         aAdd( aRetFields, "L1_MUNC"      )
2419         aAdd( aRetFields, "L1_CEPC"      )
2420         aAdd( aRetFields, "L1_ESTC"      )
2421         aAdd( aRetFields, "L1_ENDENT"    )
2422         aAdd( aRetFields, "L1_BAIRROE"    )
2423         aAdd( aRetFields, "L1_MUNE"      )
2424         aAdd( aRetFields, "L1_CEPE"      )
2425         aAdd( aRetFields, "L1_ESTE"      )
2426         aAdd( aRetFields, "L1_VOLUME"    )
2427         aAdd( aRetFields, "L1_ESPECIE"    )
2428         aAdd( aRetFields, "L1_MARCA"     )
2429         aAdd( aRetFields, "L1_NUMERO"     )
2430         aAdd( aRetFields, "L1_PLIQUI"     )
2431         aAdd( aRetFields, "L1_PBRUTO"     )
2432         aAdd( aRetFields, "L1_PLACA"      )
2433         aAdd( aRetFields, "L1_UFPLACA"    )
2434         aAdd( aRetFields, "L1_TPFRET"     )
2435         aAdd( aRetFields, "L1_FRETE"      )
2436         aAdd( aRetFields, "L1_SEGURO"     )
2437         aAdd( aRetFields, "L1_DESPESA"    )

```

```

2438     aAdd( aRetFields, "L1_VEND"      )
2439     aAdd( aRetFields, "L1_VEND2"     )
2440     aAdd( aRetFields, "L1_VEND3"     )
2441     aAdd( aRetFields, "L1_COMIS"     )
2442     aAdd( aRetFields, "L1_VALICM"    )
2443     aAdd( aRetFields, "L1_VALISS"    )
2444     aAdd( aRetFields, "L1_VALIPI"    )
2445     aAdd( aRetFields, "L1_DOC"       )
2446     aAdd( aRetFields, "L1_DOCPED"    )
2447     aAdd( aRetFields, "L1_DTLIM"     )
2448     aAdd( aRetFields, "L1_TPORC"     )
2449     aAdd( aRetFields, "L1_DESCFIN"    )
2450     If SL1->(ColumnPos("L1_VLRJUR")) > 0
2451         aAdd( aRetFields, "L1_VLRJUR" )
2452     Endif
2453     aAdd( aRetFields, "L1_CGCCART"    )
2454     If SL1->(ColumnPos("L1_INDPRES")) > 0
2455         aAdd( aRetFields, "L1_INDPRES" )
2456     Endif
2457     aAdd( aRetFields, "L1_ESPECII"    )
2458     aAdd( aRetFields, "L1_TIPOJUR"    )
2459     aAdd( aRetFields, "L1_FORMPG"     )
2460     aAdd( aRetFields, "L1_DESCNF"     )
2461     aAdd( aRetFields, "L1_ICMSRET"    )
2462     aAdd( aRetFields, "L1_VLRLIQ"     )
2463     aAdd( aRetFields, "L1_VALMERC"    )
2464
2465     //-----
2466     //Campos Adicionais que serao considerados na importacao de orcamento
2467     // definidos no Ponto de Entrada "STIMPFIELD".
2468     //-----
2469     For nI := 1 To Len(aStImpField)
2470         If SubStr(aStImpField[nI], 1, 2) == "L1"
2471             If SL1->(FieldPos(aStImpField[nI])) > 0 //Verifica se o campos existe
2472                 aAdd( aRetFields, aStImpField[nI] )
2473             EndIf
2474         EndIf
2475     Next
2476
2477     Case cTable == "SL2" //Campos da tabela SL2
2478
2479     //-----
2480     //Campos Padrao do sistema que serao considerados na importacao de orcamento
2481     //-----
2482     aAdd( aRetFields, "L2_NUM"        )
2483     aAdd( aRetFields, "L2_PRODUTO"    )
2484     aAdd( aRetFields, "L2_QUANT"      )
2485     aAdd( aRetFields, "L2_PRCTAB"     )
2486     aAdd( aRetFields, "L2_VALDESC"    )
2487     aAdd( aRetFields, "L2_TES"       )
2488     aAdd( aRetFields, "L2_NUMORIG"    )
2489     aAdd( aRetFields, "L2_RESERVA"    )

```

```

2490      aAdd( aRetFields, "L2_ENTREGA" )
2491      aAdd( aRetFields, "L2_LOJARES" )
2492      aAdd( aRetFields, "L2_TURN0" )
2493      aAdd( aRetFields, "L2_CODLAN" )
2494      aAdd( aRetFields, "L2_LOCAL" )
2495      aAdd( aRetFields, "L2_TABELA" )
2496      aAdd( aRetFields, "L2_GARANT" )
2497      aAdd( aRetFields, "L2_NSERIE" )
2498      aAdd( aRetFields, "L2_VEND" )
2499      aAdd( aRetFields, "L2_VENDIDO" )
2500      aAdd( aRetFields, "L2_CODLPRE" )
2501      aAdd( aRetFields, "L2_ITLPRE" )
2502      aAdd( aRetFields, "L2_MSMLPRE" )
2503      aAdd( aRetFields, "L2_REMLPRE" )
2504      aAdd( aRetFields, "L2_CODCONT" )
2505      aAdd( aRetFields, "L2_VALFRE" )
2506      aAdd( aRetFields, "L2_SEGURO" )
2507      aAdd( aRetFields, "L2_DESPESA" )
2508      aAdd( aRetFields, "L2_FDTENTR" )
2509      aAdd( aRetFields, "L2_FDTMONT" )
2510      aAdd( aRetFields, "L2_CF" )
2511      If SL2->( ColumnPos("L2_ITEMCOB" ) ) > 0
2512          aAdd( aRetFields, "L2_ITEMCOB" ) //Servico Financeiro Vinculado a qual item
2513      EndIf
2514      If SL2->( ColumnPos("L2_PRDCOBE" ) ) > 0
2515          aAdd( aRetFields, "L2_PRDCOBE" ) //Código do Produto Cobertura
2516      EndIf
2517      If SL2->( ColumnPos("L2_INDPAR" ) ) > 0
2518          aAdd( aRetFields, "L2_INDPAR" ) //Comissão Parceiros que indicam a loja
2519      EndIf
2520      If cMVCODBAR
2521          aAdd( aRetFields, "L2_CODBAR" )
2522      EndIf
2523      If SL2->( ColumnPos("L2_ITEMGAR" ) ) > 0 //Número do item vinculado ao produto garantia
2524          aAdd( aRetFields, "L2_ITEMGAR" )
2525      EndIf
2526      If SL2->( ColumnPos("L2_VLGAPRO" ) ) > 0 //Validade da garantia
2527          aAdd( aRetFields, "L2_VLGAPRO" )
2528      EndIf
2529      aAdd( aRetFields, "L2_LOTECTL" )
2530      aAdd( aRetFields, "L2_NLOTE" )
2531      aAdd( aRetFields, "L2_LOCALIZ" )
2532      aAdd( aRetFields, "L2_KIT" ) // Código do kit do produto
2533      aAdd( aRetFields, "L2_ABATISS" ) // Abatimento de ISS
2534
2535      //-----
2536      //Campos Adicionais que serao considerados na importacao de orcamento
2537      // definidos no Ponto de Entrada "STIMPFIELD".
2538      //-----
2539      For nI := 1 To Len(aStImpField)
2540          If SubStr(aStImpField[nI], 1, 2) == "L2"
2541              If SL2->(FieldPos(aStImpField[nI])) > 0 //Verifica se o campos existe

```

```

2542             aAdd( aRetFields, aStImpField[nI] )
2543         EndIf
2544     EndIf
2545 Next
2546
2547 Case cTable == "SL4" //Campos da tabela SL4
2548
2549     //T0 D0
2550
2551 EndCase
2552
2553 //Atualiza o array de campos, incluindo a estrutura para cada campo
2554 For nI := 1 To Len(aRetFields)
2555     aStructCmp := TamSX3(aRetFields[nI])
2556     // Nome           , Tipo           , Tamanho       , Decimais
2557     aRetFields[nI] := {aRetFields[nI],aStructCmp[3],aStructCmp[1],aStructCmp[2]}
2558 Next
2559
2560 Return aRetFields
2561
2562
2563 //-----
2564 /*{Protheus.doc} STBConfJur
2565 O valor total em L1_VLRTOT vezes a porcentagem dos juros L1_JUROS pode não coincidir com o valor acordado na Venda Assistida, que é a soma dos valores da SL4.
2566 Então estabeleci uma margem de 0.25 para deixar igual à soma da SL4, pois interfere na confirmação da forma de pagamento importada pela Retaguarda
2567 (troco de centavos ou falta de centavos) e no cálculo de troco do SAT.
2568 @param nValJuros      Valor do Juros
2569 @param nValTotal      Valor total do cupom sem acrescimos
2570 @param aSL4
2571 @return
2572 */
2573 //-----
2574 Static Function STBConfJur(nValJuros,nValTotal,aSL4)
2575
2576 Local nDif      := 0    //Resultado
2577 Local nX        := 0    //Contador
2578 Local nTotSL4   := 0    //Soma da forma de pagamento
2579
2580 //Análise de SL4, considerando somente o L4_NUM selecionado
2581 nScan := AScan( aSL4[1] , { |x| x[1] == "L4_VALOR" } )
2582 For nX := 1 to Len(aSL4)
2583     If nScan > 0
2584         nTotSL4 += aSL4[nX,nScan,2]
2585     EndIf
2586 Next
2587
2588 nDif := nTotSL4 - (nValJuros + nValTotal)
2589
2590 If Abs(nDif) <= 0.25
2591     nValJuros := nValJuros + nDif
2592 EndIf
2593

```

```

2594 Return nil
2595
2596 //-----
2597 /*{Protheus.doc} STBLMPaSL4
2598 Limpa o conteudo da vareavel aSL4Bkp
2599
2600 @return
2601 */
2602 //-----
2603 Function STBLMPaSL4()
2604 aSL4Bkp := {}
2605 Return
2606
2607 //-----
2608 /*{Protheus.doc} STBConfJur
2609 retorna o conteudo da vareavel aSL4Bkp
2610
2611 @return aSL4Bkp
2612 */
2613 //-----
2614 Function STBGetaSL4()
2615
2616 Return aSL4Bkp
2617
2618 //-----
2619 /*{Protheus.doc} STBISValCred
2620 Valida Credito na importacao de orçamento
2621
2622 @param  ntipo := Indica se a Funcao foi chamada via: 1 - Salvar Orcamento, 2 - Salvar como Venda, 3 - Salvar como Pedido
2623         aSL1  := Array com a estrutura do SL1. Bidimensional: Campo, Valor
2624         aSL2  := Array com a estrutura do SL2. Bidimensional: Campo, Valor
2625         aSL4  := Array com a estrutura do SL4. Multidimensional
2626         .T.   := Indica que é chamda do Totvspdv
2627 @return lRet      Retorna a validacao de Credito
2628 */
2629 //-----
2630 Static Function STBISValCred(ntipo,aSL1 , aSL2 , aSL4)
2631
2632 Local aArea      := GetArea()      // Armazena alias corrente
2633 Local aRet       := {}
2634
2635 aRet := LJ7AvalCred(ntipo,aSL1 , aSL2 , aSL4, .T.)
2636
2637 RestArea(aArea)
2638
2639 Return aRet
2640
2641
2642 //-----
2643 /*{Protheus.doc} STBGetPgtCPF
2644 Lê o conteúdo da variável lPerguntouCPF
2645 lPerguntouCPF: Se por acaso fez a pergunta "Deseja Imprimir CPF/CNPJ no Comprovante da Venda ?" uma vez - STR0007

```

```

2646
2647 @return lPerguntouCPF
2648 */
2649 //-----
2650 Function STBGetPgtCPF()
2651
2652 Return (lPerguntouCPF)
2653
2654
2655 //-----
2656 /*/{Protheus.doc} STBSetPgtCPF
2657 Atribui o conteudo da variável lPerguntouCPF
2658
2659 @return
2660 */
2661 //-----
2662 Function STBSetPgtCPF(lVar)
2663
2664 Default lVar := .F.
2665
2666 lPerguntouCPF := lVar
2667
2668 Return nil
2669
2670 //-----
2671 /*/{Protheus.doc} STBGetCPFRet
2672 Lê o conteúdo da variável lRetCpf
2673 lRetCPF: Opção escolhida após a pergunta "Deseja Imprimir CPF/CNPJ no Comprovante da Venda ?" - STR0007 - .T. (Sim) ou .F. (Não)
2674
2675 @return lRetCpf
2676 */
2677 //-----
2678 Function STBGetCPFRet()
2679
2680 Return lRetCpf
2681
2682 //-----
2683 /*/{Protheus.doc} STBSetCPFRet
2684 Atribui o conteudo da variável lRetCpf
2685 lRetCPF: Opção escolhida após a pergunta "Deseja Imprimir CPF/CNPJ no Comprovante da Venda ?" - STR0007 - .T. (Sim) ou .F. (Não)
2686
2687 @return NIL
2688 */
2689 //-----
2690 Function STBSetCPFRet(lVar)
2691
2692 Default lVar := .F.
2693
2694 lRetCPF := lVar
2695
2696 Return nil
2697

```

```

2698 //-----
2699 /*{Protheus.doc} STBGetDscImp
2700 Lê o conteúdo da variável nDescImp
2701 nDescImp: Armazena o valor de desconto do total da venda dos orçamentos importados
2702
2703 @param nenhum
2704 @return nDescImp, numerico
2705 */
2706 //-----
2707 Function STBGetDscImp()
2708
2709 Return nDescImp
2710
2711 //-----
2712 /*{Protheus.doc} STBSetDscImp
2713 Atribui o conteúdo da variável nDescImp
2714
2715 @param nenhum
2716 @return NIL
2717 */
2718 //-----
2719 Function STBSetDscImp(nVal)
2720
2721 Default nVal := 0
2722
2723 nDescImp := nVal
2724
2725 Return nil
2726
2727 //-----
2728 /*{Protheus.doc} STBFPgtNPe
2729 Valida se existe forma de pagamento não permitido no Totvs PDV
2730
2731 @param aSL4 - array - dados das formas de pagamentos utilizados no orçamento
2732 @param aFPgtNPerm - array - Retorna as formas de pagamentos do orçamento que não são permitidas no Totvs Pdv
2733 @return lRet -logico, se existe formas de pagamentos não permitidas no orçamento, .T., senão .F.
2734 */
2735 //-----
2736 Static Function STBFPgtNPe(aSL4,aFPgtNPerm)
2737 Local lRet := .F.
2738 Local nX := 0
2739
2740 For nX :=1 to Len(aSL4)
2741 //forma de pagamento BOL não é permitido usar no TOVS PDV.
2742 If AllTrim(aSL4[nX])[AScan( aSL4[nX] , { |x| x[1] == "L4_FORMA" } )][2]) == 'BOL'
2743 AAdd(aFPgtNPerm, 'BOL')
2744 lRet := .T.
2745 Endif
2746 Next nX
2747
2748 Return lRet
2749

```

```

2750 //-----
2751 /*/{Protheus.doc} STBExitRes
2752 Verifica se tem item de "Entrega" / "Reserva" na venda
2753
2754 @param
2755
2756 @param aSL2, Array, Array com os itens do orçamento importado.
2757
2758 @return Lógico, Retorna se tem item de "Entrega" / "Reserva" na venda
2759 */
2760 //-----
2761 Static Function STBExitRes(aSL2)
2762 Local lExist      := .F.
2763 Local nI          := 0
2764 Local nL2_RESERVA := AScan( aSL2[1] , { |x| x[1] == "L2_RESERVA" } )
2765 Local nL2_ENTREGA := AScan( aSL2[1] , { |x| x[1] == "L2_ENTREGA" } )
2766
2767 For nI := 1 To Len(aSL2)
2768     If (!Empty( aSL2[nI][nL2_RESERVA][2] ) .And. aSL2[nI][nL2_ENTREGA][2] <> "2") .OR.;
2769         (Empty( aSL2[nI][nL2_RESERVA][2] ) .And. aSL2[nI][nL2_ENTREGA][2] == "5") // 5= ENTREGA C/ PEDIDO S/ RESERVA
2770         lExist := .T.
2771         Exit
2772     EndIf
2773 Next nI
2774
2775 Return lExist
2776
2777 /*/{Protheus.doc} SetOrcP0ri
2778 Seta valor ao array statico aOrcPayOri
2779 @type Function
2780 @param aPay, array, pagamentos do orçamento impotado
2781 */
2782 Function SetOrcP0ri(aPay)
2783 aOrcPayOri := aPay
2784 Return
2785
2786 /*/{Protheus.doc} GetOrcP0ri
2787 Retorna array statico aOrcPayOri, que contém os pagamentos do orçamento importado
2788 @type Function
2789 @return aOrcPayOri, array, pagamentos do orçamento impotado
2790 */
2791 Function GetOrcP0ri()
2792 Return aOrcPayOri
2793
2794 //-----
2795 /*/{Protheus.doc} STBCliRet
2796 Defini como será gerado a gravação do cliente na importação do orçamento,
2797 Com base na configuração do parâmetro MV_LJIPCOR.
2798 MV_LJIPCOR - 0 Segue o legado sempre atualiza com os dados que venho
2799 MV_LJIPCOR - 1 Identifica se o cliente é diferente do padrão e atualiza com o cliente do PDV
2800 @param cCliente    caracter    Código do cliente informado
2801 @param cLoja       caracter    Loja do cliente informado

```

```
2802 @param cTipoCli      character  Tipo do cliente Ex: Pessoa Fisca ou Juridica.
2803 @return Nil
2804 /*/
2805 //-----
2806
2807 Static Function STBCliRet(cCliente , cLoja, cTipoCli )
2808
2809 Local nMVLJIPCOR      := SuperGetMV("MV_LJIPCOR",.F., 0) // Define a ação de acordo com a configuração do parâmetro MV_LJIPCOR
2810 Local lAtuCli        := (SuperGetMV("MV_CLIPAD",.F. ) <> STDGPBasket("SL1","L1_CLIENTE") .And. cCliente <> STDGPBasket("SL1","L1_CLIENTE")) // valida se atualiza Cliente
2811
2812 If nMVLJIPCOR == 1 .And. lAtuCli
2813     cCliente := STDGPBasket("SL1","L1_CLIENTE")
2814     cLoja    := STDGPBasket("SL1","L1_LOJA")
2815     cTipoCli := STDGPBasket("SL1","L1_TIPOCLI")
2816 EndIf
2817
2818 Return
```

Componentes/ImportSale/STDImportSale.prw

1041 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "FWMVCDEF.CH"
4
5 //-----
6 /*{Protheus.doc} STDISSL1Get
7 Busca o Orcamento pelo numero e retorna o cabeçalho num array
8 @param   cNumOrc           Numero do Orcamento
9 @param   lSL1Locked        Retorna .T. se a SL1 estiver com lock
10 @return  aSL1              Retorna cabeçalho do orçamento
11 @obs
12 @sample
13 */
14 //-----
15 Function STDISSL1Get( cNumOrc, lSL1Locked )
16
17 Local aArea           := GetArea()           // Armazena alias corrente
18 Local cOptionsTemp    := GetNextAlias()      // Armazena Proximo alias disponível
19 Local aSL1            := {}                  // Retorno da função
20 Local nI              := 0                   // Contador
21 Local cNumExp          := ""                  // Expressao L1_NUM
22 Local cQryFields      := ""
23 Local aFieldsImp      := {}
24 Local axTesSql        := {}
25 Local caxTesSql       := ""
26
27 Default cNumOrc        := ""
28 Default lSL1Locked     := .F.
29
30 ParamType 0 Var       cNumOrc                As Character    Default ""
31
32 If FindFunction("STBImpGFld")
33   aFieldsImp := STBImpGFld("SL1") //Retorna os campos que serao utilizados na query para importacao do orcamento
34 Else
35   aFieldsImp := SL1->(dbStruct()) //Todos os campos
36 EndIf
37
38 For nI := 1 To Len( aFieldsImp )
39   cQryFields += aFieldsImp[nI][1]+","
40 Next nI
41 //Retira a Ultima Virgula
42 cQryFields := Left(cQryFields,Len(cQryFields)-1)
43 cQryFields := "%"+cQryFields+"%"
44
45 If !Empty(cNumOrc)
46   If Len(Alltrim(cNumOrc)) == TamSX3("L1_NUM")[1]
47     cNumExp := "%'" + cNumOrc + "'%"
48   Else
49     cNumExp := "%'" + StrZero(Val(cNumOrc),TamSX3("L1_NUM")[1]) + "'%"
50   EndIf
51 Else
52   cNumExp := "%'"

```

```

53 EndIf
54
55 BeginSql ALIAS cOptionsTemp
56
57         SELECT %Exp:cQryFields%
58         FROM %table:SL1% SL1
59         WHERE
60             L1_FILIAL      = %xfilial:SL1%           AND
61             L1_NUM         = %exp:cNumExp%           AND
62             L1_PEDRES      = " "                     AND
63             SL1.%NotDel%
64 EndSql
65
66 axTesSql :=   GetLastQuery()
67
68 caxTesSql := axTesSql[2]
69
70 LjGrvLog( " 03 Importação do PDV Orcamento " + cNumOrc, "SQL", Nil )
71 LjGrvLog( " 04 A sql = " + caxTesSql + "Fim", "SQL", Nil )
72
73 //Ajuste no tipo de dado retornado da query (Numerico, Data, Lógico)
74 For nI := 1 To Len(aFieldsImp)
75     If aFieldsImp[nI,2]<>"C" .And. aFieldsImp[nI,2]<>"M" //M=Memo Real
76         TcSetField(cOptionsTemp,aFieldsImp[nI,1],aFieldsImp[nI,2],aFieldsImp[nI,3],aFieldsImp[nI,4])
77     EndIf
78 Next nI
79
80 DbSelectArea(cOptionsTemp)
81
82 While !(cOptionsTemp)->(EOF())
83
84     For nI := 1 To FCount()
85         AADD( aSL1 , {FieldName(nI) , FieldGet(nI)} )
86
87     Next nI
88
89     lSL1Locked := STDValLock ((cOptionsTemp)->L1_NUM)
90
91     (cOptionsTemp)->(DbSkip())
92
93 EndDo
94
95 (cOptionsTemp)->( DbCloseArea() )
96
97 DbSelectArea( "SL1" ) // Seleção de tabela (preventiva)
98
99 RestArea(aArea)
100
101 Return aSL1
102
103
104 //-----

```

```

105 /*/{Protheus.doc} STDISPafSL1Get
106 Busca o Orcamento pela numeração de DAV/PRE e retorna o cabeçalho
107 @param  cNumOrc      Numero do Orcamento(L1_NUMORC)
108 @param  cTypeNumber   Tipo de Numeração. "D" -> DAV , "P" -> PREVENDA
109 @return  aSL1         Retorna cabeçalho do orçamento
110 @obs
111 @sample
112 */
113 //-----
114 Function STDISPafSL1Get( cNumOrc , cTypeNumber )
115
116 Local aArea           := GetArea()           // Armazena alias corrente
117 Local cOptionsTemp    := GetNextAlias()       // Armazena Proximo alias disponível
118 Local aSL1            := {}                  // Retorno da função
119 Local nI              := 0                   // Contador
120 Local cNumExp          := ""                  // Expressao L1_NUM
121 Local cTpOrc          := ""                  // Expressao L1_TPORC
122 Local cQryFields      := ""
123 Local aFieldsImp      := {}
124
125 Default cNumOrc       := ""
126 Default cTpOrc       := ""
127
128 ParamType 0 Var      cNumOrc                As Character    Default ""
129
130 If FindFunction("STBImpGFld")
131   aFieldsImp := STBImpGFld("SL1") //Retorna os campos que serao utilizados na query para importacao do orcamento
132 Else
133   aFieldsImp := SL1->(dbStruct()) //Todos os campos
134 EndIf
135
136 For nI := 1 To Len( aFieldsImp )
137   cQryFields += aFieldsImp[nI][1]+","
138 Next nI
139 //Retira a Ultima Virgula
140 cQryFields := Left(cQryFields,Len(cQryFields)-1)
141 cQryFields := "%"+cQryFields+"%"
142
143
144 If !Empty(cNumOrc)
145   cNumExp := "%'" + StrZero(Val(cNumOrc),TamSX3("L1_NUMORC"))[1]) + "'"
146 Else
147   cNumExp := "%'"
148 Endif
149 cTpOrc := "%'" + cTypeNumber + "'"
150
151 BeginSql ALIAS cOptionsTemp
152
153       SELECT %Exp:cQryFields%
154       FROM %table:SL1% SL1
155       WHERE
156           L1_FILIAL      =      %xfilial:SL1%           AND

```

```

157             L1_NUMORC      =   %exp:cNumExp%           AND
158             L1_TPORC       =   %exp:cTpOrc%             AND
159             SL1.%NotDel%
160 EndSql
161
162 //Ajuste no tipo de dado retornado da query (Numerico, Data, Lógico)
163 For nI := 1 To Len(aFieldsImp)
164     If aFieldsImp[nI,2]<>"C" .And. aFieldsImp[nI,2]<>"M" //M=Memo Real
165         TcSetField(cOptionsTemp,aFieldsImp[nI,1],aFieldsImp[nI,2],aFieldsImp[nI,3],aFieldsImp[nI,4])
166     EndIf
167 Next nI
168
169 DbSelectArea(cOptionsTemp)
170
171 While !(cOptionsTemp)->(EOF())
172
173     For nI := 1 To FCount()
174
175         AADD( aSL1 , {FieldName(nI) , FieldGet(nI)} )
176
177     Next nI
178
179     (cOptionsTemp)->(DbSkip())
180
181
182 EndDo
183
184 (cOptionsTemp)->( DbCloseArea() )
185
186 DbSelectArea( "SL1" ) // Seleção de tabela (preventiva)
187
188 RestArea(aArea)
189
190 Return aSL1
191
192
193 //-----
194 /*{Protheus.doc} STDISSL2Get
195 Busca os itens do Orcamento pelo numero e retorna num array bidimensional
196
197 @param   cNumOrc           Numero do Orcamento
198 @param   lPafEcf           Indica se usa PafEcf
199 @return  aSL2              Retorna Itens do orçamento
200 @obs
201 @sample
202 */
203 //-----
204 Function STDISSL2Get( cNumOrc , lPafEcf )
205
206 Local aArea      := GetArea()    // armazena alias corrente
207 Local aSL2       := {}          // Retorno da funcao
208 Local cOptionsTemp := GetNextAlias() // Armazena Proximo alias disponível

```

```

209 Local aItens           := {}           // Armazena os itens(auxiliar do SL2)
210 Local nI               := 0           // Contador
211 Local cNumExp           := ""          // Expressao L1_NUM
212 Local cDeleted          := ""          // Status de deleção
213 Local cL1_Vendid        := PadR('N', TamSx3("L2_VENDIDO")[1]) // Tamanho do L2_VENDIDO
214 Local cQryFields        := ""
215 Local aFieldsImp        := {}
216
217 Default cNumOrc         := ""
218 Default lPafEcF         := .F.
219
220 ParamType 0 Var         cNumOrc         As Character      Default ""
221
222 If FindFunction("STBImpGFld")
223   aFieldsImp := STBImpGFld("SL2") //Retorna os campos que serao utilizados na query para importacao do orcamento
224 Else
225   aFieldsImp := SL2->(dbStruct()) //Todos os campos
226 EndIf
227
228 For nI := 1 To Len( aFieldsImp )
229   cQryFields += aFieldsImp[nI][1]+", "
230 Next nI
231 //Retira a Ultima Virgula
232 cQryFields := Left(cQryFields,Len(cQryFields)-1)
233 cQryFields := "%"+cQryFields+"%"
234
235
236 If !Empty(cNumOrc)
237   If Len(Alltrim(cNumOrc)) == TamSX3("L2_NUM")[1]
238     cNumExp := "%'" + cNumOrc + "'"
239   Else
240     cNumExp := "%'" + StrZero(Val(cNumOrc),TamSX3("L2_NUM")[1]) + "'"
241   EndIf
242 Else
243   cNumExp := "%'"
244 EndIf
245
246 cDeleted := "%'*'"
247 cL1_Vendid := "%'" + cL1_Vendid + "'"
248
249 If lPafEcF
250   //No PAFECE não usar %NotDel% pois os itens serao
251   //impressos e cancelados de acordo com campo L2_VENDIDO
252   BeginSql ALIAS cOptionsTemp
253
254       SELECT %Exp:cQryFields%
255       FROM %table:SL2% SL2
256       WHERE
257           L2_FILIAL      = %xfilial:SL2%           AND
258           L2_NUM         = %exp:cNumExp%           AND
259           ( SL2.%NotDel%
260

```

```

261                ( SL2.D_E_L_E_T_ = %exp:cDeleted% AND
262                  SL2.L2_VENDID0 = %exp:cL1_Vendid%
263                )
264            )
265        EndSql
266    EndSql
267
268 Else
269
270     BeginSql ALIAS cOptionsTemp
271
272         SELECT %Exp:cQryFields%
273         FROM %table:SL2% SL2
274         WHERE
275             L2_FILIAL      = %xfilial:SL2%           AND
276             L2_NUM         = %exp:cNumExp%           AND
277             SL2.%NotDel%
278
279     EndSql
280
281 EndIf
282
283 //Ajuste no tipo de dado retornado da query (Numerico, Data, Lógico)
284 For nI := 1 To Len(aFieldsImp)
285     If aFieldsImp[nI,2]<>"C" .And. aFieldsImp[nI,2]<>"M" //M=Memo Real
286         TcSetField(cOptionsTemp,aFieldsImp[nI,1],aFieldsImp[nI,2],aFieldsImp[nI,3],aFieldsImp[nI,4])
287     EndIf
288 Next nI
289
290 DbSelectArea(cOptionsTemp)
291
292 While !(cOptionsTemp)->(EOF())
293
294     For nI := 1 To FCount()
295
296         AADD( aItens , {FieldName(nI) , FieldGet(nI)} )
297
298     Next nI
299
300     AADD( aSL2 , aItens )
301
302     aItens := {}
303
304     (cOptionsTemp)->(DbSkip())
305
306 EndDo
307
308 (cOptionsTemp)->( DbCloseArea() )
309
310 RestArea(aArea)
311
312 Return aSL2

```

```

313
314
315 //-----
316 /*{Protheus.doc} STDISSL4Get
317 Busca as formas de pagamento do Orcamento pelo numero e retorna num array bidimensional
318
319 @param   cNumOrc           Numero do Orcamento
320 @return  aSL4              Retorna formas de pagamento do orçamento
321 @obs
322 @sample
323 */
324 //-----
325 Function STDISSL4Get( cNumOrc )
326
327 Local aArea      := GetArea()      // Armazena alçiasl corrente
328 Local aSL4       := {}             // Retorno da funcao
329 Local cOptionsTemp := GetNextAlias() // Armazena Proximo alias disponível
330 Local aPay       := {}             // Armazena formas de pagamento(auxilial aSL4)
331 Local nI         := 0              // Contador
332
333 Default cNumOrc   := ""
334
335 ParamType 0 Var    cNumOrc          As Character    Default ""
336
337 If !Empty(cNumOrc)
338     If Len(Alltrim(cNumOrc)) == TamSX3("L4_NUM")[1]
339         cNumExp := "%" + cNumOrc + "'"
340     Else
341         cNumExp := "%" + StrZero(Val(cNumOrc),TamSX3("L4_NUM")[1]) + "'"
342     EndIf
343 Else
344     cNumExp := "''"
345 EndIf
346
347 BeginSql ALIAS cOptionsTemp
348
349         SELECT      *
350         FROM %table:SL4% SL4
351         WHERE
352             L4_FILIAL      = %xfilial:SL4%           AND
353             L4_NUM         = %exp:cNumExp%           AND
354             SL4.%NotDel%
355 EndSql
356
357 DbSelectArea(cOptionsTemp)
358
359 While !(cOptionsTemp)->(EOF())
360
361     For nI := 1 To FCount()
362
363         AADD( aPay , {FieldName(nI) , FieldGet(nI)} )
364

```

```

365     Next nI
366
367     AADD( aSL4 , aPay )
368
369     aPay := {}
370
371     (cOptionsTemp)->(DbSkip())
372
373 EndDo
374
375 (cOptionsTemp)->( DbCloseArea() )
376
377 RestArea(aArea)
378
379 Return aSL4
380
381
382 //-----
383 /*{Protheus.doc} STDISGetFieldOptions
384 Busca de opções de orçamento em outro ambiente
385 @param   cField           Campo utilizado para busca
386 @param   cFieldAssigned   Conteúdo do campo para busca
387 @param   aFieldsAdd       Campos adicionais a serem buscados
388 @return  aOptions         Opções de orçamentos encontrados, contendo n orçamentos. Retorna os campos
389 @return  aOptions[1]      Numero do Orcamento -> L1_NUM
390 @return  aOptions[2]      Numero da DAV/PRE      -> L1_NUMORC
391 @return  aOptions[3]      Nome do CLiente       -> A1_NOME
392 @obs
393 @sample
394 */
395 //-----
396 Function STDISGetFieldOptions( cField , cFieldAssigned , aFieldsAdd )
397
398 Local aOptions      := {}           // Retorno funcao
399 Local aAux          := {}           // Armazena aOptions temporario
400 Local cOptionsTemp  := GetNextAlias() // Armazena Proximo alias disponível
401 Local cFieldExp      := ""           // Expressao do campo preenchido
402 Local cAddSELECT    := ""           // Expressao com campos adicionais a buscar
403 Local nI            := 0            // Contador
404
405 Default cField      := ""
406 Default cFieldAssigned := ""
407 Default aFieldsAdd   := {}
408
409 ParamType 0 Var      cField      As Character   Default ""
410 ParamType 1 Var      cFieldAssigned As Character Default ""
411 ParamType 2 Var      aFieldsAdd   As Array      Default {}
412
413 cFieldAssigned := AllTrim(cFieldAssigned)
414
415 /*
416     Monta expressão dos campos adicionais

```

```

417 /*/
418 cAddSELECT := "%"
419 For nI := 1 To Len(aFieldsAdd)
420   If SL1->(ColumnPos(aFieldsAdd[nI])) > 0
421     cAddSELECT := cAddSELECT + " , " + aFieldsAdd[nI]
422   Else
423     aDel(aFieldsAdd, nI)
424     aSize(aFieldsAdd, nI-1)
425   EndIf
426 Next nI
427 cAddSELECT := cAddSELECT + "%"
428
429 If cField = "L1_NUM" .AND. !Empty(cFieldAssigned) .AND. At(";",cFieldAssigned) == 0
430   If Len(Alltrim(cFieldAssigned)) <> TamSX3("L1_NUM")[1]
431     cFieldAssigned := StrZero(Val(cFieldAssigned),TamSX3("L1_NUM")[1])
432   EndIf
433 EndIf
434
435 /*/
436   Monta Expressão do campo de pesquisa
437 /*/
438 If SuperGetMv("MV_LJORPAR",,.F.) // "Procura de orçamento por parte do conteudo de busca?" Utiliza LIKE
439   cFieldAssigned := StrTran(cFieldAssigned,"*","%")
440
441   If (cField == "A1_NOME" .OR. cField == "A1_NREDUZ")
442     cFieldExp := "%(A1_NOME LIKE ' ' + cFieldAssigned + ' ' OR " + ;
443               "A1_NREDUZ LIKE ' ' + cFieldAssigned + ' ') AND%"
444   Else
445     cFieldExp := "%" + cField + " LIKE ' ' + cFieldAssigned + ' ' AND%"
446   EndIf
447
448 Else
449   cFieldExp := "%" + cField + " = ' ' + cFieldAssigned + ' ' AND%"
450 EndIf
451
452
453
454 BeginSql ALIAS cOptionsTemp
455
456           %NoParser%
457
458           SELECT   L1_NUM , A1_NOME , L1_NUMORC %exp:cAddSELECT%
459
460           FROM %table:SL1% SL1
461           INNER JOIN %table:SA1% SA1 ON   A1_COD =   L1_CLIENTE      AND
462                                     A1_LOJA =   L1_LOJA
463
464           WHERE
465             L1_FILIAL      =   %xfilial:SL1%      AND
466             A1_FILIAL      =   %xfilial:SA1%      AND
467             L1_DTLIM       >=   %exp:DTOS(dDataBase)% AND
468             L1_DOC         =   " "                AND
469             L1_DOCPED      =   " "                AND

```

```

469      %exp:cFieldExp%
470      SL1.%notDel%
471      SA1.%notDel%
472      (
473          (
474              L1_SERIE = " "
475              L1_PDV = " "
476              L1_STORC <> "C"
477          ) OR
478          (
479              L1_RESERVA <> " "
480              L1_STATUS <> "D"
481          )
482      )
483 EndSql
484
485 LjGrvLog( "STDISGetFi", "SQL: " + GetLastQuery()[2] )
486
487
488 While !(cOptionsTemp)->(EOF())
489
490     // Verifica se existe arquivo informando que o orcamento ja foi importado
491     // So add se o arquivo nao existir
492     If !FR271HArq ( {{ (cOptionsTemp)->L1_NUM , "" }} , .T., Nil , .T. )[1]
493
494         AADD( aAux , (cOptionsTemp)->L1_NUM )
495         AADD( aAux , (cOptionsTemp)->L1_NUMORC )
496         AADD( aAux , AllTrim((cOptionsTemp)->A1_NOME) )
497
498         For nI := 1 To Len(aFieldsAdd)
499
500             AADD( aAux , (cOptionsTemp)->&(aFieldsAdd[nI]) )
501
502         Next nI
503
504         AADD( aOptions , aAux )
505
506         aAux := {}
507
508     EndIf
509
510     (cOptionsTemp)->(DbSkip())
511
512 EndDo
513
514 (cOptionsTemp)->( DbCloseArea() )
515
516 DbSelectArea( "SL1" ) // Seleção de tabela (preventiva)
517
518 Return aOptions
519
520

```

```

521 //-----
522 /*{Protheus.doc} STDISGetOptions
523 Busca das opções de orçamento
524 @param  nOption      Opção de pesquisa. 2 -> Todos 3 -> Data
525 @param  lIsPafPdv     Conteúdo do campo para busca
526 @param  lSearchAll    Busca Todas as vendas PAFECE
527 @param  aFieldsAdd    Campos adicionais a serem buscados
528 @param  dDataMov      Data do movimento a realizar a Busca PAFECE
529 @return  aOptions     Opções de orçamentos encontrados, contendo n orçamentos. Retorna os campos
530 @return  aOptions[1]   Numero do Orcamento -> L1_NUM
531 @return  aOptions[2]   Numero da DAV/PRE      -> L1_NUMORC
532 @return  aOptions[3]   Nome do Cliente       -> A1_NOME
533 @obs
534 @sample
535 */
536 //-----
537 Function STDISGetOptions(  nOption , lIsPafPdv , lSearchAll , aFieldsAdd  ,;
538                          dDataMov
539                          )
540 Local aOptions      := {}           // Retorno da funcao
541 Local cDateExp      := ""           // Armazena Expressão de data de emissao
542 Local cOrderBy      := ""           // Expressao ordenação
543 Local cAddSELECT     := ""           // Expressao com campos adicionais a buscar
544 Local cOptionsTemp  := GetNextAlias() // Pega o proximo Alias Disponivel
545 Local nI            := 0            // Contador
546 Local aAux          := {}           // Auxiliar
547
548 Default nOption      := 0
549 Default lIsPafPdv    := .F.
550 Default lSearchAll   := .F.
551 Default aFieldsAdd   := {}
552 Default dDataMov     := dDataBase
553
554 ParamType 0 Var      nOption      As Numeric      Default 0
555 ParamType 1 Var      lSearchAll    As Logical      Default .F.
556 ParamType 2 Var      aFieldsAdd    As Array        Default {}
557
558 /*/
559   Monta expressão dos campos adicionais
560 /*/
561 If Len(aFieldsAdd) > 0
562   cAddSELECT := ""
563   For nI := 1 To Len(aFieldsAdd)
564     If SL1->(ColumnPos(aFieldsAdd[nI])) > 0
565       cAddSELECT := cAddSELECT + " , " + aFieldsAdd[nI]
566     Else
567       aDel(aFieldsAdd, nI)
568       aSize(aFieldsAdd, nI-1)
569     EndIf
570   Next nI
571   cAddSELECT := cAddSELECT + ""
572 EndIf

```

```

573
574 If nOption == 3 // DATA
575     cDateExp := "%L1_EMITSAO      = '" + DTOS(dDataBase)+'      AND%"
576 EndIf
577
578 cOrderBy := "%L1_NUM , A1_NOME, L1_NUMORC%"
579
580 Do Case
581
582     Case lIsPafPdv .AND. lSearchAll // PAF - Redução Z
583
584         // TODO: Verificar se tem que tirar a DateExp da query quando for LOJA160
585         BeginSql alias cOptionsTemp
586
587             %NoParser%
588
589             SELECT    L1_NUM , A1_NOME, L1_NUMORC %exp:cAddSELECT%
590
591             FROM %table:SL1% SL1 ,%table:SA1% SA1
592
593             WHERE     L1_FILIAL      = %xfilial:SL1%           AND
594                     A1_FILIAL      = %xfilial:SA1%           AND
595                     L1_DTLIM       < %exp:DTOS(dDataMov)%     AND
596                     L1_SERIE       = " "                     AND
597                     L1_DOC         = " "                     AND
598                     L1_PDV        = " "                     AND
599                     L1_STORC      <> "C"                     AND
600                     L1_ORCRES     = " "                     AND
601                     L1_CLIENTE    = A1_COD                   AND
602                     L1_LOJA       = A1_LOJA                   AND
603                     L1_SITUA      = " "                     AND
604                     L1_TPORC      = "P"                     AND
605                     SL1.%notDel%  AND
606                     SA1.%notDel%
607
608             ORDER BY %exp:cOrderBy%
609         EndSql
610
611     Case lIsPafPdv // PAF
612         BeginSql alias cOptionsTemp
613
614             %NoParser%
615
616             SELECT    L1_NUM ,A1_NOME, L1_NUMORC %exp:cAddSELECT%
617
618             FROM %table:SL1% SL1 ,%table:SA1% SA1
619
620             WHERE     L1_FILIAL      = %xfilial:SL1%           AND
621                     A1_FILIAL      = %xfilial:SA1%           AND
622                     L1_DTLIM       >= %exp:DTOS(dDataBase)%  AND
623                     L1_CLIENTE    = A1_COD                   AND
624                     L1_LOJA       = A1_LOJA                   AND

```

```

625         L1_PEDRES      = " "      AND
626         L1_DOCPED      = " "      AND
627         L1_DOC         = " "      AND
628         ((L1_SERIE     = " "      AND
629         L1_PDV         = " "      AND
630         L1_STORC       <> "C"      AND
631         %exp:cDateExp%
632         SL1.%notDel%    AND
633         SA1.%notDel%)    OR
634         (L1_RESERVA    <> " "      AND
635         L1_STATUS      <> "D"      ))
636
637     ORDER BY %exp:cOrderBy%
638
639     EndSql
640
641     Otherwise // Normal
642
643     BeginSql alias cOptionsTemp
644
645         %NoParser%
646
647         SELECT L1_NUM , A1_NOME , L1_NUMORC %exp:cAddSELECT%
648
649         FROM %table:SL1% SL1 ,%table:SA1% SA1
650
651         WHERE  L1_FILIAL      = %xfilial:SL1%      AND
652                A1_FILIAL      = %xfilial:SA1%      AND
653                L1_DTLIM       >= %exp:DTOS(dDataBase)% AND
654                L1_DOC         = " "      AND
655                L1_DOCPED      = " "      AND
656                L1_CLIENTE     = A1_COD      AND
657                L1_LOJA        = A1_LOJA      AND
658                L1_PEDRES      = " "      AND
659                %exp:cDateExp%
660                SL1.%notDel%    AND
661                SA1.%notDel%    AND
662                (
663                    (
664                        L1_SERIE = " "      AND
665                        L1_PDV   = " "      AND
666                        L1_STORC <> "C"      AND
667                    ) OR
668                    (
669                        L1_RESERVA <> " "      AND
670                        L1_STATUS <> "D"
671                    )
672                )
673
674     ORDER BY %exp:cOrderBy%
675
676     EndSql

```

```

677
678 EndCase
679
680 LjGrvLog( "STDISGetOp", "SQL: " + GetLastQuery()[2] )
681
682 While !(cOptionsTemp)->(EOF())
683
684     // Verifica se existe arquivo informando que o orcamento ja foi importado
685     // So add se o arquivo nao existir
686     If !FR271HArq ( {{ (cOptionsTemp)->L1_NUM , "" }} , .T., Nil , .T. )[1]
687
688         AADD( aAux , (cOptionsTemp)->L1_NUM          )
689         AADD( aAux , (cOptionsTemp)->L1_NUMORC      )
690         AADD( aAux , AllTrim((cOptionsTemp)->A1_NOME ) )
691
692
693         For nI := 1 To Len(aFieldsAdd)
694             AADD( aAux , (cOptionsTemp)->&(aFieldsAdd[nI]) )
695         Next nI
696
697         AADD( aOptions , aAux )
698
699     EndIf
700
701     aAux := {}
702
703     (cOptionsTemp)->(DbSkip())
704
705 EndDo
706
707 (cOptionsTemp)->( DbCloseArea() )
708
709 DbSelectArea( "SL1" ) // Seleção de tabela (preventiva)
710
711 Return aOptions
712
713
714 //-----
715 /*{Protheus.doc} STDISSearchField
716 Retorna a estrutura do campo de pesquisa para Interface
717
718 @param   cField      Nome do Campo
719 @return  aField -> Retorna a estrutura do campo de pesquisa para a interface
720 @obs
721 @sample
722 */
723 //-----
724 Function STDISSearchField(cField)
725
726 Local aField      := {}           // Estrutura do campo de pesquisa para interface
727 Local cLabelScreen := ""         // Label da interface
728 Local bValid      := {}|.T.     // ValiCAIXAPOSDação do campo

```

```

729
730 Default cField      := ""
731
732 ParamType 0 Var cField As Character Default ""
733
734 If !Empty(cField)
735
736     DbSelectArea("SX3")
737     SX3->(DbSetOrder(2))
738     If SX3->(DbSeek(cField))
739
740         Do Case
741
742             Case cField == "A1_CGC"
743
744                 cLabelScreen := AllTrim( X3Titulo() )
745
746             Case cField == "L1_NUMORC"
747
748                 If SuperGetMv("MV_LJPRVEN",,.T.) // Utiliza Pre-Venda:
749                     cLabelScreen := "Pre-Venda"
750                 Else
751                     cLabelScreen := "DAV" // Utiliza DAV
752                 EndIf
753
754             Otherwise
755
756                 cLabelScreen := AllTrim( X3Titulo() )
757
758             EndCase
759
760             AAdd( aField      , cLabelScreen ) // [01] Titulo do campo
761             AAdd( aField      , AllTrim( X3Descric() ) ) // [02] ToolTip do campo
762             AAdd( aField      , AllTrim( SX3->X3_CAMPO ) ) // [03] Id do Field
763             AAdd( aField      , SX3->X3_TIPO ) // [04] Tipo do campo
764             AAdd( aField      , SX3->X3_TAMANHO ) // [05] Tamanho do campo
765             AAdd( aField      , SX3->X3_DECIMAL ) // [06] Decimal do campo
766             AAdd( aField      , bValid ) // [07] Code-block de validacao do campo
767             AAdd( aField      , Nil ) // [08] Code-block de validacao When do campo
768             AAdd( aField      , StrTokArr( AllTrim( X3CBox() ),',' ) ) // [09] Lista de valores permitido do campo
769             AAdd( aField      , Nil ) // [10] Indica se o campo tem preenchimento obrigatorio
770             AAdd( aField      , Nil ) // [11] Code-block de inicializacao do campo
771             AAdd( aField      , Nil ) // [12] Indica se trata-se de um campo chave
772             AAdd( aField      , Nil ) // [13] Indica se o campo pode receber valor em uma operacao de update.
773             AAdd( aField      , .F. ) // [14] Indica se o campo e virtual
774
775         EndIf
776
777 EndIf
778
779 Return(aField)
780

```

```

781 //-----
782 /*/{Protheus.doc} STDIMPPROD
783 Importa as informacoes do produto da retaguarda, para posteriormente cadastra-lo na base local
784
785 @param
786 @return
787 @obs
788 @sample
789 */
790 //-----
791 Function STDIMPPROD( cProdCode, aFieldsProd )
792 Local aArea := GetArea()
793 Local aCampos := {} // Array de campos
794 Local aRet := {} // Parâmetro de retorno
795 Local nX := 0 // Contador
796
797 DEFAULT cProdCode := ""
798
799 LjGrvLog("Importa_Orcamento:STDImpProd","Comunicação OK")
800
801 DbSelectArea("SB1")
802 DbSetOrder(1)//B1_FILIAL+B1_COD
803 If DbSeek( xFilial("SB1")+cProdCode )
804     For nX := 1 to Len(aFieldsProd)
805         AAdd(aRet,{aFieldsProd[nX],&("SB1->" + aFieldsProd[nX]),Nil})
806     Next nX
807 EndIf
808
809 RestArea(aArea)
810
811 Return aRet
812
813
814 //-----
815 /*/{Protheus.doc} STDValLock
816 Valida se o orçamento está com lock, ou seja, se está sendo alterado por outro usuário
817 @param sNumOrc Número do Orçamento L1_NUM
818 @return lRet Se o orçamento estiver sendo utilizado retorno .T.
819 @obs
820 @sample
821 */
822 //-----
823 Static Function STDValLock(sNumOrc)
824 Local lRet := .F.
825
826 DbSelectArea("SL1")
827 DbSetOrder(1)
828
829 If SL1->(DbSeek(xfilial("SL1") + sNumOrc))
830     If SL1->(SimpleLock()) //tenta dar lock no registro, se retornar .F. significa que está com lock
831         SL1->(MsUnLock())
832     Else

```

```

833         lRet :=.T.
834     Endif
835 Endif
836
837 Return lRet
838
839
840 /*{Protheus.doc} STDAltRegL1
841     Se for recebimento de multiplos orçamentos e PDV ONLINE precisa ajustar os registros dos orçamentos
842     originais
843     @type Function
844     */
845 Function STDAltRegL1()
846
847 Local oModel := STDGPBModel()
848 Local aMultOrc := STDMultOrc()
849 Local nI := 0
850 Local cChave := ""
851
852 LjGrvLog(NIL, "Importação de Orçamento PDV Online -Atualiza os Orçamentos originais quando for Multiplos", aMultOrc )
853
854 If Len(aMultOrc) >= 1
855 // retorna L2_NUMORIG dos orçamentos, se retornar mais de 1 registro significa que foi importação de multiplos orçamentos
856 // se retornar apenas um registro significa que foi importado orçamento após lançar itens no PDV.
857
858 For nI := 1 To Len(aMultOrc)
859     cChave := aMultOrc[nI]
860     DbSelectArea("SL1")
861     DbSetOrder(1) //L1_FILIAL+L1_NUM
862     If !Empty(cChave) .AND. DbSeek(xFilial("SL1")+cChave)
863
864         RecLock("SL1",.F.)//alteracao
865
866         REPLACE SL1->L1_SERIE WITH oModel:GetValue("SL1MASTER" , "L1_SERIE")
867         REPLACE SL1->L1_SERPED WITH oModel:GetValue("SL1MASTER" , "L1_SERPED")
868         REPLACE SL1->L1_DOC WITH oModel:GetValue("SL1MASTER" , "L1_DOC")
869         REPLACE SL1->L1_DOCPED WITH oModel:GetValue("SL1MASTER" , "L1_DOCPED")
870         REPLACE SL1->L1_PDV WITH oModel:GetValue("SL1MASTER" , "L1_PDV")
871         REPLACE SL1->L1_CONFVEN WITH "SSSSSSSSNSSS"
872         REPLACE SL1->L1_SITUA WITH "FR"
873         REPLACE SL1->L1_STATUS WITH "F"
874         REPLACE SL1->L1_EMITNF WITH oModel:GetValue("SL1MASTER" , "L1_EMITNF")
875         REPLACE SL1->L1_OPERADO WITH oModel:GetValue("SL1MASTER" , "L1_OPERADO")
876         REPLACE SL1->L1_ESTACAO WITH oModel:GetValue("SL1MASTER" , "L1_ESTACAO")
877         REPLACE SL1->L1_NUMFRT WITH oModel:GetValue("SL1MASTER" , "L1_NUM")
878         REPLACE SL1->L1_IMPRIME WITH "1S"
879
880         SL1->(MsUnlock())
881         SL1->( DbSkip() )
882
883     Endif
884

```

```

885     Next nI
886
887 EndIf
888
889 Return
890
891 /*{Protheus.doc} STDMultOrc
892     Verifica se foi recebimento de multiplos Orçamentos
893     @type Function
894     */
895
896 Function STDMultOrc()
897 Local oModel          := STDGPBModel()
898 Local nX              := 0
899 Local nQtRegSL2       := STDPBLength("SL2")
900 Local aMultOrc        := {}
901
902 For nX:= 1 to nQtRegSL2
903     oModel:GetModel("SL2DETAIL"):GoLine(nX)
904     If !Empty(oModel:GetValue("SL2DETAIL", "L2_NUMORIG")) .AND. !oModel:GetModel("SL2DETAIL"):IsDeleted();
905     .AND. oModel:GetValue("SL2DETAIL", "L2_NUMORIG") <> oModel:GetValue("SL2DETAIL", "L2_NUM");
906     .AND. AScan( aMultOrc, {|x| x == oModel:GetValue("SL2DETAIL", "L2_NUMORIG")} ) == 0
907         aAdd(aMultOrc, oModel:GetValue("SL2DETAIL", "L2_NUMORIG") )
908     EndIf
909 Next nX
910
911 LjGrvLog(NIL, "Importação de Orçamento PDV Online - Pega os L2_NUMORIG", aMultOrc )
912
913 Return aMultOrc
914
915
916 /*{Protheus.doc} STDNomeVen
917 Retorna o nome do Vendedor
918 @type Function
919 @param cA3COD      , caracter, código do vendedor
920 @return cA3NReduz   , caracter, nome do vendedor
921 */
922 Function STDNomeVen(cA3COD)
923 Local cA3NReduz := ""
924 Local aArea     := GetArea()
925
926 cA3COD := PadR(cA3COD, TamSx3("A3_COD"))[1])
927 DbSelectArea("SA3")
928 SA3->(DbSetOrder(1))//A3_FILIAL+A3_COD
929
930 If SA3->(DbSeek( xFilial('SA3') + cA3COD ))
931     cA3NReduz:= " - " + SA3->A3_NREDUZ
932 Endif
933
934 RestArea(aArea)
935 Return cA3NReduz
936

```

```

937
938 /*{Protheus.doc} STDBuscaNccOrc
939     Quando ao ambiente é Central de Pdv, tem a necessidade de buscar Ncc pendents na MDK e MDJ
940     na Central para depois buscar na SE1. Para isso, busca a MDK_NUMREC na Central de Pdv para depois
941     buscar o titulo SE1 R_E_C_N_0_ da SE1 é igual a MDK_NUMREC
942     @type Function
943     @param cNumOrc, param_type, L1_NUM do Orçamento
944     @return aSelReynos, array, array com numeros da R_E_C_N_0_ dos titulos na SE1
945     */
946 Function STDBuscaNccOrc(cNumOrc)
947 Local aSelReynos      := {}
948 Local aArea           := GetArea()
949 Local cOptionsTemp    := GetNextAlias()
950 Local nI              := 0
951 Local cNumExp         := ""
952 Local cSituaExp       := ""
953 Local aFieldsImp      := {}
954 Local axTesSql        := {}
955 Local caxTesSql       := ""
956
957 Default cNumOrc       := ""
958
959 If !Empty(cNumOrc)
960     cNumExp := "%" + StrZero(Val(cNumOrc),TamSX3("L1_NUM")[1]) + "%"
961 EndIf
962
963 aFieldsImp := MDK->(dbStruct()) //Todos os campos
964
965 cSituaExp:= "%'" + "OR" + "'%"
966
967 BeginSql ALIAS cOptionsTemp
968
969     SELECT MDK_NUMREC
970     FROM %table:MDK% MDK
971     INNER JOIN %table:MDJ% MDJ ON MDJ_FILIAL = MDK_FILIAL AND MDJ.%NotDel%
972     WHERE
973         MDJ_NUMORC = MDK_NUMORC          AND
974         MDJ_SITUA  = %exp:cSituaExp%      AND
975         MDJ_NUMORC = %Exp:cNumExp%        AND
976         MDK.%NotDel%
977 EndSql
978
979 axTesSql := GetLastQuery()
980 caxTesSql := axTesSql[2]
981
982 LjGrvLog(cNumOrc, "Query para busca de MDK_NUMREC", caxTesSql )
983
984 DbSelectArea(cOptionsTemp)
985
986 While !(cOptionsTemp)->(EOF())
987
988     For nI := 1 To FCount()

```

```

989      AADD( aSelReynos , FieldGet(nI))
990      Next nI
991
992      (cOptionsTemp)->(DbSkip())
993
994 EndDo
995
996 (cOptionsTemp)->( DbCloseArea() )
997
998 RestArea(aArea)
999
1000 LjGrvLog(cNumOrc, "MDK_NUMREC Retornados", aSelReynos )
1001
1002 Return aSelReynos
1003
1004 //-----
1005 /*{Protheus.doc} STDGetValSL4
1006 Retorna o valor restante pago do orçamento retira posterior que,
1007 esse valor é referente a NCC, pois os demais valores são gerados na
1008 SL4.
1009 @param aSL4,      Array, SL4 que está sendo importada.
1010 @return nValor,   Numerico, Retorna o valor restante pago em NCC.
1011 @obs
1012 @sample
1013 */
1014 //-----
1015 Function STDGetValSL4(aSL4)
1016
1017 Local nValor      := 0                //Retrona o valor restante
1018 Local cL4NUM      := ""              //L4_NUM para busca das formas de pagamento
1019 Local aAreaSL4    := SL4->(GetArea()) //Area seleciona
1020
1021 cL4NUM := aSL4[1][Ascan(aSL4[1] , { |x| x[1] == "L4_NUM" } )][2]
1022
1023 If !Empty(cL4NUM)
1024     DbSetOrder(1) //L4_FILIAL+L4_NUM+L4_ORIGEM
1025     If SL4->(DbSeek(xFilial("SL4") + cL4NUM))
1026         While SL4->( !EOF() ) .AND. (SL4->L4_FILIAL + SL4->L4_NUM == xFilial("SL4") + cL4NUM)
1027             nValor += SL4->L4_VALOR
1028             SL4->( DbSkip() )
1029         EndDo
1030     EndIf
1031 EndIf
1032
1033 If nValor > 0
1034     nValor := STDGPBasket("SL1","L1_VLRTOT") - STIGetTotal()
1035 Else
1036     nValor := STDGPBasket("SL1","L1_VLRTOT")
1037 EndIf
1038
1039 RestArea(aAreaSL4)
1040

```

1041 Return nValor

Componentes/ImportSale/STWImportSale.prw

579 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STWIMPORTSALE.CH"
4
5 Static cMsgOrc := ""
6
7 //-----
8 /*{Protheus.doc} STWISearchOptions
9 Busca opções de orçamento
10
11 @param cField      Campo utilizado para busca
12 @param cFieldAssigned Conteúdo do campo para busca
13 @param dDataMov     Data do movimento a realizar a Busca PAFECE
14 @return aRet[1]      lUserSelect - Retorna se deve ser selecionada opção de orçamento
15 @return aRet[2]      aOptionSales - Array com as opções de orçamento
16 @obs
17 @sample
18 */
19 //-----
20 Function STWISearchOptions( cField      , cFieldAssigned, lCancSale, lSearchAll ,;
21                             dDataMov    )
22
23 Local aRet          := {}           // Retorno função
24 Local nOption       := 0           // Opção de busca
25 Local aFieldsAdd    := {}           // Campos adicionais a serem buscados juntamente com as opções
26
27 Default cField      := ""
28 Default cFieldAssigned := ""
29 Default lCancSale    := .F.
30 Default lSearchAll   := .F.
31 Default dDataMov     := dDataBase
32
33 ParamType 0 Var      cField      As Character Default ""
34 ParamType 1 Var      cFieldAssigned As Character Default ""
35 ParamType 2 Var      lCancSale    As Logical   Default .F.
36 ParamType 3 Var      lSearchAll   As Logical   Default .F.
37
38 //Tratamento para pesquisa
39 If !SuperGetMv("MV_LJORPAR",.F.) .And. (At("?",AllTrim(cFieldAssigned)) == 1 .Or. RAT("?",AllTrim(cFieldAssigned)) == Len(AllTrim(cFieldAssigned))) .And.
    !Empty(STRTRAN(cFieldAssigned,"*", ""))
40   STFMesssage(ProcName(), "ALERT", STR0018 ) //"Pesquisa Aproximada (*) desabilitada (MV_LJORPAR).
41   STFSHOWMESSAGE(ProcName())
42 EndIf
43
44 Do Case
45
46   Case SubStr( cFieldAssigned , 1 , 1 ) == "?" .And. Empty(STRTRAN(cFieldAssigned,"*", ""))
47
48     nOption := 2 // Todos orçamentos abertos
49
50   Case SubStr( cFieldAssigned , 1 , 1 ) == "/"
51

```

```

52      nOption := 3 // Pela Data do dia
53
54      Otherwise
55
56      nOption := 1 // Pelo Campo
57
58 EndCase
59
60 aFieldsAdd := STIISFieldsAdd() // Busca campos adicionais a serem apresentados
61
62 /*/
63 Busca opções de orçamento na retaguarda
64 /*/
65 aRet := STBISSearchOptions( nOption      , cField , cFieldAssigned , lSearchAll , ;
66                             aFieldsAdd , dDataMov )
67
68 If Len(aRet) = 0
69     LjGrvLog("Importa_Orcamento:STWISSearchOptions", "Parâmetro de retorno aRet : Conteúdo vazio." )
70 EndIf
71
72 Return aRet
73
74
75 //-----
76 /*/{Protheus.doc} STWISImpAllSelected
77 Importa as opções de orçamento selecionadas
78
79 @param  aSelectedSales  Opções selecionadas
80 @param  cStage          Etapa de importação. "D" - Direta , "S" - Selecionada
81 @Param  lGetOrcExpired  Pega orçamentos expirados pafecf
82 @return aRet           - Informacoes das vendas
83 @obs
84 @sample
85 /*/
86 //-----
87 Function STWISImpAllSelected( aSelectedSales , cStage , lGetOrcExpired )
88
89 Local aRet      := {}           // Retorno função
90 Local nI        := 0           // Contador
91 Local lRet      := .T.         // retorno função
92 Local aAllSales := {}           // Armazena todos os orçamentos importados
93 Local aSale     := {}           // Armazena 1 orçamento importado
94 Local aErrorSales := {}         // Armazena orçamentos que não foram importados
95 Local cErrorSales := ""         // Armazena orçamentos que não foram importados
96 Local cMsgInterface := ""       // Mensagem a ser exibida na interface
97 Local cSales     := ""         // Mensagem orçamentos
98 Local lMessage    := .F.        // Indica se apresenta mensagem na interface
99 Local lContinue   := .F.        // Indica se continua ou interrompe o procedimento
100
101 Default aSelectedSales := {}
102 Default cStage         := "D"
103 Default lGetOrcExpired := .F.

```

```

104
105 ParamType 0 Var aSelectedSales      As Array      Default {}
106 ParamType 0 Var cStage              As Character   Default "D"
107
108 STFCleanInterfaceMessage()
109
110 If !( Len(aSelectedSales) > 0 )
111     lRet := .F.
112 EndIf
113
114 If lRet
115
116     For nI := 1 To Len(aSelectedSales)
117
118         If nI = 1
119             LjGrvLog("Importa_Orcamento:STWISImpAllSelected","Chama rotina: STWISGetSale")
120         EndIf
121         aSale := STWISGetSale( aSelectedSales[nI] , lGetOrcExpired )
122
123         If Len(aSale) > 0
124
125             AADD( aAllSales ,   aSale   )
126
127             /*/
128             Atribui texto de orçamentos a serem importados (para msg)
129             /*/
130             If STFCfg("lPafEcF")
131                 cSales := aSelectedSales[nI][2] + " , " // DAV/PRE
132             Else
133                 cSales := aSelectedSales[nI][1] + " , " // L1_NUM
134             EndIf
135
136         Else
137
138             AADD( aErrorSales ,   aSelectedSales[nI]   )
139
140         EndIf
141
142     Next nI
143
144     If Len(aErrorSales) > 0
145
146         /*/
147         Se ocorreu algum problema com algum orçamento
148         /*/
149         If Len(aAllSales) > 0
150
151             /*/
152             Se vieram orçamentos com problema mas também que deram certo
153             /*/
154             For nI := 1 To Len(aErrorSales)
155

```

```

156         If ValType(aErrorSales[nI]) == "C"
157             cErrorSales := cErrorSales + aErrorSales[nI] + " , "
158         ElseIf ValType(aErrorSales[nI][1]) == "C"
159             cErrorSales := cErrorSales + aErrorSales[nI][1] + " , "
160         EndIf
161
162     Next nI
163
164     cMsgInterface := STR0001 + cErrorSales + STR0002 //"Não foi possível importar o(s) orçamento(s) "continua a importação dos outros?"
165     lMessage      := .T. // Força message
166     lContinue     := .T. // Continua para perguntar
167     LjGrvLog("Importa_Orcamento:STWISImpAllSelected",STR0001 + cErrorSales + STR0002) //"Não foi possível importar o(s) orçamento(s) "continua a importação dos
    outros?"
168
169     Else
170
171         /*/
172         Se vieram apenas orçamentos com problemas
173         /*/
174         lMessage      := .F.
175         lContinue     := .F.
176
177     EndIf
178
179     Else
180
181         lMessage      := .T. // Importacao sem mensagem
182         lContinue     := .T. // Continua procedimento
183         cMsgInterface := STR0003 + cSales + "?" //"Confirma importação do(s) orçamento(s) "
184
185     EndIf
186
187 EndIf
188 EndIf
189
190 AADD( aRet , lContinue      )
191 AADD( aRet , lMessage      )
192 AADD( aRet , cMsgInterface )
193 AADD( aRet , aAllSales     )
194 AADD( aRet , aErrorSales   )
195
196
197 If !lContinue
198     LjGrvLog("Importa_Orcamento:STWISImpAllSelected", "lContinue : " , lContinue )
199     LjGrvLog("Importa_Orcamento:STWISImpAllSelected", "Parâmetro de retorno aRet : " , aRet )
200 EndIf
201
202 Return aRet
203
204
205 //-----
206 /*/{Protheus.doc} STWISRegSale

```

```

207 Registra orçamento no POS
208
209 @param  aSelectedSale      Opção selecionada
210 @Param  lCancel            Cancela orcamento apos importar Pafecf
211 @return Nil
212 @obs
213 @sample
214 /*/
215 //-----
216 Function STWISRegSale( aAllSales , lCancel )
217
218 Local nI                := 0          // Contador
219 Local cNumLastSale      := ""         // Numero da ultima venda
220 Local cLastDoc          := ""         // Ultimo Doc
221 Local oTotal           := Nil        // Totalizador
222 Local oMdl             := Nil        // Model Pgto
223 Local oModelPay         := Nil        // Model Pgto
224 Local oModelParc        := Nil        // Model Pgto
225 Local lCancOrc         := .T.        // Se houver falha determina se será cancelado toda a operação
226 Local aCancOrc         := {}         // Orçamentos que serão cancelados na Função STIAtuGetOrc()
227
228 Default aAllSales      := {}
229 Default lCancel        := .F.
230
231 ParamType 0 Var aAllSales As Array Default {}
232
233 /*/
234     Realizar registro dos orçamentos
235 /*/
236 If Len(aAllSales) > 0
237     For nI := 1 To Len(aAllSales)
238         //Se ocorrer erro na importação é cancelado o cupom
239         LjGrvLog("Importa_Orcamento:STWISRegSale","Chama rotina: STBImportSale, nI = ", nI)
240         If !STBImportSale( aAllSales[nI] , lCancel, Len(aAllSales) > 1, @lCancOrc )
241             If lCancOrc
242                 LjGrvLog("Importa_Orcamento:STWISRegSale","Ocorreu um erro na importação")
243                 cNumLastSale := STDCSLastSale()
244                 cLastDoc     := STDCSDoc( cNumLastSale )
245                 LjGrvLog("Importa_Orcamento:STWISRegSale","Chama rotina: STWCancelSale")
246                 LjGrvLog("Importa_Orcamento:STWISRegSale","Parâmetro cLastDoc: ", cLastDoc)
247                 LjGrvLog("Importa_Orcamento:STWISRegSale","Parâmetro cNumLastSale: ", cNumLastSale)
248                 STWCancelSale(.T.,,cLastDoc, "L1_NUM",,)
249                 If nI == Len(aAllSales)
250                     LjGrvLog("Importa_Orcamento:STWISRegSale","Chama rotina: STIRegItemInterface")
251                     STIRegItemInterface()
252                 EndIf
253             Else
254                 Aadd(aCancOrc,nI)
255             EndIf
256         Else
257             //Cria arquivo para controle de orcamento no Venda Assistida
258             STBCtrImpOrc( aAllSales[nI][1][1][2], STFGetStation("PDV"), Nil, Nil, .T. )

```

```

259      EndIf
260
261      //Cancela Venda apos importar
262      //Usado para vendas expiradas PafEcf
263      If lCancel
264
265          LjGrvLog("Importa_Orcamento:STWISRegSale","Cancela venda após importar")
266          oTotal      := STFGetTot()
267          nTotalSale := oTotal:GetValue("L1_VLRTOT")
268
269          oMdl := ModelPayme()
270
271          oModelPay := oMdl:GetModel('PARCELAS')
272          oModelPay:Activate()
273          oModelPay:ClearData()
274          oModelPay:InitLine()
275
276          oModelParc := oMdl:GetModel('APAYMENTS')
277          oModelParc:Activate()
278          oModelParc:ClearData()
279          oModelParc:InitLine()
280
281          //Seta pagamento em dinheiro
282          LjGrvLog("Importa_Orcamento:STWISRegSale","Chama rotina: STICS0rc")
283          STICS0rc(dDataBase, nTotalSale)
284          LjGrvLog("Importa_Orcamento:STWISRegSale","Chama rotina: STIConfPay(.F.)")
285          STIConfPay(.F.)
286
287          cNumLastSale := STDCSLastSale()
288          cLastDoc      := STDCSDoc( cNumLastSale )
289
290          LjGrvLog("Importa_Orcamento:STWISRegSale","Chama rotina: STWCancelSale")
291          LjGrvLog("Importa_Orcamento:STWISRegSale","Parâmetro cLastDoc: ", cLastDoc)
292          LjGrvLog("Importa_Orcamento:STWISRegSale","Parâmetro cNumLastSale: ", cNumLastSale)
293          STWCancelSale( .F. , .F. , Nil , cLastDoc , ;
294                        cNumLastSale , .T. )
295
296
297      EndIf
298      Next nI
299
300      STIAtGetOr(aCanc0rc)
301
302  EndIf
303
304  Return Nil
305
306
307  //-----
308  /*{Protheus.doc} STWImportSale
309  Function Importação de Orçamentos
310

```

```

311 @param aSelectedSale      Opção selecionada
312 @param lGetOrcExpired      Importa orçamentos expirados
313 @return aSale              Retorna estrutura do orçamento
314 @obs
315 @sample
316 /*/
317 //-----
318 Function STWISGetSale( aSelectedSale , lGetOrcExpired )
319
320 Local aSale                := {}                // Retorno função
321 Local lContinue            := .T.                // Fluxo lógico
322
323 Default aSelectedSale      := ""
324 Default lGetOrcExpired     := .F.
325
326 If !( Len(aSelectedSale) > 0 )
327     lContinue := .F.
328     LjGrvLog("Importa_Orcamento:STWISGetSale", "Len(aSelectedSale) < 0" )
329 EndIf
330
331 If STFGetCfg("lPafEcf") .AND. Empty(aSelectedSale[2]) // L1_NUMORC
332
333     STFMessagem("STImportSale","STOP", STR0004 + aSelectedSale[1] + ; // "Orçamento: "
334     STR0005 + ; // " não possui número de DAV ou Pré-Venda. Em ambiente PAF-ECF não é permitido importar Orçamento "
335     STR0006 + ; // "que não seja proveniente de um DAV ou Pré-Venda! Verifique se o ambiente que gerou o Orçamento "
336     STR0007 ) // "esta habilitado para operar em modo PAF-ECF com DAV ou Pre-Venda."
337
338     lContinue := .F.
339     LjGrvLog("Importa_Orcamento:STWISGetSale",STR0004 + aSelectedSale[1] + STR0005 + STR0006 + STR0007 ) //idem a acima
340
341 EndIf
342 EndIf
343
344 /*/
345 Realiza Importação do Orçamento
346 /*/
347 If lContinue
348     If STFGetCfg("lPafEcf")
349         LjGrvLog("Importa_Orcamento:STWISGetSale","STFGetCfg(lPafEcf) = .T.")
350         LjGrvLog("Importa_Orcamento:STWISGetSale","Chama rotina remoto: STBImportR")
351         LjGrvLog("Importa_Orcamento:STWISGetSale","Utilizando L1_NUMORC")
352         LjGrvLog("Importa_Orcamento:STWISGetSale","Parâmetro aSelectedSale[2]: ", aSelectedSale[2])
353         LjGrvLog("Importa_Orcamento:STWISGetSale","Parâmetro lGetOrcExpired: ", lGetOrcExpired)
354         lContinue := STBRemoteExecute( "STBImportR" , { "L1_NUMORC" , aSelectedSale[2] , .T. , .T. , lGetOrcExpired } , NIL , .F. , @aSale )
355     Else
356         LjGrvLog("Importa_Orcamento:STWISGetSale","STFGetCfg(lPafEcf) = .F.")
357         LjGrvLog("Importa_Orcamento:STWISGetSale","Chama rotina remoto: STBImportR")
358         LjGrvLog("Importa_Orcamento:STWISGetSale","Utilizando L1_NUM")
359         LjGrvLog("Importa_Orcamento:STWISGetSale","Parâmetro aSelectedSale[1]: ", aSelectedSale[1])
360         lContinue := STBRemoteExecute( "STBImportR" , { "L1_NUM" , aSelectedSale[1] , .T. , .T. , .F. , @aSale )
361     EndIf
362 EndIf

```

```

363 If !lContinue
364     STFMessagem( "STImportSale", "STOP", STR0008 ) //"Ocorreu falha de comunicação entre os ambientes"
365     LjGrvLog("Importa_Orcamento:STWISGetSale",STR0008) //"Ocorreu falha de comunicação entre os ambientes"
366     aSale := {}
367 EndIf
368
369
370 /*/
371     Validar Retorno a aSale
372 /*/
373 If lContinue
374     If ValType(aSale) == "A"
375         If Len(aSale) > 0
376             If aSale[1] <> "OK"
377
378                 cMsgOrc := ""
379
380                 Do Case
381
382                     Case aSale[1] == "NOTFOUND"
383
384                         If STFCfgGetCf("lPafEcf") // PAF
385                             If SuperGetMv("MV_LJPRVEN",,,T.) // Utiliza Pre-Venda
386                                 STFMessagem("STImportSale","STOP",STR0009 + aSelectedSale[2] + STR0010) //" não foi encontrada na Retaguarda."
387                                 LjGrvLog("Importa_Orcamento:STWISGetSale",STR0009 + aSelectedSale[2] + STR0010) //"A Pre-Venda " ... " não foi encontrada na Retaguarda."
388                             Else
389                                 STFMessagem("STImportSale","STOP",STR0011 + aSelectedSale[2] + STR0012) //" não foi encontrada na Retaguarda."
390                                 LjGrvLog("Importa_Orcamento:STWISGetSale",STR0011 + aSelectedSale[2] + STR0012) //"A DAV " ... " não foi encontrada na Retaguarda."
391                             EndIf
392                         Else
393                             STFMessagem("STImportSale","STOP",STR0004 + aSelectedSale[1] + STR0012) //" não foi encontrado na Retaguarda."
394                             LjGrvLog("Importa_Orcamento:STWISGetSale",STR0004 + aSelectedSale[1] + STR0012) //"O Orçamento " ... " não foi encontrado na Retaguarda."
395                         EndIf
396
397                         aSale := {}
398
399                     Case aSale[1] == "JAIMPORTADO"
400
401                         If STFCfgGetCf("lPafEcf") // PAF
402                             If SuperGetMv("MV_LJPRVEN",,,T.) // Utiliza Pre-Venda
403                                 STFMessagem("STImportSale","STOP",STR0009 + aSelectedSale[2] + STR0013) //"A Pre-Venda " ... " já foi importado da Retaguarda."
404                                 LjGrvLog("Importa_Orcamento:STWISGetSale",STR0009 + aSelectedSale[2] + STR0013) //"A Pre-Venda " ... " já foi importado da Retaguarda."
405                             Else
406                                 STFMessagem("STImportSale","STOP",STR0011 + aSelectedSale[2] + STR0013) //"A DAV " ... " já foi importado da Retaguarda."
407                                 LjGrvLog("Importa_Orcamento:STWISGetSale",STR0011 + aSelectedSale[2] + STR0013) //"A DAV " ... " já foi importado da Retaguarda."
408                             EndIf
409                         Else
410                             STFMessagem("STImportSale","STOP",STR0016 + aSelectedSale[1] + STR0013) //"O Orçamento " ... " já foi importado da Retaguarda."
411                             LjGrvLog("Importa_Orcamento:STWISGetSale",STR0016 + aSelectedSale[1] + STR0013) //"O Orçamento " ... " já foi importado da Retaguarda."
412                         EndIf
413
414                         aSale := {}

```

```

415
416         Case aSale[1] == "SEMESTOQUE"
417
418             STFMessage("STImportSale","STOP",STR0016 + aSelectedSale[1] + STR0019) //"0 Orçamento " ..." não contam itens que não possuem saldo em estoque."
419             aSale := {}
420             LjGrvLog("Importa_Orcamento:STWISGetSale",STR0016 + aSelectedSale[1] + STR0019) //"0 Orçamento " ..." não contam itens que não possuem saldo em
estoque."
421
422         Case aSale[1] == "ERROR"
423
424             STFMessage("STImportSale","STOP", STR0015) //"Erro ao carregar orçamento, tente novamente"
425             aSale := {}
426             LjGrvLog("Importa_Orcamento:STWISGetSale",STR0015) //"Erro ao carregar orçamento, tente novamente"
427
428         Case aSale[1] == "EXPIRADO"
429
430             STFMessage("STImportSale","STOP", STR0017) //"Orçamento expirado"
431             aSale := {}
432             LjGrvLog("Importa_Orcamento:STWISGetSale",STR0017) //"Orçamento expirado"
433
434         Case aSale[1] == "SEMCREDITO"
435
436             cMsgOrc := STR0026 //"Não foi possível importar o orçamento, o cliente não possui limite de crédito disponível"
437             aSale := {}
438             LjGrvLog("Importa_Orcamento:STWISGetSale",STR0020) //"Cliente Sem Crédito"
439
440         Case aSale[1] == "BLOCKPE"
441
442             STFMessage("STImportSale","STOP", STR0021 + aSelectedSale[1]) //"Bloqueado Ponto de Entrada STVLDIMP, Orçamento: "
443             aSale := {}
444             LjGrvLog("Importa_Orcamento:STWISGetSale",STR0021 + aSelectedSale[1]) //"Bloqueado Ponto de Entrada STVLDIMP, Orçamento: "
445
446         Case aSale[1] == "SL1LOCK"
447
448             STFMessage("STImportSale","STOPPOPUP", STR0016 + aSelectedSale[1] + STR0022 + aSelectedSale[3] + STR0023)
449             aSale := {}
450             LjGrvLog("Importa_Orcamento:STWISGetSale",STR0016 + aSelectedSale[1] + STR0023)
451
452
453         Case aSale[1] == "FPGTONAOPERMITIDO"
454
455             STFMessage("STImportSale","STOPPOPUP", STR0004 + aSelectedSale[1] + STR0022 + aSelectedSale[3] + Chr(13) + STR0024 + " " + aSale[2])
456             LjGrvLog("Importa_Orcamento:STWISGetSale", + STR0024 + " " + aSale[2])
457             aSale := {}
458
459         Otherwise
460
461             STFMessage("STImportSale","STOP", STR0015) //"Erro ao carregar orçamento, tente novamente"
462             aSale := {}
463             LjGrvLog("Importa_Orcamento:STWISGetSale",STR0015) //"Erro ao carregar orçamento, tente novamente"
464
465     EndCase

```

```

466
467         Else
468             /*/
469             Retornar apenas a venda, sem o Status
470             /*/
471             aSale := aSale[2]
472         EndIf
473     EndIf
474     Else
475         aSale := {}
476     EndIf
477 EndIf
478
479 STFS showMessage("STImportSale")
480 If Len(aSale) = 0
481     LjGrvLog("Importa_Orcamento:STWISGetSale", "Parâmetro de retorno aSale : Conteúdo vazio." )
482 EndIf
483
484 Return aSale
485
486 //-----
487 /*/{Protheus.doc} STWISPafSearch
488 Busca de Orçamentos para cancelamento PAF-ECF
489
490 @param lSeachAll Busca Todos orçamentos
491 @param dDataMov Data do movimento a realizar a Busca PAFECE
492 @return Nil
493 @obs
494 @sample
495 /*/
496 //-----
497 Function STWISPafSearch( lSeachAll , dDataMov )
498
499 Local aAux           := {}           // Armazena retorno função temporariamente
500 Local lUserSelect    := .F.         // Indica se orçamento deve ser selecionado
501 Local aImported      := {}         // Armazena retorno da verificação de importação
502 Local aField         := STDISearchField("L1_NUMORC") // Armazena informação do campo de busca de orçamento
503 Local aOptionSales   := {}         // Array com vendas selecionadas
504 Local lCancel        := .T.         // Cancela as vendas apos importar pois as mesma estao expiradas
505
506 Default lSeachAll    := .F.
507 Default dDataMov     := dDataBase
508
509 ParamType 0 Var      lSeachAll      As Logical      Default .F.
510
511 LjGrvLog("Importa_Orcamento:STWISPafSearch", "Parâmetro lCancel : ", lCancel)
512
513 /* Chamada Busca de opções de orçamento */
514 aAux := STWISearchOptions( aField[3] , "/", STFCfg("lPafEcf"), lSeachAll ,
515                          dDataMov )
516
517 lUserSelect := aAux[1]

```

```

518 aOptionSales := aAux[2]
519
520
521 If Len(aOptionSales) > 0
522
523     LjGrvLog("Importa_Orcamento:STWISPaSearch","Chama rotina: STWISImpAllSelected")
524     LjGrvLog("Importa_Orcamento:STWISPaSearch","Parâmetro aOptionSales: ", aOptionSales)
525
526     aImported := STWISImpAllSelected( aOptionSales , "D" , lCancel )
527
528     LjGrvLog("Importa_Orcamento:STWISPaSearch","Parâmetro de retorno aImported      :", aImported)
529
530     If aImported[1]      // Continua?
531
532         LjGrvLog("Importa_Orcamento:STWISPaSearch","aImported[1] = .T.")
533         LjGrvLog("Importa_Orcamento:STWISPaSearch","Chama rotina: STWISRegSale")
534         LjGrvLog("Importa_Orcamento:STWISPaSearch","Parâmetro aImported[4]: ", aImported[4])
535
536         /* Chamar Importação direto */
537         STWISRegSale( aImported[4] , lCancel )
538
539         LjGrvLog("Importa_Orcamento:STWISPaSearch","Chama rotina: STIOpenCash")
540         STIOpenCash()
541
542     Else
543
544         LjGrvLog("Importa_Orcamento:STWISPaSearch","aImported[1] = .F.")
545         lRet := .F.
546
547     EndIf
548
549 EndIf
550
551
552 Return Nil
553
554 /*{Protheus.doc} STWGetMsg
555     Função que retorna a mensagem referente a não importação do orçamento
556     @type Function
557     @return cMsg0rc, caractere, retorna o motivo da não importação do orçamento
558 */
559 Function STWGetMsg()
560 Return cMsg0rc
561
562 /*{Protheus.doc} STIAtGetOr
563     Função que atualiza o Array aGet0rc
564     @param aGet, Array , Posição que irremos atualizar.
565     @type Function
566     @return Nil
567 */
568 Static Function STIAtGetOr(aGet)
569

```

```
570 Local nI      := 0  // Contador.
571
572 If Len(aGet) > 0
573     For nI := Len(aGet) To 1 Step -1
574         ADel(STIGet0rc(), aGet[nI] )
575     Next
576     Asize(STIGet0rc(), Len(STIGet0rc())-Len(aGet))
577 EndIf
578
579 Return
```

Componentes/Increases/STWIncrease.PRW

308 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STWINCREASE.CH"
4
5 //-----
6 /*{Protheus.doc} STWAddIncrease
7 Adiciona acréscimo
8
9 @param  nIncrease      Valor do Acréscimo
10 @param  nPercIncrease  Percentual do Juros Aplicado
11 @return lRet - Indica se adicionou acrescimo
12 @sample
13 */
14 //-----
15 Function STWAddIncrease( nIncrease , nPercIncrease)
16
17 Local lRet      := .F.
18 Local nTemAcrs  := Iif(STIGetMult() .And. STBTaxRet("NF_ACRESCI") > 0, STBTaxRet("NF_ACRESCI") + nIncrease,0)
19 Local nL1JUROS := 0
20      atribuido
21 Default nIncrease := 0
22 Default nPercIncrease := 0
23
24 ParamType 0 Var  nIncrease  As Numeric  Default 0
25 ParamType 1 Var  nPercIncrease  As Numeric  Default 0
26
27 lRet := STBTaxAlt( "NF_ACRESCI", Iif( nTemAcrs > 0, nTemAcrs, nIncrease))
28
29 If nTemAcrs > 0
30     nL1JUROS := STDGPBasket("SL1","L1_JUROS") + nPercIncrease
31 EndIf
32
33 STFSetTot( "L1_JUROS" , Iif( nL1JUROS > 0, nL1JUROS, nPercIncrease))
34 STDSPBasket("SL1","L1_VLRJUR", Iif( nTemAcrs > 0, nTemAcrs, nIncrease ))
35
36 STFRefTot()
37
38 Return lRet
39
40
41 //-----
42 /*{Protheus.doc} STWGetIncrease
43 Busca Valor do acréscimo da venda
44
45 @param
46 @return nIncrease      Retorna valor do acréscimo da venda
47 @sample
48 */
49 //-----
50 Function STWGetIncrease()
51

```

// Indica se adicionou acrescimo
// Armazena o valor de acrescimo se já atribuido
// Percentual de juros quando tem acrescimo já

```

52 Local nIncrease      := .F.           // Retorna valor acrescimo
53 Local oTotal         := STFGetTot()    // Model totalizador
54
55 nIncrease := oTotal:GetValue("L1_ACRSFIN")
56
57 Return nIncrease
58
59
60 //-----
61 /*{Protheus.doc} STWPrintIncrease
62 Impressao do acrescimo
63
64 @param  nIncrease      Valor do acrescimo
65 @return lRet           Retorna se registrou o acrescimo na impressora
66 @sample
67 */
68 //-----
69 Function STWPrintIncrease( nIncrease )
70
71 Local lRet             := .T.           // Retorna se registro acrescimo na impressora
72 Local aRetPrinter      := {}           // Armazena retorno da impressora
73 Local oTotal           := STFGetTot()   // Totalizador
74 Local lItEntrega       := STBTemEntr()  // Indica se a venda tem item de entrega
75
76 Default nIncrease      := 0
77
78 ParamType 0 Var  nIncrease      As Numeric  Default 0
79
80 If STFuseFiscalPrinter()
81
82     If STWGetIsOpenReceipt() // Cupom Aberto?
83
84         If nIncrease <= 0
85
86             nIncrease := oTotal:GetValue( "L1_ACRSFIN" ) + oTotal:GetValue( "L1_FRETE" ) + ;
87                     oTotal:GetValue( "L1_SEGURO" ) + oTotal:GetValue( "L1_DESPESA" )
88
89         EndIf
90
91         If nIncrease > 0
92
93             If lItEntrega
94                 nIncrease := STBNoRound( nIncrease * STBFactor(.F.)[1] )
95             EndIf
96
97             aRetPrinter := STFFireEvent( ProcName(0)           ,; // Nome do processo
98                                         "STPrintIncrease"      ,; // Nome do evento
99                                         {AllTrim(Str(nIncrease,14,2)) } // 01 - VALOR ACRESOIMO
100
101             If Len(aRetPrinter) > 0 .AND. ValType(aRetPrinter[1]) == "N"
102
103                 If aRetPrinter[1] == 0

```

```

104         lRet := .T.
105     Else
106         lRet := .F.
107         STFMessage("STIncrease","STOP",STR0001) //"Não foi possível registrar o Acréscimo no Total do Cupom. Operação não efetuada."
108     EndIf
109
110     Else
111         lRet := .F.
112         STFMessage("STIncrease","STOP",STR0001) //"Não foi possível registrar o Acréscimo no Total do Cupom. Operação não efetuada."
113     EndIf
114
115     Else
116         lRet := .T.
117     EndIf
118
119     Else
120         lRet := .T.
121     EndIf
122
123 Else
124     lRet := .T.
125 EndIf
126
127 STFShowMessage("STIncrease")
128
129 Return lRet
130
131
132 //-----
133 /*{Protheus.doc} STWAddFreight
134 Adiciona Frete, Seguro e Despesa no totalizador
135
136 @return Nil
137 @sample
138 */
139 //-----
140 Function STWAddFreight(aSL2)
141
142 Default aSL2 := {}
143
144 /*
145     Adiciona no totalizador frete do tipo FOB, que será tratado como acréscimo no CUPOM FISCAL.
146     Tipo CIF não adiciona no totalizador, apenas na cesta
147 */
148 If STDGPBasket( "SL1" , "L1_TPFRET" ) $ "CFTRD" // Tipo de frete. "F" -> FOB , "C" -> CIF, "T" -> Terceiros, "R" -> Próprio Remetente, "D" -> Próprio Destinatário
149     // Realiza o Rateio do Frete para SL2
150     If STDGPBasket('SL1', 'L1_FRETE') > 0 .OR. STDGPBasket('SL1', 'L1_SEGURO') > 0 .OR. STDGPBasket('SL1', 'L1_DESPESA') > 0
151         STWRecFret(aSL2,STDGPBasket('SL1', 'L1_FRETE'), STDGPBasket('SL1', 'L1_DESPESA'), STDGPBasket('SL1', 'L1_SEGURO'))
152         STWFreightRefsh()
153     EndIf
154 EndIf
155

```

```

156 Return Nil
157
158 //-----
159 /*{Protheus.doc} STWFreightRefsh
160 Atualiza Frete, Seguro e Despesa no totalizador
161
162 @param
163 @return Nil
164 @sample
165 */
166 //-----
167 Function STWFreightRefsh()
168
169 Local oTotal      := STFGetTot()    // Model totalizador
170 Local nFreDesSeg  := 0              // Valor Frete, Seguro e Despesa
171
172 STFSetTot( "L1_FRETE" , STDGPBasket( "SL1" , "L1_FRETE" ) )
173 STFSetTot( "L1_SEGURO" , STDGPBasket( "SL1" , "L1_SEGURO" ) )
174 STFSetTot( "L1_DESPESA" , STDGPBasket( "SL1" , "L1_DESPESA" ) )
175
176 nFreDesSeg := oTotal:GetValue("L1_FRETE") + oTotal:GetValue("L1_SEGURO") + oTotal:GetValue("L1_DESPESA")
177
178 STFSetTot( "L1_VLRLIQ" , oTotal:GetValue("L1_VLRTOT") + nFreDesSeg )
179 STFSetTot( "L1_VALBRUT" , oTotal:GetValue("L1_VALMERC") + oTotal:GetValue("L1_ACRSFIN") )
180 STFSetTot( "L1_VLRTOT" , oTotal:GetValue("L1_VLRTOT") + nFreDesSeg ) // Adiciona no VLRTOT por ultimo para nao duplicar
181 STFSetTot( "L1_VALICM" , STBTaxRet(Nil ,"NF_VALICM") )
182
183 // Atualiza Total
184 STFRefTot()
185
186 Return Nil
187
188 //-----
189 /* {Protheus.doc} STWRecFret
190 Calculo o valor do Frete,Seguro e Despesa
191
192 @return lRet - Ok ou não ok?
193 @obs
194 @sample
195 */
196 //-----
197
198 Static Function STWRecFret(aSl2,nTotFret,nTotDesp,nTotSegu)
199
200 Local nX      := 0
201 Local nPosEnt := 0 //Posicao campo entrega na SL2
202 Local nValItem := 0 //Valor Item sem frete
203 Local nTotITEnt := 0 //Total dos itens de entrega
204 Local nVlFrete := 0 //Valor frete item
205 Local nVlDespe := 0 //Valor despesas item
206 Local nVlSegur := 0 //Valor seguro item
207 Local lEntrega := .F. //Flag para verificar se existe item de entrega

```

```

208 Local nDifFret      := 0      //Valor auxiliar de frete
209 Local nDifDesp      := 0      //Valor auxiliar de despesas
210 Local nDifSegu      := 0      //Valor auxiliar de seguro
211 Local nUltItEnt      := 0      //Ultimo item de entrega
212 Local nTotVenda      := 0      //Total da venda
213 Local nUltItVen      := 0      //Ultimo item da venda
214 Local nFreDesSeg     := 0      //Valor frete + despesas + seguro do item
215
216 If Len(aSL2) > 0
217     nPosEnt := AScan(aSL2[1] , {|x| x[1] == "L2_ENTREGA"})
218
219     //Verifico se nos itens possui algum item de entrega
220     If nPosEnt > 0
221         For nX := 1 to Len(aSL2)
222             nFreDesSeg := STBTaxRet(nX,"IT_FRETE") + STBTaxRet(nX,"IT_SEGURO") + STBTaxRet(nX,"IT_DESPESA")
223
224             If !LjIsTesISS(STDGPBasket("SL2", "L2_PRODUTO", nX), STDGPBasket("SL2", "L2_TES", nX)) //Nao rateia Frete/Seguro/Despesas para Servico
225                 If aSL2[nX][nPosEnt][2] $ "3|5"
226                     lEntrega := .T.
227                     nTotITEnt += STBTaxRet(nX,"IT_VALMERC") - nFreDesSeg
228                     nUltItEnt := nX
229                 Else
230                     nTotVenda += STBTaxRet(nX,"IT_VALMERC") - nFreDesSeg
231                     nUltItVen := nX
232                 EndIf
233             EndIf
234         Next nX
235     EndIf
236
237     For nX := 1 to Len(aSL2)
238         nFreDesSeg := STBTaxRet(nX,"IT_FRETE") + STBTaxRet(nX,"IT_SEGURO") + STBTaxRet(nX,"IT_DESPESA")
239         nValItem := STBTaxRet(nX,"IT_VALMERC") - nFreDesSeg
240
241         If !LjIsTesISS(STDGPBasket("SL2", "L2_PRODUTO", nX), STDGPBasket("SL2", "L2_TES", nX)) //Nao rateia Frete/Seguro/Despesas para Servico
242             If lEntrega
243                 If aSL2[nX][nPosEnt][2] $ "1|2|4" //Se nao for entrega zero o valor do Frete, Seguro e Despesa
244                     STBTaxAlt("IT_FRETE" , 0, nX)
245                     STBTaxAlt("IT_DESPESA", 0, nX)
246                     STBTaxAlt("IT_SEGURO" , 0, nX)
247
248                     Loop //Pula registro pois nao trata retira se houver itens de entrega
249                 EndIf
250
251                 nVlFrete := STBArred((nTotFret * (nValItem / nTotITEnt)))
252                 nVlDespe := STBArred((nTotDesp * (nValItem / nTotITEnt)))
253                 nVlSegur := STBArred((nTotSegu * (nValItem / nTotITEnt)))
254             Else
255                 nVlFrete := STBArred((nTotFret * (nValItem / nTotVenda)))
256                 nVlDespe := STBArred((nTotDesp * (nValItem / nTotVenda)))
257                 nVlSegur := STBArred((nTotSegu * (nValItem / nTotVenda)))
258             EndIf
259

```

```

260      nDifFret += nVlFrete
261      nDifDesp += nVlDespe
262      nDifSegu += nVlSegur
263
264      //Quando for o ultimo item, verifica se teve diferenca entre o valor total e o valor calculado
265      If (lEntrega .And. nX == nUltItEnt) .Or. (!lEntrega .And. nX == nUltItVen)
266          nDifFret := nTotFret - nDifFret
267          nDifDesp := nTotDesp - nDifDesp
268          nDifSegu := nTotSegu - nDifSegu
269
270          //Caso tenha diferenca de arredondamento, é acertado no ultimo item.
271          If nDifFret > 0 .And. Abs(nDifFret) <= 0.05 //Acumulado de frete maior que o conteudo armazenado na SL1, necessario diminuir
272              nVlFrete += Abs(nDifFret)
273          ElseIf nDifFret < 0 .And. Abs(nDifFret) <= 0.05 //Acumulado de frete menor que o conteudo armazenado na SL1, necessario aumentar
274              nVlFrete -= Abs(nDifFret)
275          EndIf
276
277          If nDifDesp > 0 .And. Abs(nDifDesp) <= 0.05 //Acumulado de despesa maior que o conteudo armazenado na SL1, necessario diminuir
278              nVlDespe += Abs(nDifDesp)
279          ElseIf nDifDesp < 0 .And. Abs(nDifDesp) <= 0.05 //Acumulado de despesa menor que o conteudo armazenado na SL1, necessario aumentar
280              nVlDespe -= Abs(nDifDesp)
281          EndIf
282
283          If nDifSegu > 0 .And. Abs(nDifSegu) <= 0.05 //Acumulado de seguro maior que o conteudo armazenado na SL1, necessario diminuir
284              nVlSegur += Abs(nDifSegu)
285          ElseIf nDifSegu < 0 .And. Abs(nDifSegu) <= 0.05 //Acumulado de seguro menor que o conteudo armazenado na SL1, necessario aumentar
286              nVlSegur -= Abs(nDifSegu)
287          EndIf
288      EndIf
289
290      //Adiciona valores na Matxfis
291      STBTaxAlt("IT_FRETE" , nVlFrete ,nX)
292      STBTaxAlt("IT_DESPESA", nVlDespe ,nX)
293      STBTaxAlt("IT_SEGURO" , nVlSegur ,nX)
294
295      //Adiciona valores na cesta
296      STDSPBasket("SL2", "L2_BASEICM", STBTaxRet(nX, "IT_BASEICM"), nX)
297      STDSPBasket("SL2", "L2_VALICM" , STBTaxRet(nX, "IT_VALICM") , nX)
298      STDSPBasket("SL2", "L2_VALFRE" , nVlFrete, nX)
299      STDSPBasket("SL2", "L2_DESPESA" , nVlDespe, nX)
300      STDSPBasket("SL2", "L2_SEGURO" , nVlSegur, nX)
301      STDSaveSale(nX)
302  EndIf
303  Next nX
304
305      STDSPBasket("SL1", "L1_VALICM", STBTaxRet(, "NF_VALICM")) //Atualiza valor ICMS da SL1 na cesta
306 EndIf
307
308 Return Nil

```

Componentes/Integration/STBRaas.prw

26 linhas após remoção de metadados de autoria/data

```
1 #INCLUDE "TOTVS.CH"
2
3 Static oRAAS := Nil
4
5 //-----
6 /*{Protheus.doc} STBGetRaas()
7 Retorna objeto de RAAS Integration da variavel static
8
9 @type      function
10 @return    LjRAASIntegration, Objeto da variavel static
11 */
12 //-----
13 Function STBGetRaas()
14 Return oRAAS
15
16 //-----
17 /*{Protheus.doc} STBSetRaas()
18 Atualiza objeto de RAAS Integration da variavel static
19
20 @type      function
21 @return    LjRAASIntegration, Objeto da variavel static
22 */
23 //-----
24 Function STBSetRaas(oRaasI)
25     oRAAS := oRaasI
26 Return
```

Componentes/Interface/POSCSS.CH

51 linhas após remoção de metadados de autoria/data

```

1 #DEFINE CSS_BAR_TOP          01
2 #DEFINE CSS_BAR_BOTTOM      02
3 #DEFINE CSS_BAR_ALERT       03
4 #DEFINE CSS_BAR_BUTTON      04
5 #DEFINE CSS_BAR_INFO        05
6 #DEFINE CSS_BAR_LOGO        06
7 #DEFINE CSS_BAR_MENU        07
8 #DEFINE CSS_BTN_FOCAL       08
9 #DEFINE CSS_BTN_NORMAL      09
10 #DEFINE CSS_BTN_ATIVO       10
11 #DEFINE CSS_BROWSE          11
12 #DEFINE CSS_BROWSE_HEADER_LEFT 12
13 #DEFINE CSS_BROWSE_HEADER_RIGHT 13
14 #DEFINE CSS_LISTBOX         14
15 #DEFINE CSS_PANEL_LOGO      15
16 #DEFINE CSS_PANEL_OPTION_TOP 16
17 #DEFINE CSS_PANEL_CONTENT   17
18 #DEFINE CSS_PANEL_OPTION_BOTTOM 18
19 #DEFINE CSS_PANEL_CONTEXT   19
20 #DEFINE CSS_PANEL_HEADER    20
21 #DEFINE CSS_PANEL_FOOTER    21
22 #DEFINE CSS_BREADCUMB       22
23 #DEFINE CSS_LABEL_HEADER    23
24 #DEFINE CSS_LABEL_FOCAL     24
25 #DEFINE CSS_LABEL_NORMAL    25
26 #DEFINE CSS_GET_FOCAL       26
27 #DEFINE CSS_GET_NORMAL      27
28 #DEFINE CSS_LABEL_TOTAL     28
29 #DEFINE CSS_BG              29
30 #DEFINE CSS_LABEL_SUBTITLE  30
31 #DEFINE CSS_LABEL_TRIAL     31
32 #DEFINE CSS_BTN_BUY         32
33 #DEFINE CSS_LABEL_REQUIRED  33
34 #DEFINE CSS_GET_ERROR       34
35 #DEFINE CSS_BTN_LUPA        35
36 #DEFINE CSS_BTN_BARCODE     36
37 #DEFINE CSS_PANEL_BKGLGIN   37
38 #DEFINE CSS_CHECKBOX_DEFAULT 38
39 #DEFINE CSS_PANEL_LOGINHEADER 39
40 #DEFINE CSS_PANEL_LOGINMAIN 40
41 #DEFINE CSS_PANEL_LOGINASKREG 41
42 #DEFINE CSS_PANEL_LINEAUX   42
43 #DEFINE CSS_PANEL_LOGINFOOTER 43
44 #DEFINE CSS_CHOOSE_PAYMENT  44
45 #DEFINE CSS_PAYMENT_ITEM    45
46 #DEFINE CSS_PANEL_WIZARD    46
47 #DEFINE CSS_COMBOBOX        47
48 #DEFINE CSS_PANEL_BTN_FOOTER 48
49 #DEFINE CSS_GET_TOTAL       49
50 #DEFINE CSS_STEP_WIZ        50
51 #DEFINE CSS_LABEL_ALERT     51

```

Componentes/Interface/STIBONIFSELECTION.PRW

471 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "FWMBROWSE.CH"
2 #INCLUDE "FWMVCDEF.CH"
3 #INCLUDE "PROTHEUS.CH"
4 #INCLUDE "APWIZARD.CH"
5 #INCLUDE "FWADAPTEREAI.CH"
6 #INCLUDE 'FWBROWSE.CH'
7 #INCLUDE "POSCSS.CH"
8 #INCLUDE "STPOS.CH"
9 #INCLUDE "PARMTYPE.CH"
10 #INCLUDE "STIITEMREGISTER.CH"
11 #INCLUDE "PARMTYPE.CH"
12 #INCLUDE "FWEVENTVIEWCONSTS.CH"
13 #INCLUDE "FWADAPTEREAI.CH"
14 #INCLUDE "STIPAYNCCBC.CH"
15 #INCLUDE "STPOS.CH"
16 #INCLUDE "STIBONIFSELECTION.CH"
17
18 Static oLblSelTotal      := Nil           // Objeto do label que informa o valor total das Bonificacoes selecionadas.
19 Static oLblTotalVend     := Nil           // Label que informa o restante para pagamento
20 Static aBonificacoes     := {}
21
22 //-----
23 /*{Protheus.doc} STIBONIFSelection
24 Responsavel por montar a interface de apresentacao de Bonificacoes
25
26 @return oMainPanel - Objeto contendo o painel de visualizacao das bonificacoes
27
28 */
29 //-----
30
31 Function STIBONIFSelection(nValueItem,cCodReg,cItemGrid,cProd,;
32                           aBonifCli, lPgto,cForma,oPnlAdconal)
33 Local oPanelMVC           := STIGetPanel() // Objeto do Panel onde a interface de selecao de clientes sera criada
34 Local oMainPanel         := TPanel():New(00,00,"",oPanelMVC,,,,,;
35                                         oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2) // Painel de Get de Consulta
36
37 /*
38     Declaracao dos Objetos
39 */
40 Local oLblCab             := Nil           // Objeto do label "Bonificações do Cliente"
41 Local oLblList            := Nil           // Objeto do label "Digite aqui para filtrar"
42 Local oListPanel          := Nil           // Objeto do panel onde será criado o Browse com as bonificacoes do cliente.
43 Local oListBonif          := Nil           // Objeto do tipo TListBox, que exibe as bonificacoes do cliente.
44 Local oLblCliTotal        := Nil           // Objeto do label que informa o valor total das bonificacoes do cliente.
45 Local oTotal              := STFGetTot()   // Recebe o Objeto totalizador
46
47 /*
48     Variaveis de posicionamento de objetos
49 */
50 Local nPosVerCab          := 0.015         // Posicao Horizontal do Say "Bonificações do Cliente"
51 Local nPosVerPanListBox   := oMainPanel:nHeight/2 * 0.0666// Posicao vertical do objeto TPanel oListPanel
52 Local nPosVerTListBox     := oMainPanel:nHeight/2 * 0.05 // Posicao vertical do Say oLblList

```

```

53 Local nListPosHor      := oMainPanel:nWidth/2 * 0.05    // Posicao Horizontal do objeto TListBox oListBonif
54
55
56 /*/
57   Botão "Selecionar Cliente"
58 /*/
59 Local oButton          := Nil                          // Botao "Selecionar Cliente"
60 Local nBtnWidPerc      := 0.25                        // Percentual de Largura do Botão "Selecionar Cliente"
61 Local nBtnHeiPerc      := 0.08                        // Percentual de Altura do Botão "Selecionar Cliente"
62 Local nBtnVerPerc      := 0.8                         // Percentual da Posição Vertical do Botão "Selecionar Cliente"
63 Local nBtnHorPerc      := 0.65                        // Percentual da Posição Horizontal do Botão "Selecionar Cliente"
64
65 Local aListBonif        := {}                          // Itens do objeto oListBonif
66 Local nX                := 0                          // Contador do FOR
67 Local oListFont         := TFont():New("Courier New")   // Fonte utilizada no listbox
68
69 Local nSaldoBonif       := 0                          // Total das Bonificacoes do cliente
70 Local nTotalSel         := 0                          // Total das Bonificacoes selecionadas
71 Local oTotFont          := TFont():New( "Arial" ,Nil,-13,Nil,.T. ) // Objeto da fonte dos totalizadores
72
73 Local nValor            := 0                          // Valor do produto
74 Local nVTBonif          := 0                          // Valor total da bonificacao
75 Local cReturn           := ""                         // Objeto de retorno
76 Local cInfo             := ""                         // Informação para a tela
77 Local nTamProd          := SB1->(TamSx3("B1_COD"))[1])
78 Local cProdBonif        := "" //Produto bonificacao
79
80 Default nValueItem      := 0                          // Valor do Item
81 Default cCodReg         := ""                         // Código do Registro
82 Default cItemGrid       := ""                         // Item do Grid
83 Default cProd           := ""                         // Produto
84 Default aBonifCli       := {}                         // Array para Bonificação do Cliente
85 Default lPgto           := .F.                       // Valida a forma de pagamento
86 Default cForma          := ""                         // Forma de pagamento a ser validado
87 Default oPnlAdconal     := Nil                        // Objeto do Resumo de Pagamento
88
89 For nX := 1 To Len(aBonifCli)
90   If Len(aBonifCli[nX]) < 6
91     aSize(aBonifCli[nX], 6)
92
93     cProdBonif := AllTrim(SubStr(aBonifCli[nX,01],1, nTamProd )) //Lê o codigo do produto selecionado ref. a bonificacao
94     If Rat('-', cProdBonif) > 0 //Retira o traco no final do codigo do produt
95       cProdBonif := Left(cProdBonif, Rat('-', cProdBonif)-1)
96     EndIf
97     aBonifCli[nX, 05] := cProdBonif
98     aBonifCli[nX, 06] := .F.
99   EndIf
100
101   If !Empty(aBonifCli[nX,2])//Valor do produto possui desconto em Porcentagem
102     nValor := aBonifCli[nX,4] - ((aBonifCli[nX,4] / 100) * aBonifCli[nX,2])
103     If nValor > 0
104       aAdd(aListBonif,aBonifCli[nX,1]+STR0001+AllTrim(Str(aBonifCli[nX,4]))+STR0002;

```

```

105             +AllTrim(str(nValor)))/" Valor bonificado De :### Para:"
106         Endif
107     Else
108         If !Empty(aBonifCli[nX,3])//Valor do produto possui desconto em Valor
109             nValor := (aBonifCli[nX,4] - aBonifCli[nX,3])
110             If nValor > 0
111                 aAdd(aListBonif,aBonifCli[nX,1]+STR0001+AllTrim(Str(aBonifCli[nX,4]))+STR0002;
112                     +AllTrim(str(nValor)))/" Valor bonificado De :### Para:"
113             Endif
114         Else
115             //valor cheio do produto
116             nValor := aBonifCli[nX,4]
117             If nValor > 0
118                 aAdd(aListBonif,aBonifCli[nX,1]+STR0001+AllTrim(Str(aBonifCli[nX,4]))) // " Valor bonificado De : "
119             Endif
120         Endif
121     Endif
122
123 Next nX
124
125 If !Empty(cForma)
126     cInfo := STR0003+ cForma +STR0004/"Pague com "##" e compre produtos bonificados: "
127 Else
128     cInfo := STR0005
129 Endif
130
131 If !Empty(aListBonif)
132     /*/
133     Cabecalho da tela "
134     /*/
135     oLblCab:= TSay():New(POSVERT_CAB,POSHOR_1,{||cInfo},oMainPanel,,,,,T,,,,)
136     oLblCab:SetCSS( POSCSS (GetClassName(oLblCab), CSS_BREADCUMB ))
137
138     /* Instancia do PaineL do ListBox */
139     oListPanel := TPanel():New(nPosVerPanListBox,000,"",oMainPanel,;
140                               ,,,,;
141                               ,oMainPanel:nWidth/2.15,oMainPanel:nHeight/5)
142
143     /* Instancia do Label do ListBox */
144     oLblList:= TSay():New(000,nListPosHor,{||STR0006},oListPanel,,;
145                          oTotFont,,,,,T,,,,)//"Selecione a bonificação desejada e clique em 'Avançar'"
146     oLblList:SetCSS( POSCSS (GetClassName(oLblList), CSS_LABEL_FOCAL ))
147
148     /* Instancia do ListBox */
149     oListBonif := TListBox():Create(oListPanel, nPosVerTListBox, nListPosHor, Nil,;
150                                   aListBonif, oListPanel:nWidth/2.2, oListPanel:nHeight/2.5,,;
151                                   ,,,,T,,;
152                                   ,{||STISelectBonif(oListBonif,@aBonifCli)},oListFont)
153     oListBonif:SetCSS( POSCSS (GetClassName(oListBonif), CSS_LISTBOX ))
154
155     /* Instancia do Label que informa o restante a ser pago na venda */
156     oLblTotalVend := TSay():New(oMainPanel:nHeight/4, nListPosHor,;

```

```

157         {||STR0007+ AllTrim(Str(oTotal:GetValue("L1_VLRTOT"),12,2)) +STR0008},;
158         oMainPanel,,oTotFont,,;
159         ,,.T.,;
160         ,,,10)//"Restam " ##" para pagamento."
161     oLblTotalVend:SetCSS( POSCSS (GetClassName(oLblTotalVend), CSS_LABEL_FOCAL ))
162
163     /* Instancia do botao "Avançar" */
164     oButton := TButton():New( POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0009,oMainPanel,;
165         { || STIBONIFBasket(oListBonif,aBonifCli, lPgto,cForma,;
166             oMainPanel) }, LARGBTN,ALTURABTN,,,,.T.) //"Avançar"
167     oButton:SetCSS( POSCSS (GetClassName(oButton), CSS_BTN_FOCAL ))
168
169     /* Button: Cancelar */
170     If !Empty(cForma)
171         oBtnCa := TButton():New(POSVERT_BTNFOCAL,POSHOR_1,STR0010,oMainPanel,{|| STIPayment() },LARGBTN,ALTURABTN,,,,.T.) //"Cancelar"
172         oBtnCa:SetCSS( POSCSS(CSS_BTN_ATIVO))
173     Endif
174
175 Else
176     STIExchangePanel( { || STIPayment() } ) //Exibe tela de pagamento
177 Endif
178 //Return oMainPanel => nao pode usar esse tipo de retorno porque no Cancela ocorre travamento da tela nao permitindo selecionar nenhuma forma de pagamento
179 Return .T.
180
181 //-----
182 /*{Protheus.doc} STIBONIFBasket
183 Responsavel por inserir o produto bonificado na cesta
184
185 @return Nil
186
187 */
188 //-----
189 Function STIBONIFBasket(oListBonif,aBonifCli, lPgto,cForma,oMPanel2)
190 Local nItemLine := STDPBLength("SL2") + 1 // para acrescentar mais uma linha na cesta
191 Local cCliCode := STDGPBasket("SL1","L1_CLIENTE") // Código do Cliente
192 Local cCliLoja := STDGPBasket("SL1","L1_LOJA") // Código da Loja
193 Local cCliType := STDGPBasket("SL1","L1_TIPOCLI") // Tipo do Cliente
194 Local nPos := oListBonif:GetPos() // Posição da Lista de Bonificação
195 Local nVLBonif := STIGetBonif(.F.) // Obter Valor da Bonificação
196 Local lRegistred := .F. // verifica se o produto foi inserido na cesta
197 Local nSecItem := Seconds() // Inicia os segundos iniciais entre um item e o outro logo apos a inclusão do CPF
198 Local nMoeda := 1 // Moeda padrao
199 Local nTotItem := 0 // Subtrai os segundos atuais com o ultimo tempo registrado do item anterior
200 Local nValueTotal := 0 // valor total com a bonificacao
201 Local nVLDesc := 0 // valor do desconto
202 Local nVaLor := 0 // valor do produto com bonificacao
203 Local cProdBonif := "" // produto bonificado
204 Local cTPDesc := "" // tipo de desconto
205
206 //Criação de tela auxiliar
207 Local oPanel := STIGetPanel() // Objeto do Panel onde a interface sera criada - Painel filho
208 Local nPosHozPan := oPanel:nWidth/80 // Posicao horizontal do painel - Painel Pai

```

```

209 Local nPosAltPan      := 0           // Posicao: Altura do Painel
210 Local nTamLagPan      := 310         // Tamanho: Largura do Painel
211 Local nTamAltPan      := 200         // Tamanho: Altura do Painel
212 Local bCreatePan      := {||TPanel():New(nPosAltPan,nPosHozPan,"",oPanel,,,,,nTamLagPan,nTamAltPan)} // Bloco de codigo para criacao do Painel adicional
213 Local nC               := 1
214
215 Default oListBonif     := nil         // Objeto Lista de Bonificação
216 Default aBonifCli      := {}         // Array Bonificação Cliente
217 Default lPgto          := .F.        // Valida a forma de pagamento
218 Default cForma         := ""         // Forma de pagamento a ser validado
219 Default oMPanel2       := Nil
220
221 If !Empty(oMPanel2)
222     nPosAltPan := oMPanel2:nHeight/20 // Posicao: Altura do Painel
223 Endif
224
225 For nC := 1 to len(aBonifCli)
226
227     //verifica se o Produto foi marcado
228     If aBonifCli[nC, 06] //Item selecionado?
229
230         nTotItem += seconds() - nSecItem
231         cProdBonif := aBonifCli[nPos,5] //Produto retornado no Array
232
233         If !Empty(aBonifCli[nPos,2])//Valor do produto possui desconto em Porcentagem
234             nVLDesc := (aBonifCli[nPos,4] / 100) * aBonifCli[nPos,2]
235             nVaLor := aBonifCli[nPos,4] - nVLDesc
236             cTPDesc := "P"
237         Else
238             If !Empty(aBonifCli[nPos,3])//Valor do produto possui desconto em Valor
239                 nVaLor := aBonifCli[nPos,4] - aBonifCli[nPos,3]
240                 cTPDesc := "V"
241             Endif
242         Endif
243
244         If nVaLor == 0 //Importante
245             nVaLor := aBonifCli[nPos,4]
246         Endif
247
248         aAdd(aBonificacoes ,{cProdBonif,nVaLor,nItemLine,alltrim(cForma)})
249
250     /*/
251     Registra Item
252     /*/
253     lRegistered := STWItemReg( nItemLine , ; // Item
254                               cProdBonif , ; //Codigo Prod bonificado
255                               cCliCode , ; //Codigo Cli
256                               cCliLoja , ; //Loja Cli
257                               nMoeda , ; //Moeda
258                               nil , ; //Valor desconto
259                               cTPDesc , ; //Tipo desconto ( Percentual ou Valor ) - Tipo de desconto, se "V" - Valor ou "P" - Percentual
260                               Nil , ; //Item adicional?

```

```

261         Nil          , ;      // TES
262         cCliType     , ;      // Tipo do cliente (A1_TIPO)
263         Nil          , ;      // Registra item no cupom fiscal?
264         nValor       , ;      // Preço
265         Nil          , ;      // Tipo do Item
266         Nil          , ;      // Imprime CNPJ no cupom Fiscal
267         Nil          , ;      // Tela do POS está sendo apresentada
268         nTotItem     , ;      // Total dos segundos entre itens
269         Nil          , ;      // Serviço financeiro
270         .T.         )      // Produto bonificado
271
272     nSecItem := Seconds() // Inicia os segundos iniciais entre um item e o outro logo apos a inclusão do ultimo item
273
274     STIGridCupRefresh() // Sincroniza a Cesta com a interface
275
276     If !lPgto
277         STIExchangePanel( { || STIPanItemRegister() } ) //Exibe tela de selecao de itens
278
279     Else
280
281         STIPayment(.F.) //Inclui a Bonificacao
282
283         nVLBonif := STIGetBonif(.F.)
284
285         nValueTotal := STFSaleTotal() - nVLBonif
286
287     If nVLBonif > 0
288         //Atualiza o resumo de pagamentos = GD
289         If cForma <> "MN"
290             STIAddPay("BF", Nil, 1, Nil, Nil, nVLBonif) //Bonificacao
291         Endif
292
293         If !Empty(cForma)
294             oPnlAdconal := Eval(bCreatePan)
295
296         Do Case
297             Case(cForma == "R$")
298                 /* Tela de dinheiro */
299                 STIPayCash(oPnlAdconal)
300             Case (cForma == "CC") .Or. (cForma == "CD")
301                 // Tela do Cartao
302                 STIPayCard(oPnlAdconal, cForma, nValueTotal)
303             Case (cForma == "FI")
304                 // Tela de Financiamento
305                 STIPayFinanciam(oPnlAdconal, "FI")
306             Case (cForma == "FID")
307                 // Tela de Fidelidade
308                 STIPayShopCard(oPnlAdconal)
309             Case (cForma == "CH")
310                 /* Telado do cheque */
311                 STIPayCheck(oPnlAdconal, "")
312             Case (cForma == "VP")

```

```

313         /* Tela do vale presente */
314         STIPayGiftV(oPnlAdconal)
315     Case (cForma == "CP")
316         /* Tela da condicao de pagamento */
317         STIPayCdPg(oPnlAdconal)
318     Case (cForma == "MN")
319         /* Tela da multi negociacao */
320         STIExchangePanel(||| STIPnlMulti(Nil) })
321     Case (cForma == "NB")
322         /* Tela da Nota de Crédito Cód. Barras */
323         STIPayNCCBC(oPnlAdconal)
324     Otherwise
325         STIPayFinancial(oPnlAdconal, cForma)
326 EndCase
327 Endif
328 Endif
329 Endif
330 Exit
331 EndIf
332
333 Next nC
334
335 Return .T.
336
337
338 //-----
339 /*{Protheus.doc} STISelectBonif
340 Responsavel por realizar a marcacao do item selecionado
341
342 @aparam oListNCCs - Objeto do tipo TListBox responsavel por apresentar
343 os produtos bonificados
344 */
345 //-----
346 Static Function STISelectBonif(oListBonif,aBonifCli)
347
348 Local nLine      := 0                                // Linha da Lista de Bonificação
349 Local cTextSelection:= STR0011//"Item Selecionado - "    // Texto do Item Selecionado
350 Local nTam := Len(cTextSelection)
351 Local lDesmarca := .F.
352 Local nC := 0
353
354 Default oListBonif := nil
355 Default aBonifCli := {}
356
357 nLine := oListBonif:nAt
358
359 If nLine > 0
360     If !aBonifCli[nLine, 06] //Para evitar que o mesmo item seja selecionado mais de uma vez.
361         oListBonif:Modify(cTextSelection+oListBonif:GetSelText(),nLine) // Modifica o texto para informar que a NCC foi selecionada
362         aBonifCli[nLine, 06] := .T.
363         lDesmarca := .T.
364     Else

```

```

365         oListBonif:Modify(Substr(oListBonif:GetSelText(),nTam+1),nLine)
366         aBonifCli[nLine, 06] := .F.
367     Endif
368
369     //Demarca os demais
370     If lDesmarca
371         For nC := 1 to Len(aBonifCli)
372             If aBonifCli[nC, 06] .AND. nC <> nLine
373                 oListBonif:Select( nC )
374                 oListBonif:Modify(Substr(oListBonif:GetSelText(),nTam+1),nC)
375                 aBonifCli[nC, 06] := .F.
376             EndIf
377         Next nC
378
379         //oListBonif:nAt := nLine
380         oListBonif:Select( nLine )
381     EndIf
382 EndIf
383
384
385 Return
386
387
388 //-----
389 /*{Protheus.doc} STIGetBonif
390 Retorna o valor total dos produtos bonificados
391
392 @param
393 @return
394 @obs
395 @sample
396 */
397 //-----
398 Function STIGetBonif(lZera)
399 Local nVLTotál := 0
400 Local nX      := 0
401
402 Default lZera := .F.
403
404 If !lZera
405     For nX:= 1 to len(aBonificacoes)
406         nVLTotál += aBonificacoes[nX][2]
407     Next
408 Else
409     aBonificacoes := {}
410 Endif
411
412 Return nVLTotál
413
414
415 //-----
416 /*{Protheus.doc} STIAtuTotal

```

```

417 Atualiza o array total
418 @param
419 @return
420 @obs
421 @sample
422 */
423 //-----
424 Function STIAtuTotal(cForma,cValor)
425
426 Default cForma := ""
427 Default cValor := ""
428
429 aAdd(aBonificacoes,{cForma,cValor} )
430
431 Return Nil
432
433
434 //-----
435 /*/{Protheus.doc} STDDelBonificacao
436 Function Exclui item do array de Bonificacoes
437
438 @return lRet          Retorna se deletou com sucesso
439 @obs
440 @sample
441 */
442 //-----
443 Function STDDelBonificacao(cPosGrde,lCheca )
444 Local nX          := 0 // contador
445 Local lRet        := .F. // Retorna se deletou
446 Local cGetProd    := ""
447
448 Default cPosGrde := 0
449 Default lCheca   := .F. //Checa se o brinde é por forma de pagto, que no caso só pode existir 1 brinde por Forma de Pagto e no caso
450                        // da Forma de Pgto ser limpa e ja exista um brinde incluído é ncessario excluí-lo da cesto
451
452 //Cancelar o item
453 If !lCheca
454     For nX := 1 to len(aBonificacoes)
455         If aBonificacoes[nX][3] == cPosGrde
456             aBonificacoes[nX][2] := 0
457             lRet := .T.
458         Endif
459     Next nX
460 Else
461     For nX := 1 to len(aBonificacoes)
462         If aBonificacoes[nX][4] <> ""
463             aBonificacoes[nX][2] := 0
464             lRet := .T.
465             cGetProd := PADL(aBonificacoes[nX][3],2,"0")//Excluído o item da Cesta para atualizar o valor total da venda
466             STWItemCancel(cGetProd)
467         Endif
468     Next nX

```

```
469 Endif  
470  
471 Return lRet
```

Componentes/Interface/STIBRINDEREGISTER.prw

330 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "FWBROWSE.CH"
3 #INCLUDE "PARMTYPE.CH"
4 #INCLUDE "FWEVENTVIEWCONSTS.CH"
5 #INCLUDE "FWADAPTEREAI.CH"
6 #INCLUDE "FWMVCDEF.CH"
7 #INCLUDE "POSCSS.CH"
8 #INCLUDE "STPOS.CH"
9 #INCLUDE "STICUSTOMERSELECTION.CH"
10 #INCLUDE "STIBRINDEREGISTER.CH"
11
12 //-----
13 /*{Protheus.doc} STIPanBrindeSelection
14 Funcao responsavel pela criacao do Painel de selecao de brindes.
15
16 @return oMainPanel - Objeto contendo o painel principal de selecao de brindes
17 */
18 //-----
19
20 Function STIPanBrindeSelection(cCodReg,cItemGrid,cProd,aProducts,;
21                               lItem, cForma,oPnlAdconal)
22
23 Local oPanelMVC      := STIGetPanel()           // Objeto do Panel onde a interface de selecao de brinde sera criada
24 Local oMainPanel     := TPanel():New(00,00,"",oPanelMVC,,,,,;
25                                     oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2) // Painel de Get de Consulta
26 Local oLblCab        := Nil                    // Objeto do label
27 Local oLblProd       := Nil                    // Objeto do label
28 Local oLblList       := Nil
29 Local oGetList       := Nil                    // TComboBox Escolha do campo a ser utilizado na busca - Get Combo de Busca
30 Local cGetList       := ""                    // Variável do Get - Get Combo de Busca
31 Local oLblSearch     := Nil                    // Objeto do label "Digite aqui para filtrar"
32 Local oListFont      := TFont():New("Courier New") // Fonte utilizada no listbox
33 Local oBtnOk         := Nil                    // Objeto do botao Ok
34 Local oBtnCa        := Nil                    // Objeto do botao Cancelar
35 Local bConfirm       := {}                    // Botão de Confirmação
36 Local cGetCustomer   := ""                    // Qual vendedor
37 Local nSaldo         := 0                     // saldo do brinde em estoque
38 Local nCustPanHor    := 15                    // Posicao horizontal de oCustomerPanel
39 Local cInfo          := ""                    // texto informativo sobre a obrigatoriedade de forma de pagamento
40 Local cReturn        := ""                    // Variável de retorno
41
42 Default lItem        := .T.
43 Default cForma       := "" // Forma de pagamento obrigatoria para adquirir o brinde
44 Default oPnlAdconal := Nil // Objeto do Resumo de Pagamento
45 Default lItem        := .T.
46
47
48 If !Empty(cForma)
49   cInfo := STR0001 + cForma +STR0002 //"Pague com a forma de pagamento " ##" e ganhe 1 brinde abaixo: "
50 Endif
51
52 */

```

```

53     Objetos
54  /*/
55  If lItem
56     oLblCab:= TSay():New(POSVERT_CAB,POSHOR_1,{||STR0003},oMainPanel,,,,,.T,,,,) //"Escolha 1 Brinde, ref. ao produto: "
57
58     cReturn    := oMainPanel
59
60     If !Empty(cProd)
61         cGetCustomer:= alltrim(cProd)+" - "+Posicione( "SB1",1,xFilial( "SB1" )+cProd,"SB1->B1_DESC" )// Variável do Get
62         oLblProd:= TSay():New(POSVERT_CAB+10,POSHOR_1,{||cGetCustomer},oMainPanel,,,,,.T.,CLR_BLUE,CLR_BLUE,,)
63         oLblProd:SetCSS( POSCSS (GetClassName(oLblProd), CSS_BREADCUMB ))
64     Endif
65 Else
66     If Empty(cInfo)
67         oLblCab:= TSay():New(POSVERT_CAB+10,POSHOR_1,{||STR0004},oMainPanel,,,,,.T,,,,) //"Escolha 1 Brinde: "
68     Else
69         oLblCab:= TSay():New(POSVERT_CAB+10,POSHOR_1,{||cInfo},oMainPanel,,,,,.T,,,,)
70     Endif
71     cReturn    := .T.
72 Endif
73 oLblCab:SetCSS( POSCSS (GetClassName(oLblCab), CSS_BREADCUMB ))
74
75
76 If Empty(cForma)
77     oLblList := TSay():New(POSVERT_LABEL2 -35 ,POSHOR_1, {||STR0005}, oMainPanel,,,,,.T.)//'Código /Brinde '
78 Else
79     oLblList := TSay():New(POSVERT_LABEL2 -15 ,POSHOR_1, {||STR0005}, oMainPanel,,,,,.T.)//'Código /Brinde '
80 Endif
81 oLblList:SetCSS( POSCSS (GetClassName(oLblList), CSS_LABEL_FOCAL ))
82
83 If !Empty(cForma)
84     bConfirm := { || STIConfirmBrind(oGetList,cCodReg,cItemGrid+"MB8",cProd,.F., .T.,cForma, oMainPanel)}
85 Else
86     bConfirm := { || STIConfirmBrind(oGetList,cCodReg,cItemGrid+"MB2",cProd,lItem, .T.)}
87 Endif
88
89 If Empty(cForma)
90     oGetList := TListBox():Create(oMainPanel, POSVERT_GET2-30, POSHOR_1, Nil, aProducts, LARG_LIST_CONSULT , ALT_LIST_CONSULT ,,,,,.T.,bConfirm,oListFont)
91 Else
92     oGetList := TListBox():Create(oMainPanel, POSVERT_GET2-50, POSHOR_1, Nil, aProducts, LARG_LIST_CONSULT , ALT_LIST_CONSULT ,,,,,.T.,bConfirm,oListFont)
93 Endif
94 oGetList:SetCSS( POSCSS (GetClassName(oGetList), CSS_LISTBOX ))
95
96 /* Button: OK */
97 oBtnOk := TButton():New( POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0006,oMainPanel,bConfirm, LARGBTN,ALTURABTN,,,,.T.)//"Selecionar Brinde"
98 oBtnOk:SetCSS(POSCSS(CSS_BTN_FOCAL))
99
100 /* Button: Cancelar */
101 If !Empty(cForma)
102     oBtnCa := TButton():New(POSVERT_BTNFOCAL,POSHOR_1,STR0007,oMainPanel,{|| STIPayment() },LARGBTN,ALTURABTN,,,,.T.) //'Cancelar
103     oBtnCa:SetCSS( POSCSS(CSS_BTN_ATIVO))
104 Endif

```

```

105
106 Return cReturn //oMainPanel => nao pode retornar o objeto porque no botão cancelar o sistema trava!
107
108
109 //-----
110 /*{Protheus.doc} STICreateBrowse
111 Responsavel por criar o Browse com as informacoes dos brinde.
112
113 @param oGetList - Objeto que contem a lista de brindes.
114 */
115 //-----
116 Static Function STIConfirmBrind(oGetList,cCodReg,cItemGrid,cProd,lItem,lExibe,cForma,oMPanel2)
117
118 Local aArea := GetArea()           // Posicionamento atual da tabela
119 Local nRecno := 0                  // Número do registro
120 Local nPos   := oGetList:GetPos()  // Posição da lista
121 Local oCliModel := STDGCLiModel() // Model do Cliente
122 Local cNUM     := 0 //Numero de
123 Local cSERIE   := 0 //Numero de Serie
124 Local cPRODUT := 0 //codigo do produto
125 Local cITEM    := 0 //codigo do item
126 Local cBRINDE := 0 //codigo do brinde
127 Local cEMISSA := 0 //data de emissão do brinde
128 Local cCAIXA  := 0 //codigo do caixa
129 Local cREGRA  := 0 //codigo da regra utilizado
130 Local oModel  := STDGPBModel()    // Model da venda
131 Local nBrindes := 1 //quantidade de brindes que o cliente ira receber do mesmo produto
132 Local nItemLine := STDPBLength("MGC") + 1 // Linha do grid
133 Local cPosGrde  := "" //posicao do gride de produtos comprados
134 Local nX        := 0 //contador
135 Local cDOC      := "" //codigo da nota fiscal
136 Local nValueTotal := STFSaleTotal() //Valor total da venda
137 Local nTotal     := STFSaleTotal() // Valor total da venda
138
139 //Criação de tela auxiliar
140 Local oPanel := STIGetPanel()      // Objeto do Panel onde a interface sera criada - Painel filho
141 Local nPosHozPan := oPanel:nWidth/80 // Posicao horizontal do painel - Painel Pai
142 Local nPosAltPan := 0              // Posicao: Altura do Painel
143 Local nTamLagPan := 310            // Tamanho: Largura do Painel
144 Local nTamAltPan := 200            // Tamanho: Altura do Painel
145 Local bCreatePan := {|TPanel():New(nPosAltPan,nPosHozPan,"",oPanel,;
146                      ,,,,nTamLagPan,nTamAltPan)} // Bloco de codigo para criacao do Painel adicional
147
148 Default lExibe := .T.
149 Default cForma := ""
150 Default oMPanel2 := Nil
151
152
153 If !Empty(cForma)
154   nPosAltPan := oMPanel2:nHeight/20 // Posicao: Altura do Painel
155 Endif
156

```

```

157 If nPos > 0
158
159
160 cNUM      := oModel:GetValue("SL1MASTER" , "L1_NUM")
161 cSERIE    := oModel:GetValue("SL1MASTER" , "L1_SERIE")
162 cDOC      := oModel:GetValue("SL1MASTER" , "L1_DOC")
163 cBRINDE   := alltrim(SubStr(oGetList:aitems[nPos],1, 3 ))
164 cPosGrde  := STDGPBasket( "SL2" , "L2_ITEM" )
165
166 //Seta a regra da quantidade de brindes que o cliente recebera do mesmo produto
167 DbSelectArea("MGB")
168 MGB->(DbSetOrder(2)) //MGB_FILIAL+MGB_CODREG+MGB_CODPRO
169 If MGB->(DbSeek(xFilial("MGB")+cCodReg+cItemGrid+PadR(cBRINDE,TamSx3("MGB_CODPRO"))[1]))
170     nBrindes:= MGB->MGB_QTDPRO
171 Endif
172
173
174 For nX := 1 to nBrindes
175
176     If nItemLine > 1
177         /*/
178         Para o primeiro item nao realizar addline, pois o grid já traz linah em branco
179         /*/
180         STDPBAddLine("MGC")
181     EndIf
182
183     STDSPBasket("MGC","MGC_FILIAL" , xFilial("MGC"))
184     STDSPBasket("MGC","MGC_NUM" , cNUM)
185     STDSPBasket("MGC","MGC_PRODUT" , alltrim(cProd))
186     STDSPBasket("MGC","MGC_ITEM" , cPosGrde)
187     STDSPBasket("MGC","MGC_BRINDE" , cBRINDE)
188     STDSPBasket("MGC","MGC_EMISSA" , dDataBase)
189     STDSPBasket("MGC","MGC_SERIE" , cSERIE)
190     STDSPBasket("MGC","MGC_CAIXA" , xNumCaixa())
191     STDSPBasket("MGC","MGC_REGRA" , cCodReg)
192     STDSPBasket("MGC","MGC_SITUA" , "00")
193     STDSPBasket("MGC","MGC_DOC" , cDOC)
194     STDSPBasket("MGC","MGC_PGT0" , cForma) //necessario para poder excluir quando o cliente mudar de forma de pagto
195
196     Next
197
198 Endif
199
200 If lExibe
201     If lItem //Exibe tela de itens de produtos
202         STIExchangePanel( { || STIPanItemRegister() } )
203     Else
204
205         If Empty(cForma)
206             STIPayment(.F.)
207             STIAddPay("R$", Nil, 1, Nil, Nil,nTotal)//Total da venda
208         Else

```

```

209         STIPayment(.F.)
210
211         oPnlAdconal := Eval(bCreatePan)
212
213         Do Case
214             Case(cForma == "R$")
215                 // Tela de dinheiro
216                 STIPayCash(oPnlAdconal)
217             Case (cForma == "CC") .Or. (cForma == "CD")
218                 // Tela do Cartao
219                 STIPayCard(oPnlAdconal, cForma, nValueTotal)
220             Case (cForma == "FI")
221                 // Tela de Financiamento
222                 STIPayFinancial(oPnlAdconal, "FI")
223             Case (cForma == "FID")
224                 // Tela de Fidelidade
225                 STIPayShopCard(oPnlAdconal)
226             Case (cForma == "CH")
227                 // Telado do cheque
228                 STIPayCheck(oPnlAdconal, "")
229             Case (cForma == "VP")
230                 // Tela do vale presente
231                 STIPayGiftV(oPnlAdconal)
232             Case (cForma == "CP")
233                 // Tela da condicao de pagamento
234                 STIPayCdPg(oPnlAdconal)
235             Case (cForma == "MN")
236                 // Tela da condicao de pagamento
237                 STIExchangePanel({|| STIPnlMulti(Nil) })
238             Case (cForma == "NB")
239                 // Tela da Nota de Crédito Cód. Barras
240                 STIPayNCCBC(oPnlAdconal)
241         EndCase
242     Endif
243
244     Endif
245 Endif
246
247 RestArea(aArea)
248 Return .T.
249
250 //-----
251 /*{Protheus.doc} STBPrintBrinde
252 Imprime relatorio gerencial da conferencia de caixa
253
254 @param
255 @return .T.
256 @obs
257 @sample
258 */
259 //-----
260 Function STBPrintBrinde()

```

```

261 Local oModel      := STDGPBModel()
262 Local nQtdItens    := STDPBLength("MGC") // quantidade total de registros na cesta MGC
263 Local nX           := 0 // variavel de controle de loop
264 Local cTexto       := '' // Conteudo do relatorio
265 Local cProd        := "" // Codigo do produto
266 Local nQtd         := 0 // Quantidade de produtos(brindes ou bonificados)
267
268 For nX := 1 to nQtdItens
269     If oModel:GetModel("MGCBRINDE"):acols[nX][13] == .F.
270         If !Empty(oModel:GetModel("MGCBRINDE"):acols[nX][5])
271             nQtd := nQtd + 1
272         Endif
273     Endif
274 Next nX
275
276
277 If nQtd <> 0
278     cTexto := CRLF + CRLF
279     cTexto += STR0008 + '.....: ' + dtoc(oModel:GetModel("MGCBRINDE"):acols[1][6]) + CRLF //"Data"
280     cTexto += STR0009 + '.....: ' + oModel:GetModel("MGCBRINDE"):acols[1][8] + CRLF //"Caixa"
281     cTexto += CRLF + CRLF
282     cTexto += STR0010 + ' |'+ STR0011 + ' ' + CRLF //"Brinde"##"Descrição"
283     cTexto += CRLF
284
285     For nX := 1 to nQtdItens
286         If oModel:GetModel("MGCBRINDE"):acols[nX][13] == .F. // não esta deletado
287             cProd:= oModel:GetModel("MGCBRINDE"):acols[nX][5]
288             cTexto += alltrim(cProd) + '|' + Posicione( "SB1",1,xFilial( "SB1" )+cProd,"SB1->B1_DESC" ) + Space(3) + CRLF
289             cProd:= ""
290         Endif
291     Next nX
292
293     cTexto += CRLF + CRLF
294     cTexto += STR0012 + '.....: ' + AllTrim(str(nQtd)) //"Total"
295
296     //Relatorio da conferencia de caixa
297     STWManagReportPrint( cTexto ,1 )
298 Endif
299
300 Return Nil
301
302
303 //-----
304 /*{Protheus.doc} STIQtdBrindes
305 Atualiza o contador de brindes
306
307 @param
308 @return
309 @obs
310 @sample
311 */
312 //-----

```

```
313 /*
314 Function STIQtdBrindes()
315 Local cRet      := ""                      //Variavel de retorno
316 Local oModel    := STDGPBModel()          // Model de venda
317 Local nQtdItens := If( AliasInDic("MGC"),STDPBLength("MGC"),0) // quantidade total de registros na cesta MGC
318 Local nCont     := 0                      // Contador de brindes ok
319 Local nX        := 0                      // variavel de controle
320
321 For nX := 1 to nQtdItens
322     If oModel:GetModel("MGCBRINDE"):acols[nX][13] == .F. // não esta deletado
323         nCont:= nCont + 1
324     Endif
325 Next nX
326
327 cRet := alltrim(str(nCont)+ ' ' + "Brindes")
328
329 Return cRet
330 /*
```